

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Ứng dụng bài toán Multi-Agent Pathfinding
trong game bắn xe tăng nhiều người chơi**

DƯƠNG QUANG ĐÔNG

Dong.DQ225808@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: ThS. Nguyễn Tiến Thành

Khoa: Khoa học máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Ứng dụng bài toán Multi-Agent Pathfinding
trong game bắn xe tăng nhiều người chơi

DƯƠNG QUANG ĐÔNG

Dong.DQ225808@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: ThS. Nguyễn Tiên Thành

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Trước hết, em xin gửi lời cảm ơn chân thành đến thầy ThS. Nguyễn Tiến Thành, người đã tận tình hướng dẫn và góp ý cho em trong suốt quá trình thực hiện đồ án. Em cũng xin cảm ơn các thầy cô Trường Công nghệ Thông tin và Truyền thông, Đại học Bách khoa Hà Nội đã trang bị cho em nền tảng kiến thức để hoàn thành công việc này.

Em xin cảm ơn gia đình và bạn bè đã luôn ở bên động viên, là chỗ dựa để em kiên trì đến ngày hoàn thành. Và cuối cùng, em cảm ơn chính bản thân mình vì đã không bỏ cuộc trước những lúc khó khăn nhất.

Lấy cảm hứng từ câu nói của Albert Einstein dưới đây, em mong muốn sẽ luôn giữ được niềm say mê khám phá, học hỏi và sáng tạo trên con đường nghiên cứu cũng như sự nghiệp sau này.

“Where the world ceases to be the scene of our personal hopes and wishes, where we face it as free beings admiring, asking, observing, there we enter the realm of art and science.”

“Nơi thế giới không còn là sân khấu cho những ước vọng và mong muốn của chúng ta, nơi chúng ta đối diện với nó như những sinh linh tự do, ngưỡng mộ, hỏi đáp, quan sát, đó là lúc chúng ta bước vào vương quốc của nghệ thuật và khoa học.”

— Albert Einstein

Hà Nội, tháng 6 năm 2026

Sinh viên thực hiện

Dương Quang Đông

TÓM TẮT NỘI DUNG ĐỒ ÁN

Bài toán tìm đường đa tác nhân (Multi-Agent Pathfinding – MAPF) yêu cầu lập kế hoạch đồng thời cho nhiều agent di chuyển từ vị trí xuất phát đến đích mà không va chạm nhau. Trong bối cảnh game chiến thuật thời gian thực, bài toán này càng trở nên khó khăn hơn khi môi trường thay đổi liên tục, đòi hỏi các agent phải tái lập kế hoạch nhanh theo thời gian thực. Các giải pháp tìm đường truyền thống cho agent đơn lẻ như A* không đảm bảo tránh va chạm giữa nhiều agent, trong khi các phương pháp tối ưu toàn cục như CBS đòi hỏi chi phí tính toán quá lớn cho môi trường thời gian thực.

Đồ án này nghiên cứu và triển khai ba cấp độ thuật toán MAPF trong một game bắn xe tăng 2D nhiều người chơi xây dựng trên nền tảng Unity Engine: (1) A* đơn agent làm baseline, (2) PIBT (Priority Inheritance with Backtracking) triển khai trực tiếp trong Unity C# để phối hợp đa agent không va chạm, và (3) PIBT C++/TCP tích hợp máy chủ lập kế hoạch bên ngoài Unity qua kết nối TCP, mở rộng khả năng tận dụng thuật toán hiệu năng cao viết bằng C++. Game sử dụng các bản đồ chuẩn MovingAI MAPF benchmark, đảm bảo kết quả thực nghiệm có thể so sánh với nghiên cứu cộng đồng.

Đồ án đóng góp: (i) hệ thống đọc và dựng bản đồ động từ định dạng .map tại thời điểm chạy; (ii) triển khai A* và PIBT trong C# tích hợp hoàn toàn với vật lý Unity; (iii) giao thức TCP kết nối Unity và máy chủ PIBT C++; (iv) cải tiến mô hình xoay của agent dựa trên EPIBT để loại bỏ hiện tượng xoay giật; và (v) hệ thống backtest tự động với môi trường tĩnh và chương ngại động, xuất kết quả CSV để phân tích định lượng. Thực nghiệm trên 5 bản đồ, 3 thuật toán và 5 mức số lượng agent (6, 12, 24, 36, 72), 6 lần lặp mỗi tổ hợp trên cả môi trường tĩnh và động (tổng 900 run) cho thấy cả ba thuật toán đều hoàn thành nhiệm vụ trên 4/5 bản đồ ở mức cơ bản; PIBT có số lần tái hoạch định nhỉnh hơn A* đôi chút do cơ chế phối hợp đa agent; tỉ lệ phá Eagle đạt đỉnh ở dải mật độ trung gian (24–36 agent) rồi giảm khi mật độ quá cao; và khi bật chương ngại động, hệ thống vẫn duy trì được mục tiêu, với số lần phục hồi sau kẹt tăng rõ nhất ở PIBT.

Sinh viên thực hiện
Dương Quang Đông

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài	2
1.3 Định hướng giải pháp.....	3
1.4 Bố cục đồ án	4
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU	5
2.1 Khảo sát hiện trạng	5
2.1.1 Các giải pháp điều hướng trong game hiện tại.....	5
2.1.2 Khảo sát bản đồ benchmark MAPF.....	6
2.2 Tổng quan chức năng.....	6
2.2.1 Tác nhân hệ thống.....	6
2.2.2 Biểu đồ use case tổng quan	7
2.3 Đặc tả use case chi tiết	7
2.3.1 Nhóm A – Chơi đơn.....	7
2.3.2 Nhóm B – Chơi nhiều người	10
2.3.3 Nhóm C – Tạm dừng và điều hướng phiên chơi	16
2.3.4 Nhóm D – Backtest (chỉ bản local/Editor)	17
2.4 Yêu cầu phi chức năng.....	20
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG..	22
3.1 Bài toán MAPF	22
3.2 Thuật toán A*	22
3.3 Thuật toán PIBT	23
3.4 Unity Engine và C#	25
3.5 Giao tiếp TCP giữa Unity và máy chủ C++	25

3.6 Các tiêu chí đánh giá thuật toán điều hướng	26
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	27
4.1 Thiết kế kiến trúc.....	27
4.1.1 Lựa chọn kiến trúc phần mềm	27
4.1.2 Thiết kế tổng quan	27
4.2 Thiết kế chi tiết	28
4.2.1 Thiết kế giao diện	28
4.2.2 Thiết kế các lớp MAPF chính.....	29
4.2.3 Thiết kế cơ sở dữ liệu	31
4.2.4 Thiết kế luồng mạng nhiều người chơi	32
4.3 Xây dựng ứng dụng	34
4.3.1 Công cụ và thư viện sử dụng	34
4.3.2 Kết quả đạt được.....	34
4.3.3 Minh họa các chức năng chính	35
4.4 Kiểm thử và đánh giá.....	40
4.4.1 Cấu hình thực nghiệm	40
4.4.2 Ba thuật toán điều hướng và cách đo số lần tái hoạch định.....	42
4.4.3 Kết quả thực nghiệm – Môi trường tĩnh	44
4.4.4 Kết quả thực nghiệm – Môi trường có chướng ngại động.....	45
4.4.5 Khảo sát khả năng mở rộng theo số lượng agent	48
4.4.6 Nhận xét và đánh giá.....	52
4.5 Triển khai	53
4.5.1 Triển khai cục bộ	54
4.5.2 Hạ tầng triển khai trên Google Cloud Platform	54
4.5.3 Tên miền và chứng chỉ TLS.....	55

4.5.4 Nginx – Reverse proxy và phục vụ WebGL	55
4.5.5 Máy chủ PIBT C++ và mô hình client-server TCP	56
4.5.6 Tích hợp liên tục và triển khai liên tục (CI/CD)	56
4.5.7 Sơ đồ triển khai tổng thể.....	57
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	59
5.1 Hệ thống đọc bản đồ động và dựng lưới thống nhất	59
5.2 Triển khai A* thời gian thực với tái hoạch định định kỳ	59
5.3 Tích hợp PIBT phối hợp đa agent không va chạm.....	61
5.4 Cầu nối TCP đến máy chủ PIBT C++	62
5.5 Cải tiến mô hình xoay của Enemy Agent dựa trên EPIBT.....	63
5.6 Hệ thống backtest tự động với chương ngại động	65
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	67
6.1 Kết luận.....	67
6.2 Hạn chế của đồ án.....	67
6.3 Hướng phát triển	68
TÀI LIỆU THAM KHẢO	69
PHỤ LỤC	71
A. KẾT QUẢ BACKTEST CHI TIẾT	71
A.1 Kết quả môi trường tĩnh	71
A.2 Kết quả môi trường có chương ngại động.....	72
B. NHÓM USE CASE TRIỂN KHAI VÀ VẬN HÀNH.....	73
B.1 Đặc tả use case UC-E1 – Triển khai PIBT/Relay lên GCP	73
B.2 Đặc tả use case UC-E2 – Chạy dedicated server.....	74
B.3 Tóm tắt quy trình triển khai và vận hành	74

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quan của hệ thống	7
Hình 2.2	Biểu đồ use case nhóm A – Chơi đơn	8
Hình 2.3	Use case UC-A1 – Cấu hình & bắt đầu ván chơi đơn	8
Hình 2.4	Use case UC-A2 – Điều khiển xe tăng	9
Hình 2.5	Use case UC-A3 – Kết thúc ván chơi	10
Hình 2.6	Biểu đồ use case nhóm B – Chơi nhiều người (LAN)	11
Hình 2.7	Biểu đồ use case nhóm B – Chơi nhiều người (Internet)	11
Hình 2.8	Use case UC-B1 – Tạo phòng LAN	11
Hình 2.9	Use case UC-B2 – Tham gia phòng LAN	12
Hình 2.10	Use case UC-B3 – Tạo phòng Internet	12
Hình 2.11	Use case UC-B4 – Tham gia phòng Internet	13
Hình 2.12	Use case UC-B5 – Phòng chờ & bắt đầu trận	14
Hình 2.13	Use case UC-B6 – Chơi mạng nhiều người	15
Hình 2.14	Use case UC-B7 – Rời phòng / Hủy kết nối	15
Hình 2.15	Biểu đồ use case nhóm C – Tạm dừng và điều hướng	16
Hình 2.16	Use case UC-C1 – Tạm dừng / Tiếp tục ván	16
Hình 2.17	Use case UC-C2 – Quay lại menu chính	17
Hình 2.18	Biểu đồ use case nhóm D – Backtest (local/Editor)	18
Hình 2.19	Use case UC-D1 – Cấu hình backtest	18
Hình 2.20	Use case UC-D2 – Chạy backtest tự động	18
Hình 2.21	Use case UC-D3 – Xem kết quả backtest	19
Hình 4.1	Biểu đồ gói UML của hệ thống Unity MAPF	28
Hình 4.2	Sơ đồ hoạt động luồng giao diện chế độ chơi đơn đã triển khai	29
Hình 4.3	Biểu đồ lớp các thành phần tìm đường và điều phối agent . .	30
Hình 4.4	Luồng dữ liệu MAPF: từ tệp bản đồ đến chuyển động agent	31
Hình 4.5	Sơ đồ hoạt động luồng Host/Join nhiều người chơi qua Internet	32
Hình 4.6	Biểu đồ trình tự tạo phòng chơi nhiều người qua Internet . .	33
Hình 4.7	Biểu đồ trình tự sẵn sàng và bắt đầu ván chơi nhiều người qua Internet	34
Hình 4.8	Màn hình menu chính với hai chế độ SINGLE PLAY và MULTIPLAYER INTERNET	36
Hình 4.9	Chế độ chơi đơn – màn chọn mẫu/màu xe tăng	36
Hình 4.10	Chế độ chơi đơn – màn chọn bản đồ và thuật toán AI	37
Hình 4.11	Một trận chơi đơn đang diễn ra	37

Hình 4.12	Chế độ nhiều người – chọn vai trò Host/Join	38
Hình 4.13	Chế độ nhiều người – chọn bản đồ và số agent đối thủ	39
Hình 4.14	Phòng chờ nhiều người phía chủ phòng (Host)	39
Hình 4.15	Một trận nhiều người đang diễn ra	40
Hình 4.16	Hai màn hình kết thúc trận của chế độ chơi đơn	40
Hình 4.17	Quy trình thực nghiệm backtest tự động	41
Hình 4.18	Giao diện cấu hình backtest: chọn bản đồ, số lần chạy, số lượng agent và bật/tắt chương ngại động	42
Hình 4.19	Thời gian hoàn thành trung bình theo bản đồ (môi trường tĩnh)	45
Hình 4.20	Tổng số lần tái hoạch định trung bình theo bản đồ, thang log- arit (môi trường tĩnh)	45
Hình 4.21	Backtest A* trên Alpha32: tắt (trái, chỉ thùng tĩnh màu tím) và bật (phải, xuất hiện thêm khối kim loại động màu đen) chế độ chương ngại động	46
Hình 4.22	Một lượt backtest A* với chương ngại động trên Alpha32: bảng chỉ số bên phải hiển thị số lần replan và máu Eagle theo thời gian	47
Hình 4.23	So sánh số lần tái hoạch định giữa môi trường tĩnh và có chương ngại động	47
Hình 4.24	Thời gian hoàn thành trung bình theo số lượng agent (môi trường tĩnh)	49
Hình 4.25	Tổng số lần tái hoạch định trung bình theo số lượng agent, thang logarit (môi trường tĩnh)	49
Hình 4.26	Tỉ lệ timeout theo số lượng agent (môi trường tĩnh)	50
Hình 4.27	Số lần phục hồi sau kẹt theo số lượng agent (môi trường động, thang logarit)	51
Hình 4.28	Sơ đồ triển khai hệ thống trên GCP với Nginx, Registry và các dịch vụ trò chơi	58
Hình 5.1	Biểu đồ trình tự chu kỳ tái hoạch định của A*	60
Hình 5.2	Biểu đồ trình tự một tick lập kế hoạch của PIBT C#	61
Hình 5.3	Biểu đồ trình tự trao đổi thông điệp PIBT qua TCP	62
Hình 5.4	Lộ trình chuyển từ PIBT sang EPIBT cho Enemy Agent: khái niệm trong bài báo (vấn đề xoay hướng), phản thích ứng cho game (multi-action operation, kế thừa operation) và phân tích hợp máy chủ (phiên lập kế hoạch, trả về hành động đầu tiên)	63
Hình B.1	Biểu đồ use case nhóm E – Triển khai & vận hành	73

Hình B.2	Use case UC-E1 – Triển khai PIBT/Relay lên GCP	73
Hình B.3	Use case UC-E2 – Chạy dedicated server	74

DANH MỤC BẢNG BIỂU

Bảng 2.1	So sánh các giải pháp điều hướng đa agent	5
Bảng 2.2	Năm bản đồ benchmark sử dụng trong đồ án	6
Bảng 2.3	Đặc tả use case UC-A1 – Cấu hình & bắt đầu ván chơi đơn .	8
Bảng 2.4	Đặc tả use case UC-A2 – Điều khiển xe tăng	9
Bảng 2.5	Đặc tả use case UC-A3 – Kết thúc ván chơi	10
Bảng 2.6	Đặc tả use case UC-B1 – Tạo phòng LAN	11
Bảng 2.7	Đặc tả use case UC-B2 – Tham gia phòng LAN	12
Bảng 2.8	Đặc tả use case UC-B3 – Tạo phòng Internet	13
Bảng 2.9	Đặc tả use case UC-B4 – Tham gia phòng Internet	13
Bảng 2.10	Đặc tả use case UC-B5 – Phòng chờ & bắt đầu trận	14
Bảng 2.11	Đặc tả use case UC-B6 – Chơi mạng nhiều người	15
Bảng 2.12	Đặc tả use case UC-B7 – Rời phòng / Huỷ kết nối	16
Bảng 2.13	Đặc tả use case UC-C1 – Tạm dừng / Tiếp tục ván	16
Bảng 2.14	Đặc tả use case UC-C2 – Quay lại menu chính	17
Bảng 2.15	Đặc tả use case UC-D1 – Cấu hình backtest	18
Bảng 2.16	Đặc tả use case UC-D2 – Chạy backtest tự động	19
Bảng 2.17	Đặc tả use case UC-D3 – Xem kết quả backtest	19
Bảng 3.1	Số ô và số chuỗi ô đạt được theo độ dài thao tác	24
Bảng 4.1	Cấu trúc trường của hai tệp CSV kết quả backtest	31
Bảng 4.2	Công cụ và thư viện sử dụng	34
Bảng 4.3	Thống kê mã nguồn C# (tháng 6/2026)	35
Bảng 4.4	Cấu hình thực nghiệm	41
Bảng 4.5	Cơ chế phát sinh lệnh tái hoạch định của ba thuật toán . . .	43
Bảng 4.6	Số lần tái hoạch định của từng agent trong một lượt chạy trên bản đồ Mansion (6 agent)	43
Bảng 4.7	Kết quả thực nghiệm môi trường tĩnh (trung bình 6 lần lặp, 6 agent)	44
Bảng 4.8	Kết quả thực nghiệm môi trường có chướng ngại động (trung bình 6 lần lặp, 6 agent)	46
Bảng 4.9	Kết quả theo số lượng agent, môi trường tĩnh (trung bình trên 5 bản đồ $\times$ 6 lần lặp)	48
Bảng 4.10	Kết quả theo số lượng agent, môi trường có chướng ngại động (trung bình trên 5 bản đồ $\times$ 6 lần lặp)	50

Bảng 4.11	Số lần tái hoạch định theo bản đồ và số lượng agent (môi trường tĩnh); † = lượt timeout	52
Bảng 4.12	Thành phần hạ tầng GCP	54
Bảng 4.13	Ba khối cấu hình Nginx trên máy ảo GCP	56
Bảng 4.14	Các workflow CI/CD trên GitHub Actions	57
Bảng A.1	Kết quả backtest chi tiết – môi trường tĩnh ($n = 6$)	71
Bảng A.2	Kết quả backtest chi tiết – môi trường có chướng ngại động ($n = 6$)	72
Bảng B.1	Đặc tả use case UC-E1 – Triển khai PIBT/Relay lên GCP	73
Bảng B.2	Đặc tả use case UC-E2 – Chạy dedicated server	74
Bảng B.3	Năm giai đoạn của quy trình triển khai end-to-end	75

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
AI	Trí tuệ nhân tạo (Artificial Intelligence)
CBS	Tìm kiếm dựa trên xung đột (Conflict-Based Search)
CSV	Tệp giá trị phân tách bằng dấu phẩy (Comma-Separated Values)
FPS	Số khung hình trên giây (Frames Per Second)
HUD	Bảng thông tin trên màn hình (Heads-Up Display)
LAN	Mạng cục bộ (Local Area Network)
LNS2	Tìm kiếm lân cận lớn lần 2 (Large Neighborhood Search 2)
LOS	Đường ngắm thẳng (Line of Sight)
MAPF	Tìm đường đa tác nhân (Multi-Agent Pathfinding)
PIBT	Kế thừa ưu tiên có hoàn tác (Priority Inheritance with Backtracking)
TCP	Giao thức điều khiển truyền vận (Transmission Control Protocol)
URP	Quy trình kết xuất đa năng của Unity (Universal Render Pipeline)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

Chương 1 giới thiệu bối cảnh, động lực và phạm vi của đề án. Từ bài toán điều hướng đa tác nhân trong game, chương này xác định mục tiêu cụ thể, đề xuất định hướng giải pháp và trình bày tổng quan cấu trúc báo cáo.

1.1 Đặt vấn đề

Trong các trò chơi điện tử chiến thuật thời gian thực (Real-Time Strategy – RTS), điều hướng đồng thời nhiều nhân vật AI đến mục tiêu là một thách thức kỹ thuật cốt lõi. Khi số lượng nhân vật AI tăng lên, xác suất xảy ra va chạm, tình trạng deadlock và đường đi không hiệu quả cũng tăng theo. Đây chính là bài toán MAPF Multi-Agent Pathfinding – được nghiên cứu rộng rãi trong lĩnh vực trí tuệ nhân tạo và robot học [1].

MAPF yêu cầu lập kế hoạch đồng thời cho k agent trên đồ thị lưới để đảm bảo mỗi agent đến đích trong thời gian tối thiểu mà không va chạm nhau tại bất kỳ bước thời gian nào. Trong môi trường game thời gian thực, bài toán này đặt ra thêm ràng buộc về thời gian tính toán: thuật toán phải phản hồi gần như tức thời – A^* cỡ mili-giây mỗi khung hình, còn bộ lập kế hoạch phối hợp đa agent trong dưới một giây mỗi bước – và phải tái lập kế hoạch liên tục khi môi trường thay đổi.

Các thuật toán tìm đường agent đơn như A^* [2] được sử dụng phổ biến trong game nhờ độ đơn giản và hiệu quả, nhưng khi áp dụng độc lập cho nhiều agent, chúng không đảm bảo tránh va chạm. Các thuật toán MAPF tối ưu như CBS [3] đảm bảo tính tối ưu toàn cục nhưng có độ phức tạp tính toán tăng theo hàm mũ với số agent, không phù hợp cho game thời gian thực. PIBT (Priority Inheritance with Backtracking) [4] là một hướng tiếp cận cân bằng: chạy theo bước thời gian, hoàn chỉnh và hiệu quả cho môi trường thời gian thực, song chấp nhận thay thế lời giải tối ưu toàn cục bằng lời giải khả thi đủ tốt theo từng bước thời gian để đạt tốc độ phản hồi thời gian thực.

PIBT trong đề án được tham khảo từ cuộc thi điều phối robot The League of Robot Runners (LoRR) [5] do Amazon Robotics tài trợ, nơi nhóm Team No Man's Sky không chỉ giành giải nhất ở một hạng mục mà còn đứng nhất cả ba bảng đấu Combined, Planner và Scheduler – đồng thời đoạt giải Line Honours (nhất toàn đoàn với số lời giải tốt nhất nhiều nhất) trong Main Round 2024. Tuy nhiên, các cài đặt đoạt giải đều ở dạng máy chủ C++ chạy benchmark, không thể chơi hay tương tác trực tiếp. Đề án kế thừa ý tưởng và thuật toán của Team No Man's Sky rồi đưa vào Unity, để người chơi có thể trực tiếp trải nghiệm và quan sát hành vi

phối hợp đa agent một cách trực quan ngay trong game.

Độ khó của bài toán đến từ hai phía. Về quy mô, các lời giải dẫn đầu ở LoRR phải điều phối tới hàng nghìn, thậm chí hơn mười nghìn agent và trả lời trong ngân sách thời gian chặt chẽ mỗi bước; tuy game của đồ án chạy ở quy mô nhỏ hơn nhiều, việc khảo sát từ 6 đến 72 agent đã đủ để bộc lộ ảnh hưởng của mật độ agent tới hiệu năng, kể cả hiện tượng tồn tại một dải mật độ tối ưu. Về tích hợp, đưa một bộ lập kế hoạch MAPF vào engine vật lý thời gian thực đặt ra ràng buộc mà benchmark thuần túy không có: agent là xe tăng có hướng thân, phải xoay trước khi tiến, nên lời giải trên lưới phải được chuyển thành chuyển động vật lý mượt mà – ràng buộc xoay này về sau được xử lý bằng cải tiến EPIBT ở Mục 5.5.

Thực tế trong ngành game, phần lớn các tựa game RTS và tower-defense hiện tại sử dụng thuật toán tìm đường đơn giản hoặc Unity NavMesh mà không giải quyết bài toán va chạm đa agent ở lớp lập kế hoạch. Điều này dẫn đến các hiện tượng nhân vật xếp hàng chờ hoặc chen lấn không tự nhiên, làm giảm chất lượng trải nghiệm người chơi.

Đồ án này xuất phát từ nhu cầu nghiên cứu và so sánh thực nghiệm ba cấp độ thuật toán MAPF – từ baseline A* đến PIBT phối hợp đa agent và PIBT tích hợp máy chủ C++ bên ngoài – trong bối cảnh game bắn xe tăng 2D nhiều người chơi xây dựng trên Unity Engine. Kết quả sẽ cung cấp bằng chứng thực nghiệm về ưu nhược điểm của từng hướng tiếp cận trong môi trường game thực tế.

1.2 Mục tiêu và phạm vi đề tài

Các sản phẩm và nghiên cứu liên quan hiện tại có thể phân thành ba nhóm. Nhóm thứ nhất là các game thương mại sử dụng tìm đường đơn giản: phần lớn game RTS và tower-defense dùng A* hoặc Unity NavMesh cho từng agent độc lập, không giải quyết va chạm đa agent ở lớp lập kế hoạch, dẫn đến hiện tượng nhân vật xếp hàng hoặc chen lấn không tự nhiên. Nhóm thứ hai là các nghiên cứu MAPF học thuật như CBS, EECBS hay PIBT, được nghiên cứu sâu trong cộng đồng trí tuệ nhân tạo nhưng thường chỉ được đánh giá trên benchmark thuần túy mà không tích hợp trực tiếp vào engine game thương mại. Nhóm thứ ba là một số nghiên cứu thử tích hợp MAPF vào game [6], song thường chỉ chạy trên môi trường giả lập đơn giản, thiếu một engine vật lý đầy đủ.

Từ ba nhóm trên, có thể thấy khoảng trống hiện tại là thiếu một so sánh thực nghiệm đầy đủ giữa các cấp độ MAPF trong môi trường game có vật lý thực tế (Unity physics) trên tập bản đồ benchmark chuẩn. Đây chính là khoảng trống mà đồ án hướng tới lấp đầy.

Trên cơ sở đó, đề án đặt ra bốn mục tiêu cụ thể. Thứ nhất, xây dựng một game bắn xe tăng 2D nhiều người chơi trên Unity, sử dụng bản đồ chuẩn MovingAI MAPF benchmark. Thứ hai, triển khai và tích hợp ba cấp độ thuật toán MAPF – A* làm baseline, PIBT C# và PIBT C++/TCP – chia sẻ cùng tầng điều khiển vật lý để có thể so sánh khách quan. Thứ ba, xây dựng hệ thống backtest tự động đo lường định lượng trên năm bản đồ, kèm chế độ chương ngại động để kiểm tra khả năng tái hoạch định. Thứ tư, phân tích, so sánh kết quả và rút ra nhận xét về khả năng ứng dụng của từng thuật toán trong game thời gian thực.

Về phạm vi, đề án giới hạn ở game 2D góc nhìn từ trên xuống, môi trường lưới rời rạc (discrete grid), khảo sát từ 6 đến 72 agent AI (năm mức: 6, 12, 24, 36, 72) trong kịch bản thực nghiệm, và không xét tối ưu hóa đa mục tiêu phức tạp.

1.3 Định hướng giải pháp

Từ phân tích ở Mục 1.2, đề án đề xuất ba bước triển khai:

Thứ nhất, xây dựng hạ tầng bản đồ thống nhất dựa trên đọc tệp .map MovingAI và dựng lưới ô tại thời điểm chạy trong Unity, thay vì bake cứng vào scene. Cách này cho phép thử nghiệm trên nhiều bản đồ mà không cần thay đổi code.

Thứ hai, triển khai tuần tự ba thuật toán MAPF chia sẻ cùng tầng điều khiển vật lý (TankController), đảm bảo hành vi vật lý nhất quán khi so sánh thuật toán. A* đóng vai trò baseline; PIBT C# thêm phối hợp đa agent ngay trong Unity; PIBT C++/TCP mở rộng tích hợp máy chủ ngoài.

Thứ ba, xây dựng hệ thống backtest tự động chạy kịch bản, thu thập chỉ số và xuất CSV, cho phép so sánh định lượng trên tập bản đồ lớn mà không cần can thiệp thủ công. Thêm vào đó, module chương ngại động (spawn/despawn thùng cản đường trực tiếp trên path của agent) dùng để stress-test khả năng tái hoạch định của từng thuật toán.

Một nguyên tắc xuyên suốt ba bước trên là bảo đảm tính công bằng khi so sánh: cả ba thuật toán dùng chung một bộ nạp bản đồ, chung tầng điều khiển vật lý TankController và chung bộ chỉ số đo lường, nên khác biệt quan sát được phản ánh đúng bản chất thuật toán chứ không phải do chênh lệch hạ tầng. Nhờ vậy, kết luận rút ra từ thực nghiệm có độ tin cậy cao và có thể tái lập.

Ngoài ba bước trên, thiết kế hệ thống còn hướng tới khả năng mở rộng quy mô về sau trên ba trục: tăng số lượng agent AI, tăng số người chơi đồng thời, và mở rộng hạ tầng máy chủ (đưa PIBT server lên cloud, cân bằng tải). Các hướng mở rộng này được trình bày chi tiết ở Chương 6.

Đóng góp chính: hệ thống tích hợp MAPF đầy đủ trong Unity với ba cấp độ

thuật toán, cải tiến mô hình xoay của Enemy Agent dựa trên EPIBT, cơ sở hạ tầng backtest định lượng, và bộ kết quả thực nghiệm trên benchmark chuẩn.

1.4 Bố cục đề án

Phần còn lại của báo cáo đề án tốt nghiệp này được tổ chức như sau.

Chương 2 trình bày khảo sát hiện trạng các giải pháp điều hướng đa agent trong game, phân tích yêu cầu chức năng và phi chức năng của hệ thống, đặc tả các use case chính thông qua biểu đồ use case và đặc tả luồng sự kiện.

Chương 3 trình bày nền tảng lý thuyết và công nghệ được sử dụng: bài toán MAPF và cách biểu diễn, thuật toán A* và heuristic Manhattan, thuật toán PIBT và cơ chế kế thừa ưu tiên có hoàn tác, cùng với lý do lựa chọn Unity Engine và giao thức TCP cho tích hợp máy chủ ngoài.

Chương 4 trình bày phân tích thiết kế kiến trúc hệ thống theo mô hình phân tầng, thiết kế chi tiết các lớp MAPF AI, quá trình xây dựng ứng dụng với thống kê mã nguồn, và đánh giá thực nghiệm với số liệu backtest trên 900 run gồm môi trường tĩnh và môi trường có chướng ngại động (năm mức agent từ 6 đến 72).

Chương 5 trình bày sáu đóng góp kỹ thuật nổi bật: hệ thống đọc bản đồ động, triển khai A* thời gian thực, tích hợp PIBT phối hợp đa agent, cầu nối TCP đến máy chủ C++, cải tiến mô hình xoay của Enemy Agent dựa trên EPIBT, và hệ thống backtest tự động với chướng ngại động.

Chương 6 tổng kết kết quả đã đạt được, so sánh với mục tiêu đề ra, và định hướng phát triển trong tương lai.

Kết chương 1: Chương này đã xác định bài toán MAPF trong game thời gian thực, nêu rõ khoảng trống nghiên cứu và đề xuất hướng giải quyết bằng cách tích hợp và so sánh ba thuật toán (A*, PIBT C#, PIBT C++/TCP) trong môi trường Unity. Các chương tiếp theo sẽ trình bày chi tiết quá trình phân tích, thiết kế, triển khai và đánh giá hệ thống.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương 1 đã xác định bài toán MAPF và đề xuất định hướng giải pháp. Chương này phân tích chi tiết hiện trạng các giải pháp điều hướng đa agent trong game, từ đó xác định yêu cầu chức năng và phi chức năng của hệ thống cần xây dựng.

2.1 Khảo sát hiện trạng

2.1.1 Các giải pháp điều hướng trong game hiện tại

Các giải pháp điều hướng agent trong game thương mại và nghiên cứu học thuật có thể phân thành bốn nhóm chính:

(1) Unity NavMesh: Unity Engine cung cấp hệ thống dẫn đường tự động dựa trên navigation mesh 3D. NavMesh tính toán đường đi cho từng agent độc lập và hỗ trợ tránh va chạm theo thời gian thực qua *NavMeshAgent*. Ưu điểm: tích hợp sẵn, dễ sử dụng, hỗ trợ môi trường 3D phức tạp. Nhược điểm: không giải quyết bài toán MAPF (không đảm bảo tránh va chạm ở lớp lập kế hoạch cho môi trường lưới rời rạc), không tương thích với benchmark .map chuẩn MAPF.

(2) A* đơn agent: Thuật toán A* [2] với heuristic Manhattan trên lưới ô vuông là giải pháp phổ biến nhất trong game 2D. Mỗi agent tìm đường độc lập mà không biết kế hoạch của agent khác. Ưu điểm: đơn giản, hiệu quả, dễ triển khai. Nhược điểm: va chạm giữa các agent phải xử lý ở lớp vật lý, không có phối hợp ở lớp lập kế hoạch, hiệu năng suy giảm khi số agent lớn do xử lý va chạm phản ứng.

(3) CBS và thuật toán MAPF tối ưu: Conflict-Based Search [3] đảm bảo giải pháp tối ưu toàn cục (tổng số bước di chuyển nhỏ nhất). Ưu điểm: tối ưu, hoàn chỉnh. Nhược điểm: độ phức tạp tính toán tăng theo hàm mũ với số agent và mật độ va chạm, không thực tế cho game thời gian thực với nhiều agent.

(4) PIBT: Priority Inheritance with Backtracking [4] là thuật toán MAPF chạy theo bước thời gian, hoàn chỉnh trên bản đồ có đường đi tồn tại, với độ phức tạp thực tế phù hợp cho môi trường thời gian thực. PIBT không đảm bảo tối ưu toàn cục nhưng thực hành cho kết quả tốt và đã được dùng trong các cuộc thi điều phối robot quốc tế [5].

Bảng 2.1 tổng hợp so sánh bốn giải pháp theo các tiêu chí phù hợp với yêu cầu của đề án.

Bảng 2.1: So sánh các giải pháp điều hướng đa agent

Tiêu chí	NavMesh	A* đơn	CBS	PIBT
Tránh va chạm đa agent	Vật lý	Vật lý	Lập kế hoạch	Lập kế hoạch
Tính tối ưu	Không	Không	Có	Không
Thời gian thực	Có	Có	Không	Có
Tương thích bản đồ .map	Không	Có	Có	Có
Tích hợp Unity	Có sẵn	Tự triển khai	Tự triển khai	Tự triển khai
Hỗ trợ ràng buộc xoay	Không	Không	Không	Có (EPIBT)
Phù hợp đồ án	Không	Baseline	Không	Chính

Từ phân tích trên, đồ án lựa chọn A* làm baseline để có điểm so sánh quen thuộc, và PIBT làm thuật toán chính vì cân bằng tốt giữa hiệu quả và khả năng thực thi thời gian thực.

Một khoảng trống đáng chú ý là phần lớn nghiên cứu MAPF học thuật (CBS, PIBT và các biến thể) được đánh giá trên benchmark thuần túy ở dạng lưới trừu tượng, nơi agent dịch chuyển tức thời giữa các ô và không chịu ràng buộc của một engine vật lý. Khi đưa vào game thực tế, agent là vật thể có quán tính, có hướng thân và va chạm vật lý, nên một lời giải hợp lệ trên lưới vẫn có thể không thực thi chính xác ở lớp vật lý (xoay giật, kẹt do quán tính). Đồ án lấp khoảng trống này bằng cách cho cả ba thuật toán chạy trên cùng một tầng điều khiển vật lý của Unity (TankController) và bổ sung mô hình hành động có xoay cho PIBT, nhờ đó kết quả so sánh phản ánh đúng hành vi trong môi trường game.

2.1.2 Khảo sát bản đồ benchmark MAPF

MovingAI MAPF benchmark [7] cung cấp hàng nghìn bản đồ với định dạng .map chuẩn, phân loại theo môi trường (mê cung, trong nhà, game). Đồ án sử dụng 5 bản đồ đại diện cho 5 kịch bản điển hình (Bảng 2.2).

Bảng 2.2: Năm bản đồ benchmark sử dụng trong đồ án

Nhãn	Tệp bản đồ	Kích thước	Ô đi được	Đặc điểm
Alpha32	random-32-32-10.map	32×32	922 (90%)	Thoảng, ít tường
Mansion	ht_mansion_n.map	133×270	8959 (25%)	Hành lang hẹp
Chantry	ht_chantry.map	162×141	7461 (33%)	Phòng kín
Gallows	lt_gallowstemplar_n.map	251×180	10021 (22%)	Vùng hẹp nhiều
Maze128	maze-128-128-10.map	128×128	14818 (90%)	Mê cung lớn

2.2 Tổng quan chức năng

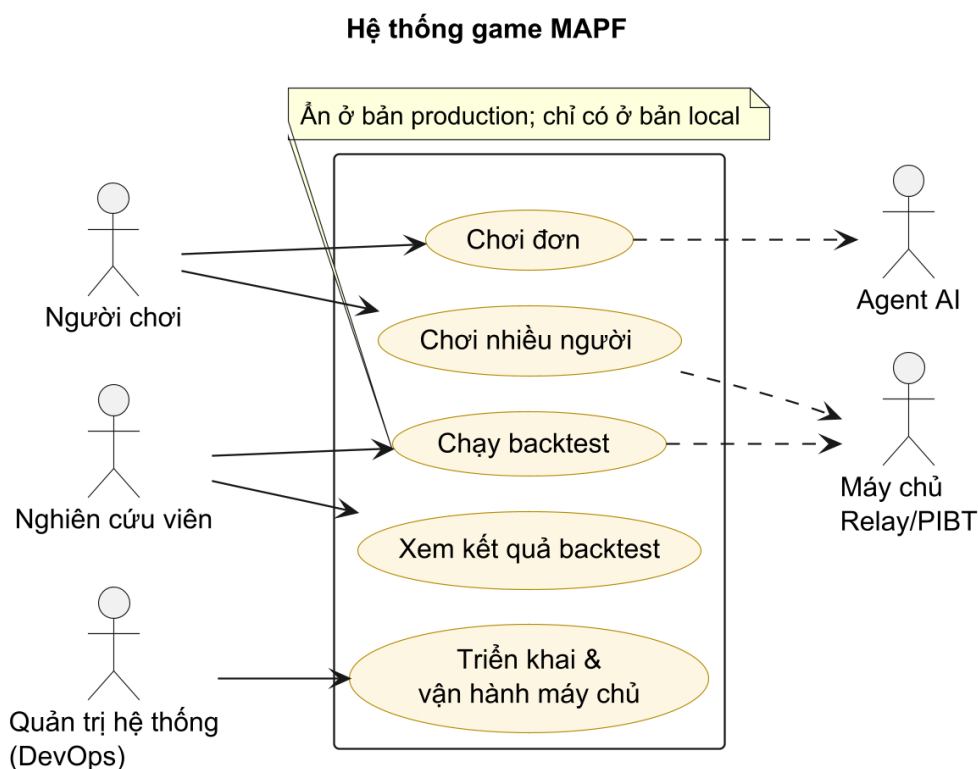
2.2.1 Tác nhân hệ thống

Hệ thống tương tác với năm tác nhân. **Người chơi** (Player) trực tiếp điều khiển xe tăng, chọn chế độ chơi và cấu hình trước trận; trong chế độ nhiều người, người chơi đảm nhận một trong hai vai **Chủ phòng** (Host) hoặc **Người tham gia** (Client). **Nghiên cứu viên** (Researcher) cấu hình, chạy và xem kết quả backtest – chức năng

chỉ tồn tại trong bản chạy local/Editor và bị ẩn ở bản production. **Quản trị hệ thống** (DevOps) triển khai và vận hành máy chủ relay/PIBT trên hạ tầng đám mây. Hai tác nhân hệ thống là **Agent AI** (Enemy) – do thuật toán MAPF điều khiển để tấn công Eagle Base, và **Máy chủ PIBT** (C++ qua TCP hoặc Relay đám mây) – thực thi thuật toán PIBT bên ngoài Unity.

2.2.2 Biểu đồ use case tổng quan

Hình 2.1 trình bày biểu đồ use case tổng quan với năm nhóm chức năng mức cao. Các nhóm này được bóc tách thành use case chi tiết ở Mục 2.3 (nhóm A, B, C, D); nhóm E (triển khai & vận hành) đặt ở Phụ lục B.



Hình 2.1: Biểu đồ use case tổng quan của hệ thống

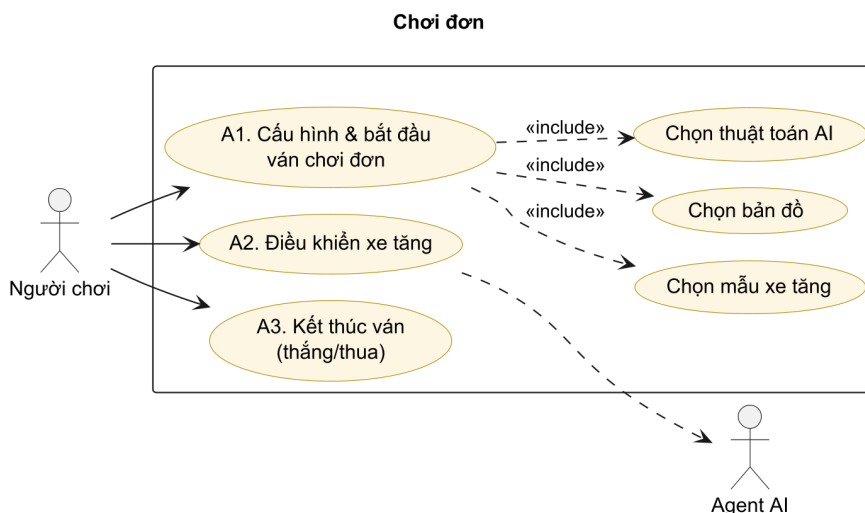
2.3 Đặc tả use case chi tiết

Use case được đánh số theo nhóm: A (chơi đơn), B (chơi nhiều người), C (tạm dừng và điều hướng phiên chơi), D (backtest). Nhóm E (triển khai & vận hành) đặt ở Phụ lục B. Mỗi use case kèm một biểu đồ thu gọn và một bảng đặc tả theo mẫu chuẩn gồm: mã, tên, tác nhân, mô tả, tiền điều kiện, luồng sự kiện chính, luồng thay thế/ngoại lệ và hậu điều kiện.

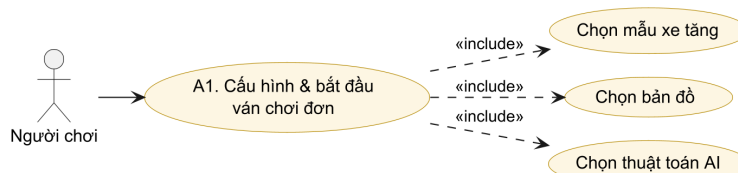
2.3.1 Nhóm A – Chơi đơn

Người chơi vào chế độ chơi đơn bằng nút **SINGLE PLAY** ở menu chính. Chế độ này được thiết kế nhằm cung cấp trải nghiệm độc lập, cho phép người chơi tham gia trò chơi mà không cần kết nối mạng hay tương tác với người chơi khác.

Hình 2.2 minh họa biểu đồ use case tổng quan cho nhóm chức năng này, bao gồm ba use case chính nối tiếp nhau thành một luồng hoàn chỉnh. Cụ thể, quy trình bắt đầu bằng việc người chơi thiết lập các thông số cấu hình bản đồ, độ khó, thuật toán AI và khởi tạo ván đấu mới (A1). Trong quá trình chơi, người chơi sẽ thao tác trực tiếp để điều khiển xe tăng di chuyển, né tránh và bắn đạn tiêu diệt các xe tăng địch do hệ thống điều khiển (A2). Cuối cùng, khi thỏa mãn điều kiện kết thúc (người chơi bị tiêu diệt, cờ đại bàng bị phá hủy hoặc tiêu diệt hết địch), hệ thống sẽ tính toán kết quả và khép lại ván đấu (A3).



Hình 2.2: Biểu đồ use case nhóm A – Chơi đơn

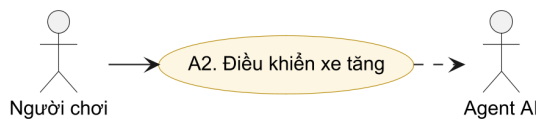


Hình 2.3: Use case UC-A1 – Cấu hình & bắt đầu ván chơi đơn

Bảng 2.3: Đặc tả use case UC-A1 – Cấu hình & bắt đầu ván chơi đơn

Mã use case	UC-A1
Tên use case	Cấu hình và bắt đầu ván chơi đơn
Tác nhân chính	Người chơi
Mô tả	Người chơi chọn mẫu xe tăng, bản đồ và thuật toán AI rồi khởi động một ván chơi đơn chống các agent AI.
Tiền điều kiện	Ứng dụng đang ở menu chính.

Luồng sự kiện chính	1. Từ menu chính chọn SINGLE PLAY. 2. Chọn mẫu xe tăng. 3. Chọn bản đồ .map. 4. Chọn thuật toán AI (A*, PIBT hoặc PIBT_TCP). 5. Bấm bắt đầu. 6. Hệ thống đọc tệp .map, dựng lưới ô và spawn xe tăng người chơi cùng Eagle Base. 7. MapScenarioBootstrap spawn các agent AI theo thuật toán đã chọn.
Luồng thay thế / ngoại lệ	2a. Không chọn thủ công → dùng lựa chọn đã lưu (PlayerPrefs) hoặc mặc định Alpha32/A*. 4a. Chọn PIBT_TCP nhưng máy chủ C++ chưa chạy → báo lỗi kết nối. 6a. Tệp .map không tìm thấy → dùng Alpha32.
Hậu điều kiện	Ván chơi đơn bắt đầu; agent AI dùng đúng thuật toán đã chọn.

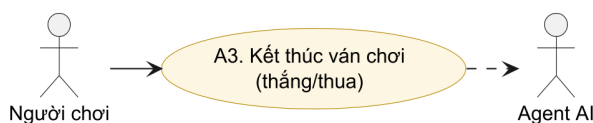


Hình 2.4: Use case UC-A2 – Điều khiển xe tăng

Bảng 2.4: Đặc tả use case UC-A2 – Điều khiển xe tăng

Mã use case	UC-A2
Tên use case	Điều khiển xe tăng
Tác nhân chính	Người chơi
Mô tả	Trong trận, người chơi điều khiển thân xe, xoay tháp pháo và bắn đạn để phòng thủ Eagle Base.
Tiền điều kiện	Đang trong một ván chơi.
Luồng sự kiện chính	1. Người chơi nhập lệnh di chuyển (bàn phím), xoay tháp (chuột) và bắn (chuột trái). 2. PlayerInput chuyển lệnh tới TankController. 3. TankMover áp dụng vật lý Rigidbody2D; Turret bắn đạn lấy từ object pool. 4. Đạn va chạm mục tiêu và gây sát thương qua Damagable.
Luồng thay thế / ngoại lệ	3a. Va chạm tường (lớp ObstaclesMovement) → xe tăng không đi xuyên.

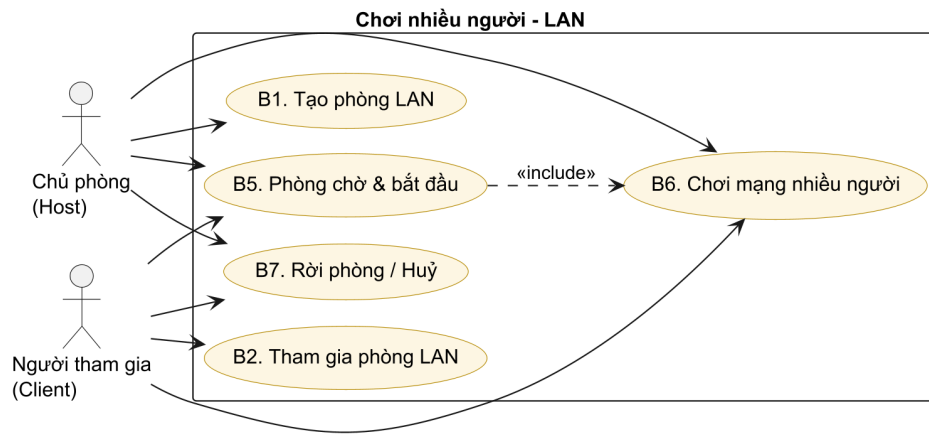
Hậu điều kiện	Trạng thái vị trí xe tăng và đạn được cập nhật.
----------------------	---

**Hình 2.5:** Use case UC-A3 – Kết thúc ván chơi**Bảng 2.5:** Đặc tả use case UC-A3 – Kết thúc ván chơi

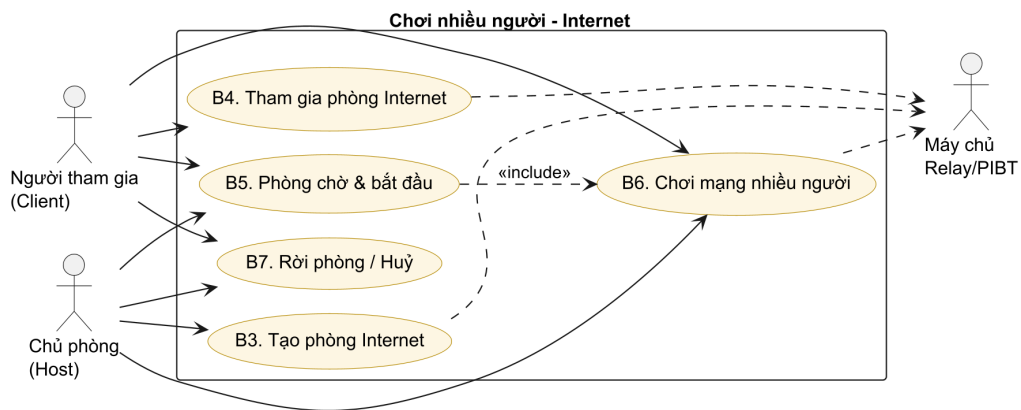
Mã use case	UC-A3
Tên use case	Kết thúc ván chơi (thắng/thua)
Tác nhân chính	Người chơi, Agent AI
Mô tả	Ván kết thúc khi Eagle Base bị phá hủy (agent thắng) hoặc người chơi tiêu diệt hết agent (người chơi thắng).
Tiền điều kiện	Đang trong một ván chơi.
Luồng sự kiện chính	1. Khi Eagle Base hết máu → MapGameOverController hiển thị màn hình thua. 2. Khi không còn agent sống và Eagle còn máu → MapWinController hiển thị màn hình thắng.
Luồng thay thế / ngoại lệ	1a. Trong backtest, run chạm ngưỡng thời gian (timeout) → ghi kết quả Timeout.
Hậu điều kiện	Màn hình kết quả hiển thị; người chơi có thể chơi lại hoặc về menu.

2.3.2 Nhóm B – Chơi nhiều người

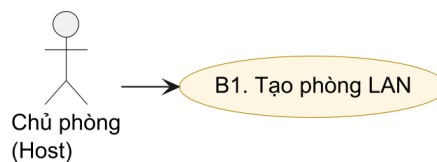
Cả chế độ LAN và Internet đều được truy cập qua nút **MULTIPLAYER INTERNET** ở menu chính, dẫn tới màn chọn *Host* hoặc *Join*. Hệ thống hỗ trợ chơi nhiều người trên cả mạng LAN và Internet. Trên LAN (Hình 2.6), người chơi tạo phòng (B1) hoặc dò tìm và tham gia phòng (B2). Trên Internet (Hình 2.7), hệ thống dùng Unity Relay: chủ phòng tạo phòng và nhận mã phòng (B3), người tham gia vào bằng mã hoặc URL (B4). Cả hai môi trường dùng chung phòng chờ (B5), vòng chơi đồng bộ (B6) và xử lý rời phòng (B7).



Hình 2.6: Biểu đồ use case nhóm B – Chơi nhiều người (LAN)



Hình 2.7: Biểu đồ use case nhóm B – Chơi nhiều người (Internet)

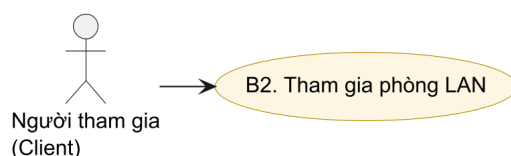


Hình 2.8: Use case UC-B1 – Tạo phòng LAN

Bảng 2.6: Đặc tả use case UC-B1 – Tạo phòng LAN

Mã use case	UC-B1
Tên use case	Tạo phòng LAN
Tác nhân chính	Chủ phòng (Host)
Mô tả	Người chơi tạo một phòng trên mạng LAN, khởi tạo máy chủ Fish-Net và công bố phòng để các máy khác dò tìm.
Tiền điều kiện	Các máy ở cùng mạng LAN.

Luồng sự kiện chính	1. Chủ phòng chọn bản đồ/chế độ và màu xe tăng, rồi chọn "Host". 2. LanSessionManager khởi tạo NetworkManager (Fish-Net); LanDiscovery phát quảng bá phòng. 3. Giao diện chuyển sang màn Hosting, hiển thị thông tin phòng và nút START.
Luồng thay thế / ngoại lệ	2a. Cổng mạng bị chiếm → báo lỗi và quay lại màn chọn chế độ.
Hậu điều kiện	Phòng LAN đang chạy và chờ người tham gia.



Hình 2.9: Use case UC-B2 – Tham gia phòng LAN

Bảng 2.7: Đặc tả use case UC-B2 – Tham gia phòng LAN

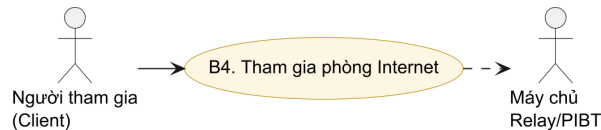
Mã use case	UC-B2
Tên use case	Tham gia phòng LAN
Tác nhân chính	Người tham gia (Client)
Mô tả	Người chơi dò tìm trong mạng LAN hoặc nhập địa chỉ để tham gia một phòng có sẵn.
Tiền điều kiện	Có một phòng LAN đang chạy trong cùng mạng.
Luồng sự kiện chính	1. Người chơi chọn "Join". 2. LanDiscovery liệt kê các phòng tìm thấy, hoặc người chơi nhập địa chỉ thủ công. 3. Kết nối qua LanNetworkBridge; JoinApprovalPayload xác thực yêu cầu vào phòng. 4. Người chơi vào phòng chờ.
Luồng thay thế / ngoại lệ	2a. Địa chỉ sai → báo lỗi kết nối, quay lại màn nhập địa chỉ.
Hậu điều kiện	Người tham gia đang ở trong phòng chờ.



Hình 2.10: Use case UC-B3 – Tạo phòng Internet

Bảng 2.8: Đặc tả use case UC-B3 – Tạo phòng Internet

Mã use case	UC-B3
Tên use case	Tạo phòng Internet
Tác nhân chính	Chủ phòng (Host)
Tác nhân phụ	Máy chủ Relay/Registry
Mô tả	Tạo phòng chơi qua Internet bằng Unity Relay; hệ thống đăng ký vào registry và sinh mã phòng kèm đường dẫn tham gia.
Tiền điều kiện	Có kết nối Internet; dịch vụ Relay và registry khả dụng.
Luồng sự kiện chính	1. Chủ phòng chọn bản đồ/chế độ và màu xe tăng, rồi chọn tạo phòng Internet. 2. RelayManager.CreateRoomAsync cấp allocation và joinCode. 3. InternetSessionClient đăng ký registry, nhận về mã phòng, joinUrl, webUrl. 4. Hiển thị mã phòng; có thể sao chép qua WebClipboard (bản WebGL).
Luồng thay thế / ngoại lệ	2a. Relay hoặc registry lỗi → báo lỗi và hủy thao tác.
Hậu điều kiện	Phòng Internet sẵn sàng; mã phòng được công bố cho người tham gia.

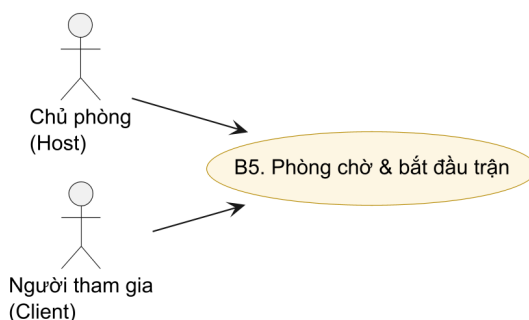


Hình 2.11: Use case UC-B4 – Tham gia phòng Internet

Bảng 2.9: Đặc tả use case UC-B4 – Tham gia phòng Internet

Mã use case	UC-B4
Tên use case	Tham gia phòng Internet
Tác nhân chính	Người tham gia (Client)
Tác nhân phụ	Máy chủ Relay/Registry
Mô tả	Người chơi tham gia phòng Internet bằng mã phòng hoặc URL chia sẻ.
Tiền điều kiện	Có mã phòng/URL hợp lệ; có kết nối Internet.

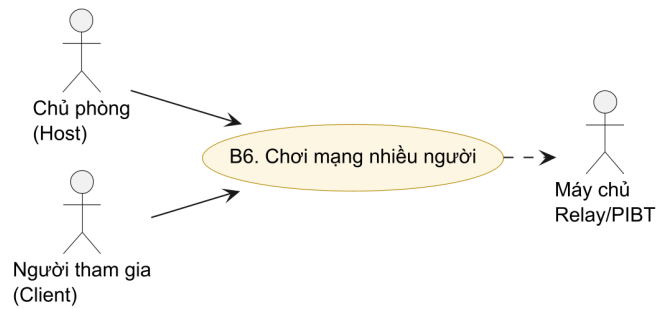
Luồng sự kiện chính	1. Người chơi nhập mã phòng hoặc URL (InternetJoinParser tách mã). 2. InternetSessionClient tra cứu registry, lấy địa chỉ host và loại transport (ưu tiên webHost cho bản WebGL). 3. RelayManager.JoinRoomAsync kết nối bằng joinCode. 4. Người chơi vào phòng chờ.
Luồng thay thế / ngoại lệ	1a. Mã sai hoặc registry không tìm thấy phòng → báo lỗi. 2a. Bản WebGL dùng endpoint web qua PIBTWebRelayClient.
Hậu điều kiện	Người tham gia đang ở trong phòng chờ Internet.



Hình 2.12: Use case UC-B5 – Phòng chờ & bắt đầu trận

Bảng 2.10: Đặc tả use case UC-B5 – Phòng chờ & bắt đầu trận

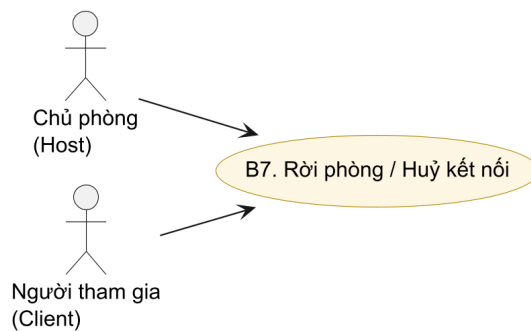
Mã use case	UC-B5
Tên use case	Phòng chờ và bắt đầu trận
Tác nhân chính	Chủ phòng, Người tham gia
Mô tả	Trong phòng chờ, mỗi người chơi bật trạng thái sẵn sàng; chủ phòng bắt đầu trận khi tất cả đã sẵn sàng.
Tiền điều kiện	Người chơi đã ở trong phòng (LAN hoặc Internet).
Luồng sự kiện chính	1. Mỗi người chơi bấm READY (đảo trạng thái _localReady). 2. Trạng thái sẵn sàng được đồng bộ tới mọi máy. 3. Chủ phòng bấm START (RequestStartServerRpc). 4. Hệ thống đồng bộ bản đồ, spawn xe tăng cho mọi người chơi và Eagle Base chung.
Luồng thay thế / ngoại lệ	3a. Còn người chưa sẵn sàng → nút START bị vô hiệu hóa.
Hậu điều kiện	Trận nhiều người bắt đầu đồng bộ trên mọi máy.



Hình 2.13: Use case UC-B6 – Chơi mạng nhiều người

Bảng 2.11: Đặc tả use case UC-B6 – Chơi mạng nhiều người

Mã use case	UC-B6
Tên use case	Chơi mạng nhiều người
Tác nhân chính	Chủ phòng, Người tham gia
Tác nhân phụ	Máy chủ Relay/PIBT
Mô tả	Nhiều người chơi cùng phòng thủ Eagle Base; trạng thái xe tăng, đạn và Eagle được đồng bộ thời gian thực; số agent AI tỉ lệ theo số người chơi.
Tiền điều kiện	Trận đã được bắt đầu (UC-B5).
Luồng sự kiện chính	1. Mỗi người chơi điều khiển xe tăng của mình (UC-A2). 2. LanGameCoordinator/NetworkManager đồng bộ trạng thái qua Relay hoặc LAN. 3. Số agent bằng số người chơi nhân với hệ số; các agent tấn công Eagle Base chung. 4. Trận kết thúc đồng bộ trên mọi máy (UC-A3).
Luồng thay thế / ngoại lệ	2a. Một người tham gia mất kết nối → chủ phòng nhận thông báo và tiếp tục với số người còn lại.
Hậu điều kiện	Kết quả chung được hiển thị đồng bộ trên mọi máy.



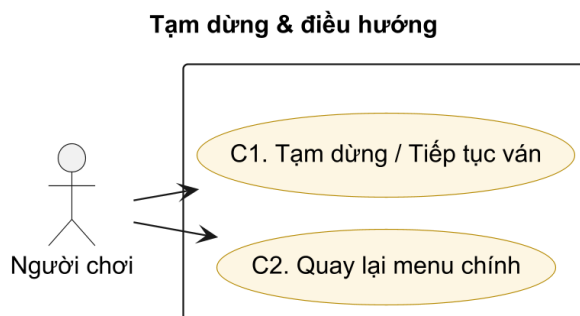
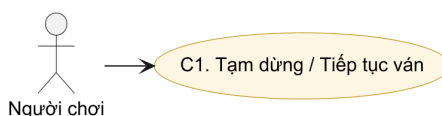
Hình 2.14: Use case UC-B7 – Rời phòng / Huỷ kết nối

Bảng 2.12: Đặc tả use case UC-B7 – Rời phòng / Hủy kết nối

Mã use case	UC-B7
Tên use case	Rời phòng hoặc hủy kết nối
Tác nhân chính	Chủ phòng, Người tham gia
Mô tả	Người chơi hủy thao tác kết nối hoặc rời phòng; hệ thống xử lý ngắt kết nối sạch và giải phóng tài nguyên.
Tiền điều kiện	Đang ở màn kết nối hoặc đang trong phòng.
Luồng sự kiện chính	1. Người chơi bấm Cancel hoặc Rời phòng. 2. Hệ thống đóng kết nối, giải phóng allocation Relay (nếu có) và quay lại menu.
Luồng thay thế / ngoại lệ	1a. Chủ phòng rời → phòng đóng, các người tham gia nhận thông báo và trở về menu.
Hậu điều kiện	Người chơi trở về menu; tài nguyên mạng được giải phóng.

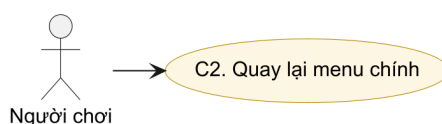
2.3.3 Nhóm C – Tạm dừng và điều hướng phiên chơi

Trong khi chơi (đơn hoặc nhiều người), người chơi có thể tạm dừng và tiếp tục ván (C1) hoặc quay lại menu chính (C2) thông qua lớp phủ tạm dừng. Hình 2.15 là biểu đồ use case nhóm này.

**Hình 2.15:** Biểu đồ use case nhóm C – Tạm dừng và điều hướng**Hình 2.16:** Use case UC-C1 – Tạm dừng / Tiếp tục ván**Bảng 2.13:** Đặc tả use case UC-C1 – Tạm dừng / Tiếp tục ván

Mã use case	UC-C1
Tên use case	Tạm dừng và tiếp tục ván
Tác nhân chính	Người chơi

Mô tả	Người chơi tạm dừng ván đang chơi và tiếp tục lại khi muốn.
Tiền điều kiện	Đang trong một ván chơi.
Luồng sự kiện chính	1. Người chơi bấm nút tạm dừng. 2. Hệ thống hiển thị lớp phủ PAUSED với hai nút RESUME và MENU. 3. Bấm RESUME → ván chơi tiếp tục từ trạng thái trước đó.
Luồng thay thế / ngoại lệ	2a. Trong chế độ nhiều người, ApplyNetworkPause đồng bộ trạng thái tạm dừng phù hợp giữa các máy.
Hậu điều kiện	Ván chơi tiếp tục bình thường.



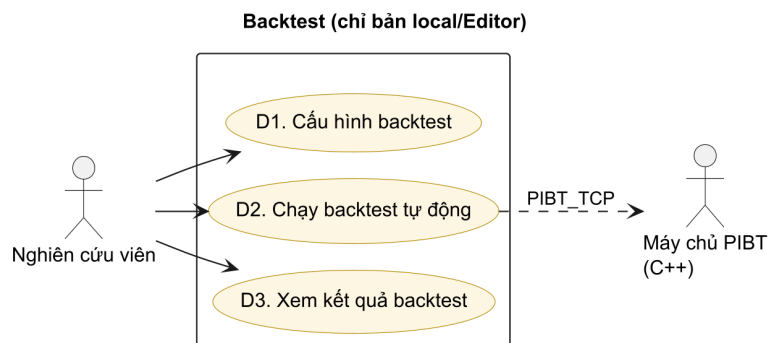
Hình 2.17: Use case UC-C2 – Quay lại menu chính

Bảng 2.14: Đặc tả use case UC-C2 – Quay lại menu chính

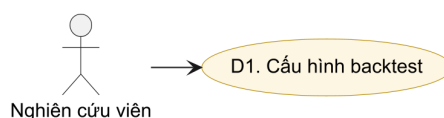
Mã use case	UC-C2
Tên use case	Quay lại menu chính
Tác nhân chính	Người chơi
Mô tả	Từ lớp phủ tạm dừng, người chơi thoát ván hiện tại và trở về menu chính.
Tiền điều kiện	Đang ở lớp phủ tạm dừng (đã tạm dừng ván).
Luồng sự kiện chính	1. Người chơi bấm MENU trong lớp phủ tạm dừng. 2. Hệ thống kết thúc ván hiện tại và tải lại scene Menu.
Luồng thay thế / ngoại lệ	1a. Trong chế độ nhiều người, rời ván đồng nghĩa rời phòng (xem UC-B7).
Hậu điều kiện	Người chơi trở về menu chính.

2.3.4 Nhóm D – Backtest (chỉ bản local/Editor)

Backtest là quy trình nghiệp vụ quan trọng nhất từ góc độ nghiên cứu, dùng để so sánh định lượng ba thuật toán. Ở bản production, chức năng backtest bị ẩn – người dùng chỉ thấy các chế độ chơi; muốn chạy backtest phải dùng bản local (Editor). Hình 2.18 là biểu đồ use case nhóm backtest.



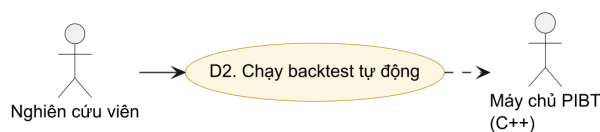
Hình 2.18: Biểu đồ use case nhóm D – Backtest (local/Editor)



Hình 2.19: Use case UC-D1 – Cấu hình backtest

Bảng 2.15: Đặc tả use case UC-D1 – Cấu hình backtest

Mã use case	UC-D1
Tên use case	Cấu hình backtest
Tác nhân chính	Nghiên cứu viên
Mô tả	Cấu hình ma trận thực nghiệm: chọn bản đồ, thuật toán, số lần lặp, bật/tắt chương ngại động và số agent.
Tiền điều kiện	Chạy bản local/Editor; nếu chọn PIBT_TCP thì máy chủ C++ đang chạy.
Luồng sự kiện chính	- Mở BacktestConfigUI (nút BACKTEST chỉ hiện trong Editor). - Chọn tổ hợp bản đồ × thuật toán × số lần lặp, bật/tắt chương ngại động, đặt số agent. - Lưu cấu hình vào BacktestMode.
Luồng thay thế / ngoại lệ	1a. Bản production không hiển thị nút BACKTEST → use case không khả dụng.
Hậu điều kiện	Cấu hình thực nghiệm sẵn sàng để chạy.



Hình 2.20: Use case UC-D2 – Chạy backtest tự động

Bảng 2.16: Đặc tả use case UC-D2 – Chạy backtest tự động

Mã use case	UC-D2
Tên use case	Chạy backtest tự động
Tác nhân chính	Nghiên cứu viên
Tác nhân phụ	Máy chủ PIBT (C++)
Mô tả	Hệ thống lặp qua mọi tổ hợp cấu hình, mỗi run chạy kịch bản đến khi Eagle bị phá hoặc hết thời gian, thu thập chỉ số định lượng.
Tiền điều kiện	Đã cấu hình backtest (UC-D1).
Luồng sự kiện chính	1. Người dùng bấm Start. 2. BacktestRunner lặp qua từng tổ hợp (map × algorithm × rep). 3. Mỗi run spawn agent, chạy kịch bản, ghi chỉ số (replan, số ô đi, thời gian, kết quả). 4. Sau khi hết các run, xuất backtest_summary_*.csv, backtest_agents_*.csv và backtest_chart_*.html.
Luồng thay thế / ngoại lệ	3a. PIBT_TCP mất kết nối → ghi Outcome=ConnectionError và tiếp tục. 3b. Run chậm timeout → ghi Outcome=Timeout.
Hậu điều kiện	Tệp CSV và HTML chart xuất hiện trong thư mục BacktestResults/.



Hình 2.21: Use case UC-D3 – Xem kết quả backtest

Bảng 2.17: Đặc tả use case UC-D3 – Xem kết quả backtest

Mã use case	UC-D3
Tên use case	Xem kết quả backtest
Tác nhân chính	Nghiên cứu viên
Mô tả	Nghiên cứu viên xem kết quả định lượng qua tệp CSV và biểu đồ HTML.
Tiền điều kiện	Đã chạy ít nhất một đợt backtest và sinh tệp kết quả.

Luồng sự kiện chính	1. Mở thư mục BacktestResults/. 2. Xem backtest_summary_*.csv để nắm kết quả tổng hợp. 3. Mở backtest_chart_*.html để xem biểu đồ trực quan. 4. Mở backtest_agents_*.csv để xem chi tiết chỉ số từng agent.
Luồng thay thế / ngoại lệ	3a. Thiếu Python/matplotlib → không sinh được PNG nhưng HTML vẫn hiển thị bảng số liệu.
Hậu điều kiện	Người dùng nắm được kết quả định lượng của đợt thực nghiệm.

2.4 Yêu cầu phi chức năng

Hiệu năng: Thuật toán A* phải hoàn thành tìm đường cho mỗi agent trong dưới 5 ms trên bản đồ nhỏ (Alpha32), để tổng thời gian tính toán mỗi khung hình không vượt quá 16 ms (60 FPS). Bộ giải PIBT – cả bản C# chạy trong tiến trình lẫn bản C++ giao tiếp qua TCP – phải trả về kết quả lập kế hoạch cho mỗi bước phối hợp trong dưới 1 giây (không quá 1 giây), đủ nhanh để người chơi không cảm nhận được độ trễ và bảo đảm trải nghiệm chơi mượt mà.

Tính ổn định: Mỗi run backtest phải tái lập được với cùng seed; hệ thống không được crash trong suốt 900 run liên tiếp.

Tính tương thích: Ứng dụng chạy trên Windows 10/11 với Unity 6000.3.x; chế độ WebGL tương thích Chrome và Firefox. Tập .map phải đọc được theo chuẩn MovingAI không cần chỉnh sửa.

Khả năng mở rộng: Thêm thuật toán mới chỉ cần tạo lớp agent mới kế thừa interface chung, không cần thay đổi BacktestRunner hay TankController.

Tính quan sát được: Hệ thống phải trực quan hóa hành vi thuật toán ngay trong game – vẽ đường đi dự kiến của agent và hiển thị chỉ số thời gian thực (số lần tái hoạch định, trạng thái kết nối server, thời gian còn lại của run) – để người dùng quan sát trực tiếp khác biệt giữa ba thuật toán mà không cần đọc log thô.

Tính triển khai được: Hệ thống phải đóng gói và triển khai được lên hạ tầng đám mây để phục vụ demo trực tuyến – bản WebGL chạy trên trình duyệt không cần cài đặt, máy chủ PIBT đặt sau reverse proxy có HTTPS, và quy trình triển khai được tự động hóa để tái lập môi trường nhất quán (chi tiết phần triển khai ở Chương 4).

Kết chương 2: Chương này đã phân tích bốn nhóm giải pháp điều hướng đa agent và xác nhận A* (baseline) cùng PIBT (thuật toán chính) là lựa chọn phù hợp nhất với yêu cầu đề án. Năm bản đồ benchmark chuẩn đã được chọn để đảm bảo tính so sánh được với nghiên cứu cộng đồng. Hệ thống tác nhân và các use case theo nhóm A, B, C, D (cùng nhóm E ở phụ lục) đã được đặc tả chi tiết theo mẫu chuẩn kèm biểu đồ, làm cơ sở cho thiết kế hệ thống trong Chương 3 và Chương 4.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương 2 đã xác định A* và PIBT là hai thuật toán cần triển khai, Unity Engine là nền tảng phát triển, và TCP là giao thức tích hợp máy chủ ngoài. Chương này trình bày cơ sở lý thuyết và lý do lựa chọn từng công nghệ.

3.1 Bài toán MAPF

MAPF được định nghĩa trên đồ thị $G = (V, E)$ với k agent. Mỗi agent i có vị trí xuất phát $s_i \in V$ và đích $g_i \in V$. Tại mỗi bước thời gian, agent di chuyển sang ô lân cận hoặc đứng yên. Ràng buộc: (i) *vertex conflict* – hai agent không được ở cùng ô cùng bước; (ii) *edge conflict* – hai agent không được đổi chỗ cho nhau trong cùng một bước. Mục tiêu: tìm tập kế hoạch $\pi_1, \dots, \pi_k$ tối thiểu hóa tổng chi phí (số bước di chuyển) và không vi phạm ràng buộc [1].

Chất lượng lời giải thường được đo bằng hai hàm mục tiêu: *makespan* – số bước của agent về đích muộn nhất, và *sum-of-costs* (tổng chi phí) – tổng số bước của tất cả agent. Tìm lời giải tối ưu theo hai hàm này là bài toán NP-khó [1]; do đó, khi số agent lớn và có ràng buộc thời gian thực, các thuật toán dựa trên luật như PIBT chấp nhận hi sinh tính tối ưu toàn cục để đổi lấy tốc độ phản hồi.

Trong bối cảnh game thời gian thực, bài toán được đơn giản hóa: agent không cần đạt đích cố định mà liên tục tiến gần mục tiêu cố định (Eagle Base), và phải tái lập kế hoạch khi bản đồ thay đổi (chướng ngại mới xuất hiện, agent bị tiêu diệt).

Cũng cần phân biệt giữa MAPF *cổ điển* (offline, một lần) – mọi agent có đích cố định và toàn bộ lời giải được tính trước khi thực thi – với MAPF *trọn đời* (Lifelong MAPF, viết tắt LMAPF) – agent liên tục được gán nhiệm vụ mới và hệ thống lập kế hoạch lại theo từng bước thời gian. Kịch bản của đồ án gần với LMAPF: đích là Eagle Base nhưng môi trường thay đổi liên tục (chướng ngại động, agent bị tiêu diệt), nên lời giải phải được cập nhật trực tuyến mỗi bước thay vì tính trọn một lần. Đây là lý do đồ án chọn họ thuật toán chạy theo bước thời gian (A* tái hoạch định định kỳ và PIBT) thay vì các solver offline tối ưu.

3.2 Thuật toán A*

A* [2] tìm đường ngắn nhất trên đồ thị có trọng số bằng cách mở rộng các nút theo hàm ước lượng $f(n) = g(n) + h(n)$, trong đó $g(n)$ là chi phí thực từ điểm xuất phát đến nút n , và $h(n)$ là heuristic ước lượng chi phí từ n đến đích. Khi $h(n)$ nhất quán (consistent), A* đảm bảo tìm được đường ngắn nhất.

Trong đồ án, heuristic Manhattan $h(n) = |x_n - x_g| + |y_n - y_g|$ được dùng cho lưới 4 láng giềng (lên, xuống, trái, phải), đảm bảo tính nhất quán và phù hợp với

cấu trúc bản đồ dạng ô vuông. A^* được chọn làm baseline vì: (i) đã được kiểm chứng rộng rãi trong game 2D; (ii) hiệu quả trên lưới thưa; (iii) dễ tích hợp và hiểu kết quả.

Về cài đặt, A^* duy trì hai tập nút: *open set* chứa các nút chờ mở rộng (lấy ra theo $f(n)$ nhỏ nhất qua một hàng đợi ưu tiên) và *closed set* chứa các nút đã xét. Trên lưới 4 láng giềng, heuristic Manhattan vừa *chấp nhận được* (không bao giờ ước lượng vượt quá chi phí thật) vừa *nhất quán*, nên A^* không phải mở lại nút đã đóng mà vẫn bảo đảm đường ngắn nhất. Chi phí tính toán tỉ lệ với số ô được mở rộng: trên bản đồ thoáng vùng mở rộng nhỏ nên A^* chạy rất nhanh, còn trên mê cung A^* phải dò gần như toàn bộ không gian khiến chi phí tăng vọt – điều này dự báo trước hành vi khác biệt của A^* giữa năm bản đồ ở Chương 4.

Hạn chế của A^* trong bối cảnh MAPF: mỗi agent tính đường độc lập, không có cơ chế phối hợp, dẫn đến va chạm và replan phản ứng liên tục.

3.3 Thuật toán PIBT

PIBT (Priority Inheritance with Backtracking) [4] là thuật toán MAPF chạy theo bước thời gian (time-step based). Tại mỗi bước, các agent được xếp hạng ưu tiên; agent ưu tiên cao hơn di chuyển trước, agent ưu tiên thấp hơn kế thừa ưu tiên nếu bị chặn (*priority inheritance*) và có thể hoàn tác bước di chuyển nếu không tìm được ô trống (*backtracking*).

Để minh họa cơ chế, xét tình huống sau: agent A (ưu tiên cao) muốn tiến vào ô đang bị agent B (ưu tiên thấp) chiếm. Theo kế thừa ưu tiên, B tạm thời nhận mức ưu tiên của A và buộc phải tìm một ô trống để nhường đường; nếu quanh B còn ô hợp lệ, B dịch sang đó và A tiến vào – cả hai cùng tiến mà không va chạm. Ngược lại, nếu B bị vây kín không còn ô nào hợp lệ, thuật toán hoàn tác: A rút lại nước đi và thử hướng khác. Nhờ đệ quy kế thừa–hoàn tác này, PIBT gỡ xung đột cục bộ chỉ trong một lần quét theo thứ tự ưu tiên, đạt độ phức tạp gần tuyến tính theo số agent trong trường hợp trung bình.

Ưu điểm của PIBT: hoàn chỉnh trên bất kỳ bản đồ nào có đường đi tồn tại; thời gian tính toán thực tế tỷ lệ tuyến tính với số agent trong trường hợp trung bình; phù hợp với môi trường thời gian thực. PIBT được chọn vì nó cân bằng tốt giữa hiệu năng và khả năng phối hợp đa agent, và đã được dùng thành công trong cuộc thi điều phối robot quốc tế [5].

Tính hoàn chỉnh của PIBT được bảo đảm khi đồ thị thỏa một điều kiện hình học: mọi cặp ô kề nhau đều nằm trên một chu trình đơn có độ dài tối thiểu ba. Điều kiện này cho phép agent ưu tiên cao luôn đẩy được agent chặn đường ra khỏi ô mong

muốn; những bản đồ có nhiều hành lang cắt một ô vi phạm điều kiện nên dễ phát sinh deadlock cục bộ hơn, đòi hỏi cơ chế thoát kẹt hỗ trợ.

PIBT tiếp tục là nền tảng của nhiều phương pháp MAPF hiện đại. Gần đây, biến thể mở rộng EPIBT [8] bổ sung các thao tác đa hành động (multi-action operations) kết hợp graph guidance và large neighborhood search, đạt kết quả tốt nhất hiện tại trên bài toán Lifelong MAPF trực tuyến có ràng buộc xoay hướng. Điều này cho thấy hướng tiếp cận PIBT mà đề án lựa chọn vẫn đang được nghiên cứu tích cực và bám sát hiệu năng hàng đầu.

Một điểm cần lưu ý khi áp dụng PIBT cho xe tăng là *mô hình hành động có xoay* (rotation action model). PIBT cổ điển giả định mỗi agent có thể bước sang một ô kề bất kỳ ngay trong một bước thời gian; nhưng xe tăng có hướng thân và phải xoay trước khi tiến, nên trạng thái của agent gồm cả vị trí lẫn hướng, với bốn hành động cơ bản: tiến, xoay phải, xoay trái và chờ. Khi đó, việc di chuyển sang một ô bên cạnh có thể tốn hai đến ba bước (xoay rồi tiến) thay vì một bước. Đây chính là cơ sở cho cải tiến EPIBT được trình bày ở Mục 5.5, nơi mỗi agent nhìn trước một chuỗi hành động ngắn để tính được chi phí xoay mà PIBT một bước bỏ sót.

EPIBT khái quát ý tưởng này bằng *thao tác đa hành động* (multi-action operation): thay vì chọn một hành động đơn, mỗi agent chọn cả một chuỗi hành động ngắn có độ dài cố định rồi chỉ thực thi hành động đầu tiên ở bước hiện tại. Độ dài ba là tối thiểu để từ một hướng bất kỳ vẫn tới được mọi ô kề, nhờ đó agent dự đoán trước được chi phí xoay; tăng độ dài cho tầm nhìn xa hơn nhưng làm số phương án tăng nhanh. Bảng 3.1 minh họa sự đánh đổi này theo số liệu của [8].

Bảng 3.1: Số ô và số chuỗi ô đạt được theo độ dài thao tác

Độ dài thao tác	Số ô đạt được	Số chuỗi ô khác nhau
1	2	2
2	5	6
3	11	17
4	21	48
5	35	136

Hai cơ chế hỗ trợ của EPIBT là *kế thừa thao tác* (tái dùng chuỗi đã chọn ở bước trước, bỏ hành động đầu và nối thêm một bước chờ) và *xét lại agent* có giới hạn để gỡ các xung đột phức tạp. Phần cài đặt cụ thể cùng cách tích hợp vào máy chủ C++ được trình bày ở Mục 5.5.

Đề án tiến hành triển khai thuật toán PIBT theo hai phương pháp tiếp cận khác nhau. Phương pháp thứ nhất là cài đặt trực tiếp trong môi trường Unity bằng ngôn ngữ C# thông qua lớp GridEnemyAgentPIBT, giúp hệ thống hoạt động độc lập mà

không cần phụ thuộc vào hạ tầng bên ngoài. Phương pháp thứ hai là giao tiếp qua giao thức TCP với một máy chủ C++ có tốc độ xử lý cao, được hiện thực hóa thông qua lớp `GridEnemyAgentPIBT_TCP`, nhằm minh chứng cho khả năng tích hợp một hệ thống tìm đường (planner) độc lập bên ngoài Unity.

3.4 Unity Engine và C#

Unity Engine [9] phiên bản 6000.3.10f1 (Unity 6) được lựa chọn vì: (i) hỗ trợ xuất bản đa nền tảng bao gồm WebGL; (ii) hệ thống vật lý 2D (`Rigidbody2D`, `Collider2D`) hoàn chỉnh; (iii) hệ sinh thái lớn và tài liệu đầy đủ; (iv) hỗ trợ scripting C# với `IL2CPP` cho hiệu năng tốt. Unity NavMesh không được dùng làm hệ thống điều hướng chính vì không tương thích với định dạng bản đồ `.map` và không hỗ trợ ràng buộc MAPF ở lớp lập kế hoạch.

C# được chọn (thay vì C++) vì tích hợp trực tiếp với Unity mà không cần lớp binding bổ sung, giúp code dễ đọc và kiểm tra trong quá trình nghiên cứu.

Cụ thể, đồ án khai thác các thành phần vật lý 2D của Unity làm nền chung cho cả ba thuật toán: `Rigidbody2D` và `Collider2D` cho chuyển động và va chạm có quán tính của xe tăng; hệ thống *layer* và *layer mask* để tách biệt nhóm tường, đạn và agent, phục vụ kiểm tra tầm nhìn (line-of-sight) khi bắn; cùng `Physics2D.Linecast` để phát hiện trúng đích. Chính lớp vật lý này khiến việc tích hợp MAPF trở nên không tầm thường: lời giải rời rạc trên lưới phải được chuyển thành lực đẩy và mô-men xoay hợp lệ, đồng thời chấp nhận sai số do quán tính – điều mà benchmark MAPF thuần túy không phải xử lý. Đây cũng là lý do đồ án giữ `TankController` làm điểm hội tụ chung: cả người chơi lẫn ba thuật toán AI đều điều khiển xe tăng qua cùng một giao diện, bảo đảm so sánh công bằng giữa các thuật toán.

Các lựa chọn thay thế đã xem xét: Godot Engine (thiếu hệ sinh thái C# MAPF), Unreal Engine (quá phức tạp cho 2D top-down), pygame (thiếu vật lý và mạng).

3.5 Giao tiếp TCP giữa Unity và máy chủ C++

Để tích hợp thuật toán PIBT viết bằng C++ (từ cuộc thi robot đua The League of Robot Runners 2024) vào Unity mà không cần port toàn bộ sang C#, đồ án sử dụng kết nối TCP/IP theo mô hình client–server. Phía Unity đóng vai trò client: sau mỗi bước, Unity gửi trạng thái hiện tại gồm vị trí các agent và kích thước bản đồ, rồi nhận lại ô mục tiêu tiếp theo cho từng agent. Phía máy chủ C++ đóng vai trò server: chạy thuật toán PIBT, duy trì trạng thái lập kế hoạch và lắng nghe trên cổng 7777 (localhost).

Hướng tiếp cận PIBT này vẫn đang được nghiên cứu tích cực: bản mở rộng

EPIBT [8] (phiên bản đầy đủ được chấp nhận tại hội nghị AAAI-26) còn vượt qua chính lời giải vô địch LoRR 2024 trên bài toán Lifelong MAPF có ràng buộc xoay hướng. Việc nhúng máy chủ PIBT C++ vào Unity vì vậy không chỉ là giải pháp kỹ thuật mà còn là cách đưa một dòng thuật toán đang dẫn đầu vào trải nghiệm chơi trực quan.

TCP được chọn thay vì UDP vì đảm bảo thứ tự và toàn vẹn gói tin – yếu tố quan trọng khi mỗi gói lệnh di chuyển đều cần thiết và không chấp nhận mất mát. Giao tiếp qua mạng nội bộ (localhost/WSL) đảm bảo độ trễ thấp đủ cho ứng dụng thời gian thực.

Về mô hình tương tác, đồ án chọn kiểu *đồng bộ theo nhịp* (synchronous cadence): mỗi bước, Unity gửi trạng thái và chờ máy chủ trả lời trước khi áp dụng hành động, thay vì để hai phía chạy bất đồng bộ tự do. Cách này giữ trạng thái hai phía luôn nhất quán và giúp việc gỡ lỗi, tái lập kết quả đơn giản hơn, đổi lại độ trễ một nhịp mỗi bước. Máy chủ duy trì một *phiên lập kế hoạch* riêng cho mỗi kết nối để lưu trạng thái xuyên suốt ván chơi, còn phía Unity có cơ chế tự kết nối lại khi đường truyền gián đoạn, nhờ đó trận đấu không bị treo khi mạng không ổn định. Chi tiết định dạng thông điệp và vòng đời phiên được trình bày ở Mục 5.4.

3.6 Các tiêu chí đánh giá thuật toán điều hướng

Để so sánh ba thuật toán một cách khách quan, đồ án dựa trên một tập tiêu chí thống nhất, bắt nguồn từ các chỉ số chuẩn của bài toán MAPF [1] nhưng được điều chỉnh cho bối cảnh game. *Tỉ lệ hoàn thành nhiệm vụ* (success rate) đo phần trăm lượt mà agent phá được Eagle Base trước khi hết giờ – tương ứng khái niệm tính hoàn chỉnh trên thực tế. *Thời gian hoàn thành* (makespan thực thi) đo độ dài một lượt cho tới khi kết thúc, phản ánh tốc độ tiếp cận mục tiêu. *Số lần tái hoạch định* (replan count) đo tần suất thuật toán phải tính lại đường, gián tiếp thể hiện mức độ phối hợp và độ ổn định của lời giải. *Số lần phục hồi sau kẹt* (recovery count) đo số lần agent kích hoạt cơ chế thoát kẹt cục bộ, nổi bật trong môi trường động. Cuối cùng, *khả năng mở rộng* (scalability) đo mức độ các chỉ số trên biến thiên khi tăng số agent. Toàn bộ tiêu chí được đo tự động qua hệ thống backtest và phân tích định lượng ở Chương 4.

Kết chương 3: Chương này đã trình bày cơ sở lý thuyết của MAPF, A* và PIBT (gồm cả mô hình hành động có xoay và tập tiêu chí đánh giá), đồng thời giải thích lý do lựa chọn từng công nghệ dựa trên yêu cầu xác định ở Chương 2. Các nền tảng lý thuyết và kỹ thuật này là cơ sở để Chương 4 trình bày chi tiết thiết kế và triển khai hệ thống.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

Hệ thống được xây dựng theo kiến trúc phân tầng kết hợp với kiến trúc thành phần (layered + component-based), đặc trưng của các dự án trên nền tảng Unity Engine, gồm năm tầng theo chiều phụ thuộc từ dưới lên (biểu đồ gói tổng thể ở Hình 4.1). Tầng dữ liệu ở dưới cùng chứa tệp bản đồ định dạng .map chuẩn MovingAI MAPF benchmark và các ScriptableObject lưu thông số vật lý xe tăng và đạn, hoàn toàn độc lập với logic gameplay. Trên đó là tầng lõi MAPF gồm các lớp tìm đường và điều phối đa tác nhân (GridAStarPathfinder, GridEnemyAgent, GridEnemyAgentPIBT, GridEnemyAgentPIBT_TCP), không phụ thuộc vào giao diện người dùng. Tiếp nối là tầng bootstrap, đóng vai trò khởi tạo bản đồ thông qua lớp MapLoader. Tầng này còn đảm nhiệm việc sinh ra (spawn) các nhân vật và kết nối tầng lõi thuật toán với hệ thống vật lý của Unity, thông qua hai lớp MapTankTestBootstrap cùng MapScenarioBootstrap. Tầng gameplay gồm TankController, TankMover và Turret xử lý vật lý, điều khiển và bắn đạn cho cả agent AI lẫn người chơi. Trên cùng là tầng backtest với BacktestRunner, BacktestConfigUI và BacktestMode điều khiển kịch bản tự động, thu thập chỉ số và xuất CSV.

Việc giữ tầng lõi MAPF tách biệt với giao diện và vật lý giúp thay thế thuật toán (A^* , PIBT C#, PIBT C++/TCP) mà không thay đổi phần còn lại của hệ thống.

4.1.2 Thiết kế tổng quan

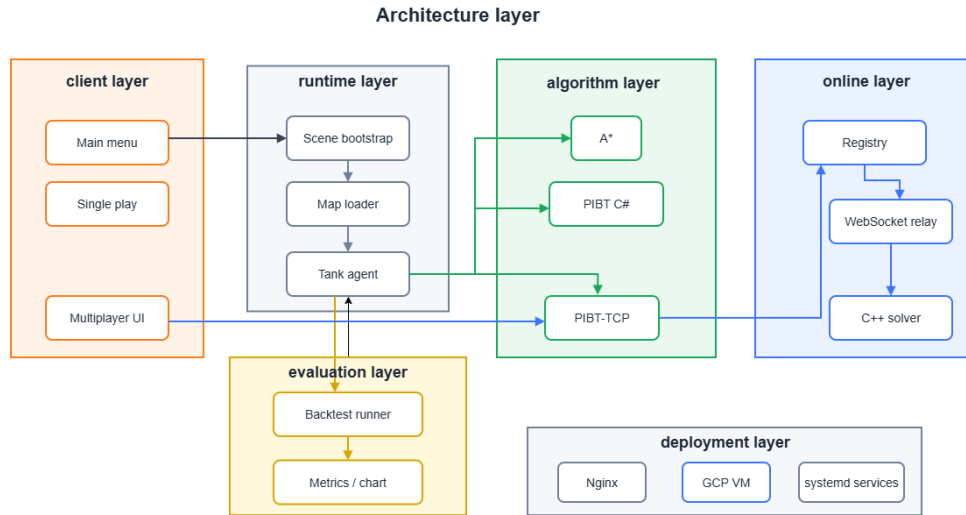
Hình 4.1 mô tả biểu đồ gói UML của hệ thống. Kiến trúc tổng thể được xây dựng tuân thủ theo nguyên lý phân tầng nghiêm ngặt, đảm bảo tính đóng gói và dễ dàng bảo trì. Các gói thành phần được tổ chức thành ba nhóm chính theo chiều dọc, đi từ mức nền tảng hệ thống cho tới mức tương tác với người dùng:

Thứ nhất, nhóm hạ tầng bản đồ và tệp dữ liệu nằm ở tầng thấp nhất. Nhóm này chịu trách nhiệm lưu trữ các tham số thiết lập môi trường, thông số vật lý của các thực thể (ScriptableObject), cũng như quản lý thao tác nạp bản đồ từ định dạng chuẩn của cộng đồng nghiên cứu MAPF.

Thứ hai, nhóm lõi thuật toán MAPF và điều khiển xe tăng nằm ở tầng giữa, đóng vai trò cốt lõi của hệ thống. Tầng này chứa các thuật toán tìm đường (A^* , PIBT) và các lớp xử lý hành vi của tác nhân. Nó hoạt động hoàn toàn độc lập, tiếp nhận

dữ liệu từ tầng dưới và cung cấp các giao tiếp (API) điều khiển cho tầng trên.

Thứ ba, nhóm bootstrap, backtest và giao diện người dùng nằm ở tầng trên cùng. Tầng bootstrap thực hiện chức năng khởi tạo và kết nối (wiring) các thành phần khi bắt đầu trò chơi. Tầng backtest hỗ trợ chạy tự động các kịch bản thử nghiệm để thu thập dữ liệu. Cuối cùng, tầng giao diện chịu trách nhiệm hiển thị thông tin trực quan và tiếp nhận thao tác điều khiển từ người chơi.



Hình 4.1: Biểu đồ gói UML của hệ thống Unity MAPF

Đặc tính quan trọng của thiết kế này là chiều phụ thuộc chỉ đi một chiều từ trên xuống dưới. Các thành phần ở tầng thấp hoàn toàn không phụ thuộc hay cần biết về sự tồn tại của các thành phần ở tầng cao hơn. Điều này giúp loại bỏ triệt để vấn đề vòng phụ thuộc ngược (circular dependency), giúp mã nguồn dễ dàng nâng cấp hoặc tích hợp các thuật toán mới trong tương lai.

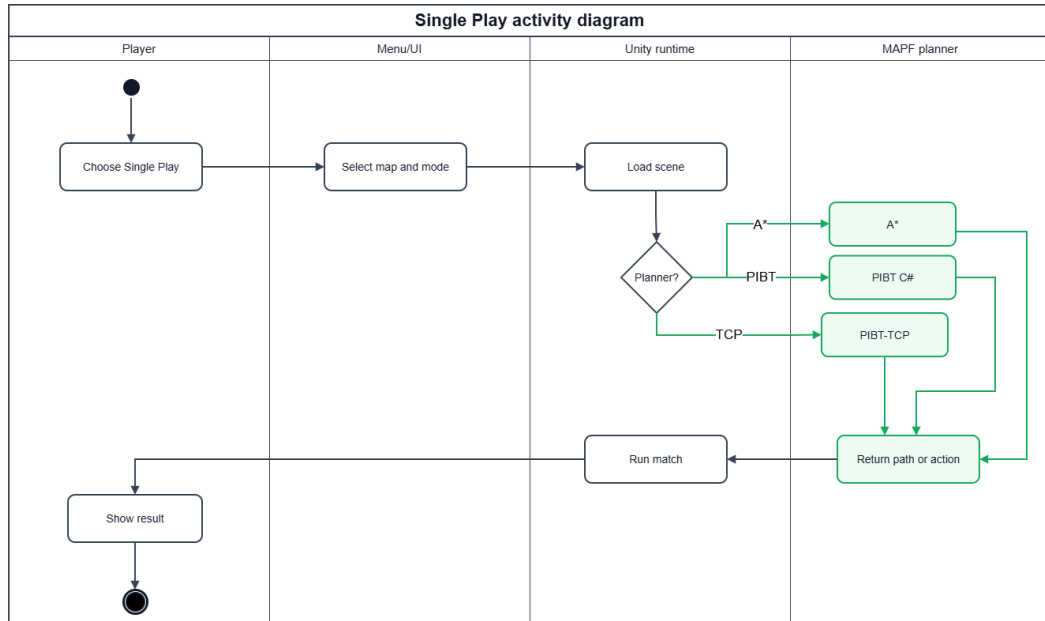
4.2 Thiết kế chi tiết

4.2.1 Thiết kế giao diện

Ứng dụng chạy trên màn hình độ phân giải tối thiểu 1280×720 , tỷ lệ 16:9, và hỗ trợ xuất bản WebGL để chạy trên trình duyệt. Giao diện gồm ba màn hình chính. Menu chính cho phép người dùng lựa chọn chế độ chơi đơn, chơi nhiều người qua Internet hoặc chạy backtest. Giao diện gameplay sử dụng góc nhìn từ trên xuống (top-down 2D), kèm bảng HUD hiển thị điểm máu Eagle Base, số đếm các xe tăng địch còn sống, trong phần backtest hiển thị thêm thời gian và số lần tái hoạch định. Giao diện cấu hình backtest cho phép chọn bản đồ, thuật toán, số lần lặp và bật/tắt chướng ngại động, đồng thời hiển thị thanh tiến độ và log trực tiếp.

Hình 4.2 mô tả sơ đồ hoạt động của luồng giao diện chế độ chơi đơn đã được triển khai, trải qua bốn vùng trách nhiệm (swimlane): người chơi, lớp giao diện menu, runtime của Unity và lõi lập kế hoạch MAPF. Người chơi chọn chế độ và

bản đồ, hệ thống nạp scene tương ứng, sau đó tùy theo thuật toán được chọn (A*, PIBT C# hay PIBT TCP) mà runtime gọi đúng nhánh lập kế hoạch để sinh đường đi hoặc hành động cho agent, rồi chạy trận đấu và hiển thị kết quả. Sơ đồ này cho thấy lựa chọn thuật toán được tách thành một điểm rẽ nhánh trong luồng, nhờ đó việc thay thuật toán không làm thay đổi các bước giao diện phía trước và phía sau.

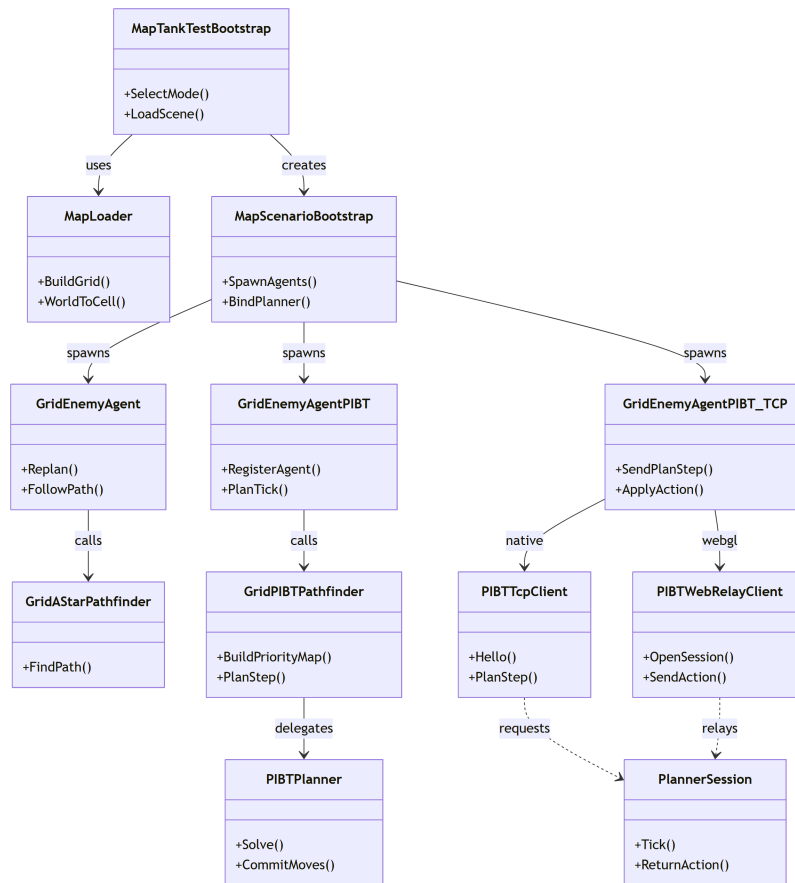


Hình 4.2: Sơ đồ hoạt động luồng giao diện chế độ chơi đơn đã triển khai

4.2.2 Thiết kế các lớp MAPF chính

Hệ thống AI MAPF được triển khai trong ba biến thể song song nhau, dùng chung tầng điều khiển xe tăng, như mô tả trong biểu đồ lớp ở Hình 4.3. Biến thể thứ nhất là GridEnemyAgent kết hợp GridAStarPathfinder, cài đặt thuật toán A* với heuristic Manhattan trên lưới 4 láng giềng và tái hoạch định kỳ theo replanInterval; mỗi agent hoạt động độc lập, không có cơ chế phối hợp. Biến thể thứ hai là GridEnemyAgentPIBT, cài đặt thuật toán PIBT trực tiếp trong Unity C#, trong đó tại mỗi bước toàn bộ agent chạy đồng thời qua một bộ giải PIBT dùng chung để đảm bảo không va chạm ở lớp lập kế hoạch. Biến thể thứ ba là GridEnemyAgentPIBT_TCP, gửi vị trí các agent qua TCP tới máy chủ C++ (PIBT/EPIBT), nhận lại ô mục tiêu tiếp theo cho từng agent rồi điều khiển vật lý tương tự hai biến thể trên.

Biểu đồ lớp cho thấy ba biến thể agent đều kế thừa cùng một mạch điều khiển: chúng nhận lưới từ MapLoader, được MapScenarioBootstrap sinh ra, và ủy thác phân tích toán đường đi cho một bộ tìm đường tương ứng (GridAStarPathfinder, GridPIBTPathfinder hoặc client TCP/WebRelay). Riêng biến thể TCP tách hẳn phần lập kế hoạch sang tiến trình ngoài: PIBTTcpClient dùng cho bản chạy gốc

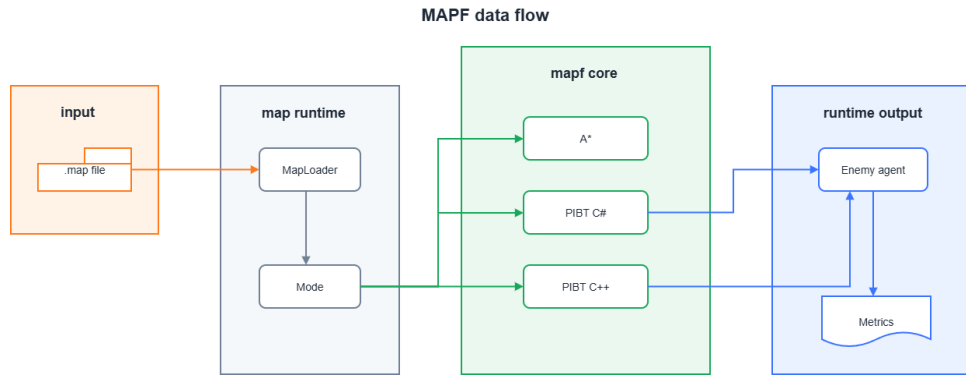


Hình 4.3: Biểu đồ lớp các thành phần tìm đường và điều phối agent

(native) và PIBTWebRelayClient dùng cho bản WebGL, cả hai cùng trao đổi với một PlannerSession ở phía máy chủ. Nhờ điểm chung là tất cả đều điều khiển xe tăng qua hai phương thức HandleMoveBody và HandleTurretMovement của lớp TankController – hoàn toàn giống cơ chế điều khiển của người chơi – hành vi vật lý luôn nhất quán và công bằng khi so sánh các thuật toán. Bên cạnh đó, cả xe tăng người chơi và xe tăng địch đều dùng chung một bộ tham số chuyển động (TankMovementData) và di chuyển ở một tốc độ không đổi như nhau; nhờ vậy mọi khác biệt về kết quả giữa ba thuật toán chỉ đến từ chất lượng quyết định điều hướng chứ không phải từ chênh lệch tốc độ giữa các xe tăng.

Hình 4.4 mô tả chi tiết luồng dữ liệu MAPF, bắt đầu từ khâu nạp tệp bản đồ cho đến khi tạo ra chuyển động thực tế của từng agent trong một chu kỳ tái hoạch định. Đầu tiên, dữ liệu bản đồ được phân tích để trích xuất các ô lưới (grid) hợp lệ. Sau đó, hệ thống gán trạng thái cho các agent và đẩy vào lõi PIBT. Tại mỗi bước mô phỏng, thuật toán tính toán ô mục tiêu tiếp theo cho từng xe tăng sao cho không xảy ra va chạm. Cuối cùng, kết quả này được chuyển đổi thành tham số lực đẩy và góc xoay để truyền vào TankController thực thi chuyển động.

Biểu đồ lớp (Hình 4.3) và luồng dữ liệu (Hình 4.4) mô tả *cấu trúc tĩnh* của ba



Hình 4.4: Luồng dữ liệu MAPF: từ tập bản đồ đến chuyển động agent

biến thể. Phần hành vi động theo thời gian của từng thuật toán – thời điểm tái hoạch định, dữ liệu trao đổi ở mỗi bước và cách áp dụng hành động trả về – được trình bày kèm phân tích đóng góp ở Chương 5, gồm biểu đồ trình tự của A* (Hình 5.1), của PIBT C# (Hình 5.2) và của cầu nối PIBT C++/TCP (Hình 5.3). Cách bố trí này tránh lặp lại cùng một biểu đồ ở hai chương, đồng thời giữ cho mục thiết kế tập trung vào quan hệ giữa các lớp.

4.2.3 Thiết kế cơ sở dữ liệu

Dự án không sử dụng cơ sở dữ liệu quan hệ mà lưu trữ dữ liệu dưới hai dạng tệp. Tệp bản đồ .map theo định dạng văn bản chuẩn MovingAI MAPF benchmark, với header gồm type, height, width, map và phần thân là ma trận ký tự (. cho ô đi được, @ cho tường), không thay đổi trong thời gian chạy. Kết quả backtest được ghi ra hai tệp CSV sau mỗi đợt chạy, có cấu trúc trường tóm tắt trong Bảng 4.1: tệp backtest_summary_*.csv lưu một dòng cho mỗi run (mức kịch bản), còn tệp backtest_agents_*.csv lưu một dòng cho mỗi agent trong mỗi run (mức tác nhân). Bốn trường khóa Run, Map, Algorithm, Rep xuất hiện ở cả hai tệp, cho phép ghép (join) dữ liệu hai mức khi phân tích.

Bảng 4.1: Cấu trúc trường của hai tệp CSV kết quả backtest

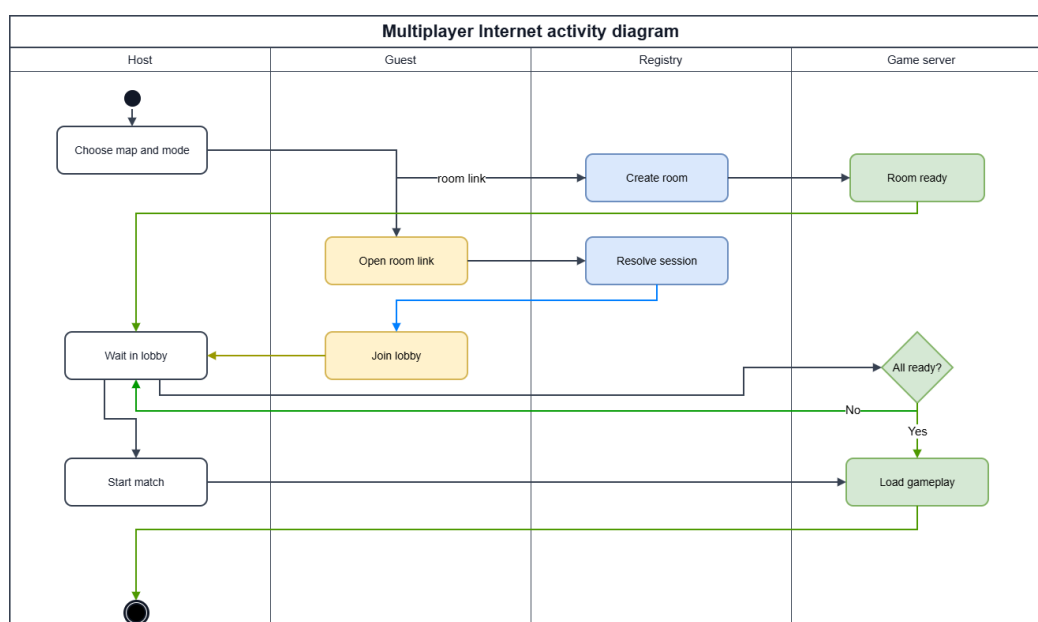
Tệp	Các trường chính
backtest_summary	Run, Map, Algorithm, Rep, Outcome, Duration_s, EagleHP, EagleHPLostPct, AgentCount, EnemiesAlive, TotalReplans, TotalRecoveries, TotalShots, TotalCells
backtest_agents	Run, Map, Algorithm, Rep, Outcome, AgentName, Replans, Recoveries, Shots, CellsVisited, InitialPathLen, DeadAtEnd

Tệp tóm tắt phục vụ so sánh tổng thể giữa các thuật toán (các biểu đồ trong mục 4.4), còn tệp mức agent phục vụ phân tích sâu hành vi từng xe tăng, ví dụ độ dài đường đi ban đầu so với số ô thực tế đã đi qua. Cách lưu phẳng bằng CSV giúp dữ

liệu đọc trực tiếp được bằng các công cụ bảng tính và thư viện Python mà không cần thiết lập máy chủ cơ sở dữ liệu.

4.2.4 Thiết kế luồng mạng nhiều người chơi

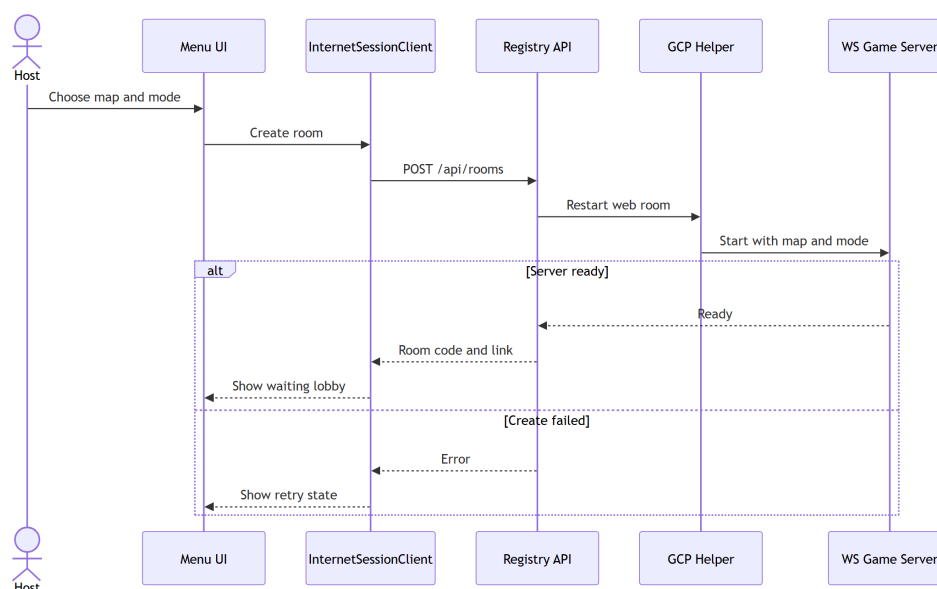
Chế độ nhiều người chơi được thiết kế theo hướng kết nối qua Internet để hai người chơi ở hai máy khác nhau có thể cùng tham gia một trận. Hình 4.5 mô tả sơ đồ hoạt động mức cao của luồng Host/Join: một người tạo phòng (Host) và người còn lại tham gia (Join) bằng mã phòng, sau đó cả hai vào phòng chờ trước khi trận bắt đầu. Bên cạnh đó, hệ thống vẫn giữ lại chế độ chơi mạng nội bộ (LAN) dựa trên framework Fish-Net cho trường hợp hai máy ở cùng mạng cục bộ; phần này tập trung mô tả nhánh Internet vì đây là luồng được sử dụng chính khi triển khai trực tuyến.



Hình 4.5: Sơ đồ hoạt động luồng Host/Join nhiều người chơi qua Internet

Quá trình kết nối qua Internet gồm hai giai đoạn. Giai đoạn tạo phòng được mô tả trong Hình 4.6: từ menu, lớp `InternetSessionClient` gửi yêu cầu tạo phòng tới dịch vụ đăng ký (Registry API); dịch vụ này yêu cầu thành phần GCP Helper khởi động một máy chủ trò chơi WebSocket với đúng bản đồ và chế độ. Nếu máy chủ sẵn sàng, hệ thống trả về mã phòng cùng đường liên kết và hiển thị phòng chờ cho Host; nếu thất bại, giao diện chuyển sang trạng thái cho phép thử lại.

Biểu đồ trong Hình 4.6 có sáu thành phần tham gia: người chơi giữ vai trò Host, lớp giao diện Menu, `InternetSessionClient` ở phía client, dịch vụ Registry API, thành phần GCP Helper và máy chủ trò chơi WebSocket. Ở luồng thành công (nhánh *Server ready*), yêu cầu tạo phòng đi tuần tự qua từng lớp: Host chọn bản đồ và chế độ trên Menu, `InternetSessionClient` gọi `POST /api/rooms` tới

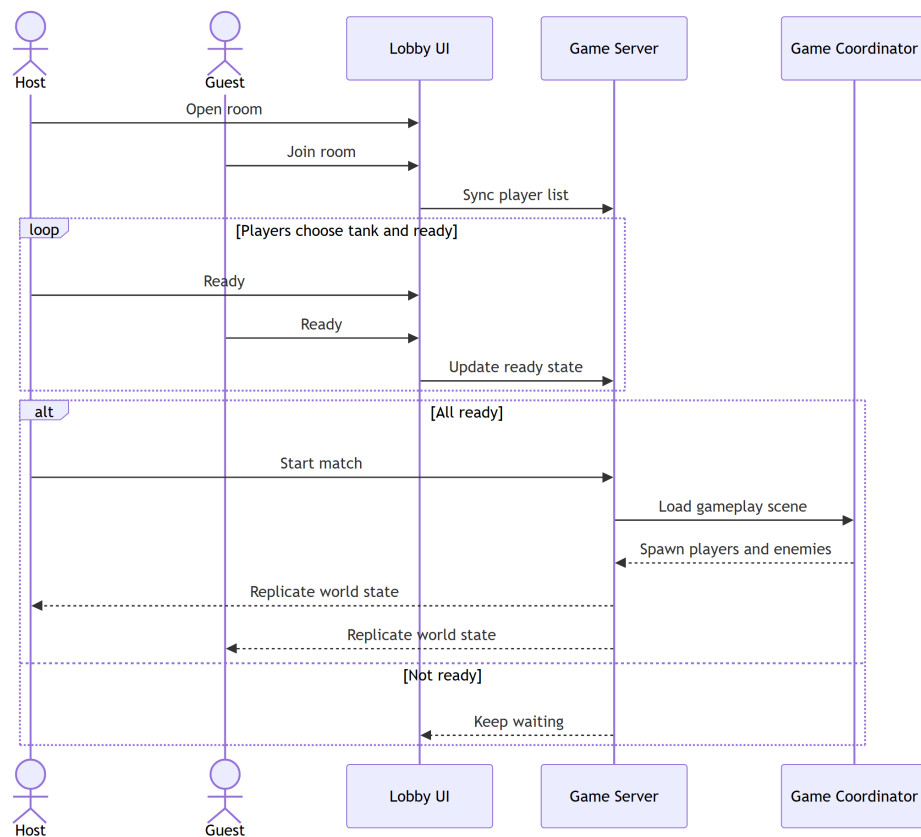


Hình 4.6: Biểu đồ trình tự tạo phòng chơi nhiều người qua Internet

Registry API, Registry yêu cầu GCP Helper khởi động lại một phòng web, Helper ra lệnh cho máy chủ WebSocket nạp đúng bản đồ và chế độ; máy chủ báo *Ready* ngược về Registry, và mã phòng cùng đường liên kết được trả về để client mở phòng chờ. Ở nhánh *Create failed*, lỗi được đẩy ngược về client và giao diện chuyển sang trạng thái cho phép thử lại thay vì treo. Việc tách Registry và Helper khỏi máy chủ trò chơi giúp một dịch vụ đăng ký nhẹ quản lý được vòng đời nhiều phòng mà không phải giữ kết nối trần đầu, đồng thời cô lập phần cấp phát hạ tầng GCP khỏi logic gameplay.

Giai đoạn sẵn sàng và bắt đầu ván chơi được mô tả trong Hình 4.7. Sau khi người chơi tham gia phòng, danh sách người chơi được đồng bộ qua máy chủ; mỗi người chọn xe tăng và bấm sẵn sàng, trạng thái này được cập nhật liên tục. Khi tất cả đã sẵn sàng, Host bắt đầu trận: máy chủ nạp scene gameplay, sinh xe tăng người chơi và các agent đối thủ, sau đó liên tục đồng bộ (replicate) trạng thái thế giới về cho mọi máy để đảm bảo các bên nhìn thấy cùng một diễn biến trận đấu.

Biểu đồ trong Hình 4.7 tập trung vào pha đồng bộ trạng thái phòng chờ. Sau khi Host mở phòng và Guest tham gia, máy chủ trò chơi duy trì một danh sách người chơi chung và phát lại (broadcast) cho mọi máy. Vòng lặp *Players choose tank and ready* cho thấy mỗi khi một người đổi xe tăng hoặc bật/tắt sẵn sàng, trạng thái được gửi lên máy chủ rồi cập nhật cho tất cả, nhờ đó hai máy luôn thấy cùng một danh sách. Điều kiện bắt đầu trận được kiểm soát ở nhánh *All ready*: chỉ khi mọi người đã sẵn sàng, Host mới kích hoạt *Start match*; máy chủ nạp scene gameplay, Game Coordinator sinh xe tăng người chơi và các agent đối thủ, sau đó máy chủ liên tục đồng bộ (replicate) trạng thái thế giới về cả Host lẫn Guest. Nếu còn người chưa



Hình 4.7: Biểu đồ trình tự sẵn sàng và bắt đầu ván chơi nhiều người qua Internet

sẵn sàng (nhánh *Not ready*), hệ thống tiếp tục chờ; đây cũng là cơ chế bảo đảm không máy nào vào trận sớm hơn máy khác.

4.3 Xây dựng ứng dụng

4.3.1 Công cụ và thư viện sử dụng

Bảng 4.2 liệt kê các công cụ, thư viện và phiên bản sử dụng trong dự án.

Bảng 4.2: Công cụ và thư viện sử dụng

Mục đích	Công cụ / Thư viện	Phiên bản
Engine trò chơi	Unity Engine (URP 2D)	6000.3.10f1
Ngôn ngữ lập trình	C#	.NET 8 (IL2CPP)
Thuật toán MAPF phía server	PIBT/EPIBT (C++, WSL Ubuntu)	commit 3f89632
Giao tiếp Unity – C++ server	TCP socket (127.0.0.1:7777)	–
Chuẩn bản đồ thực nghiệm	MovingAI MAPF benchmark	2019
Môi trường phát triển	Visual Studio 2022 + Rider	17.x / 2024.x
Kiểm soát phiên bản	Git	2.44
Xuất / vẽ biểu đồ	Python 3.14 + matplotlib 3.11	–

4.3.2 Kết quả đạt được

Bảng 4.3 tổng hợp thống kê mã nguồn C# của hệ thống.

Về mặt phần mềm, đồ án đã hoàn thành một số kết quả cụ thể. Hệ thống đọc

Bảng 4.3: Thống kê mã nguồn C# (tháng 6/2026)

Mô-đun	Tệp	Dòng lệnh
Lỗi MAPF AI (A*, PIBT, TCP client)	8	3 572
Bootstrap và cấu hình kịch bản	6	3 617
Backtest (runner, UI, spawner)	4	1 820
Gameplay (controller, mover, turret, đạn)	12	2 648
Lobby và giao diện mạng	9	3 460
Các lớp hỗ trợ (pool, damage, util)	38	3 006
Tổng cộng	77	18 123

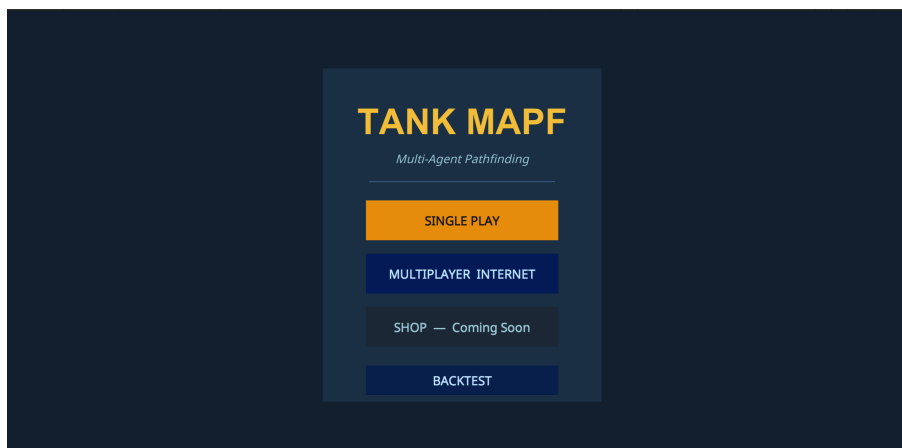
và dựng bản đồ động từ tệp .map chuẩn MAPF, hỗ trợ năm bản đồ kích thước từ 32×32 đến 251×180 ô. Ba thuật toán tìm đường đa tác nhân gồm A*, PIBT C# và PIBT C++/TCP hoạt động ổn định trên cùng năm bản đồ với số agent khảo sát từ 6 đến 72. Hệ thống backtest tự động chạy được 900 run (450 tĩnh và 450 động) và xuất kết quả ra CSV cùng biểu đồ. Chế độ chương ngại động hoạt động thông qua DynamicObstacleSpawner, ép agent phải tái hoạch định ngay lập tức khi đường đi bị chặn. Ngoài ra, hệ thống còn hỗ trợ chế độ nhiều người chơi qua Internet (và mạng LAN nội bộ dựa trên framework Fish-Net).

4.3.3 Minh họa các chức năng chính

Phần này minh họa giao diện thực tế của sản phẩm qua ảnh chụp màn hình, theo trình tự từ menu, luồng chơi đơn, luồng chơi nhiều người đến màn hình kết thúc trận. Các use case đã được đặc tả ở Chương 2; phần này cho thấy hiện thực tương ứng.

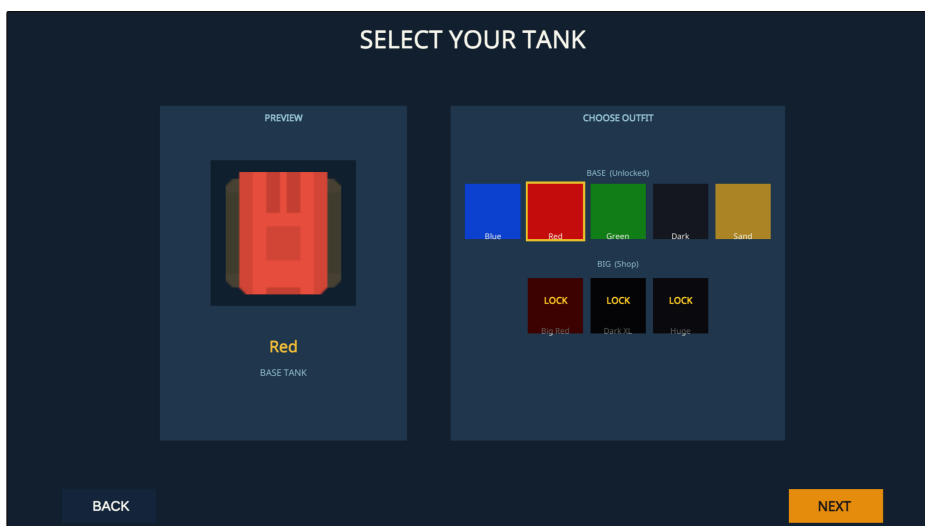
Màn hình menu chính. Hình 4.8 là điểm vào của ứng dụng. Người chơi chọn **SINGLE PLAY** để vào chế độ chơi đơn (nhóm use case A) hoặc **MULTIPLAYER INTERNET** để vào chế độ nhiều người (nhóm use case B); nút mở backtest chỉ xuất hiện ở bản chạy local/Editor.

Về bố cục, màn hình menu được tổ chức tối giản quanh tiêu đề **TANK MAPF** kèm phụ đề *Multi-Agent Pathfinding*, phía dưới là bốn mục xếp dọc. Hai mục đầu **SINGLE PLAY** và **MULTIPLAYER INTERNET** là lối vào hai nhóm chức năng chính của sản phẩm. Mục **SHOP** mang nhãn *Coming Soon* và đang vô hiệu hóa, giữ chỗ cho hướng mở rộng kho giao diện xe tăng. Mục **BACKTEST** mở thẳng công cụ thực nghiệm tự động được mô tả ở Mục 4.4 và chỉ hiện trên bản chạy local/Editor; bản WebGL công khai ẩn mục này để người chơi thông thường không kích hoạt nhằm tiến trình chạy hàng trăm kịch bản. Việc quy mọi lối vào về một màn hình giúp chuyển đổi liền mạch giữa trải nghiệm chơi và quy trình đánh giá định lượng trong cùng một ứng dụng.



Hình 4.8: Màn hình menu chính với hai chế độ SINGLE PLAY và MULTIPLAYER INTERNET

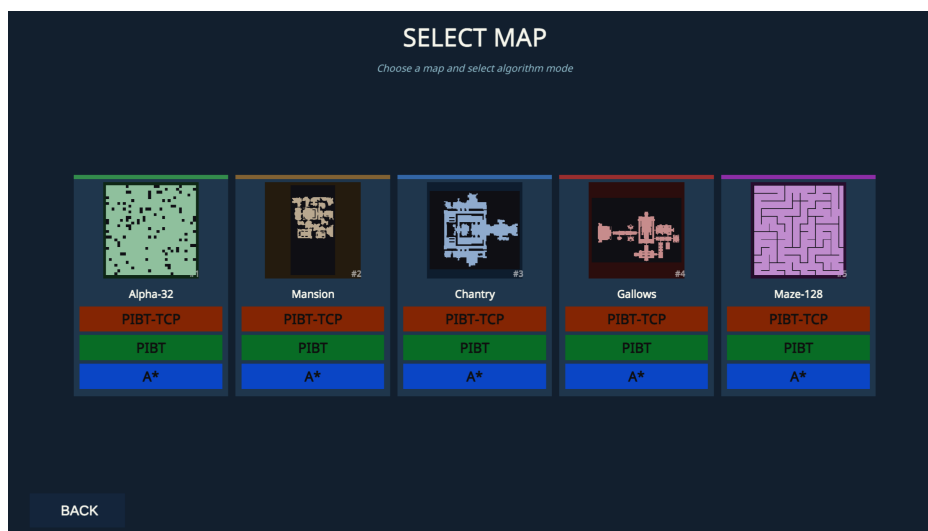
Luồng chơi đơn. Sau khi chọn SINGLE PLAY, người chơi trước hết chọn mẫu và màu xe tăng (Hình 4.9) – đúng trình tự đặc tả ở use case UC-A1.



Hình 4.9: Chế độ chơi đơn – màn chọn mẫu/màu xe tăng

Màn hình **SELECT YOUR TANK** ở Hình 4.9 chia thành hai khu. Khu *PREVIEW* bên trái dựng mô hình xe tăng kèm tên màu đang chọn (ví dụ *Red – Base Tank*), cho người chơi xem trước ngay diện mạo sẽ dùng trong trận. Khu *CHOOSE OUTFIT* bên phải xếp bảng màu thành hai hàng: hàng *Base* gồm các màu cơ bản đã mở khóa (Blue, Red, Green, Dark, Gold) và một hàng cao cấp đang ở trạng thái **LOCK** – giữ chỗ cho phần mở rộng kho giao diện về sau. Hai nút **BACK** và **NEXT** ở chân màn hình duy trì luồng thiết lập theo thứ tự tuyến tính. Tách riêng bước chọn xe tăng giúp dữ liệu ngoại hình – vốn chỉ mang tính hiển thị – không trộn lẫn với các cấu hình ảnh hưởng tới trận đấu ở bước kế tiếp. Sau khi xác nhận, người chơi chuyển sang chọn bản đồ và thuật toán AI (Hình 4.10).

Màn hình trong Hình 4.10 gộp hai lựa chọn vào cùng một bước: mỗi thẻ bản đồ



Hình 4.10: Chế độ chơi đơn – màn chọn bản đồ và thuật toán AI

hiển thị ảnh thu nhỏ của một bản đồ benchmark MovingAI (Alpha-32, Mansion, Chantry, Gallows, Maze-128) kèm ngay bên dưới ba nút thuật toán A*, PIBT và PIBT-TCP, nên người chơi chọn đồng thời bản đồ và thuật toán điều khiển agent chỉ bằng một thao tác. Đây đúng là năm bản đồ và ba thuật toán được dùng trong thực nghiệm ở mục 4.4; nhờ vậy kịch bản chơi đơn và kịch bản backtest chạy trên cùng một tập cấu hình, người chơi có thể quan sát trực quan chính các tình huống mà phân đánh giá định lượng đo đạc.

Khi vào trận (Hình 4.11), toàn bộ tường và vật cản được dựng từ tệp .map tại thời điểm chạy; người chơi điều khiển xe tăng phòng thủ Eagle Base trong khi các agent AI tự tìm đường tấn công.

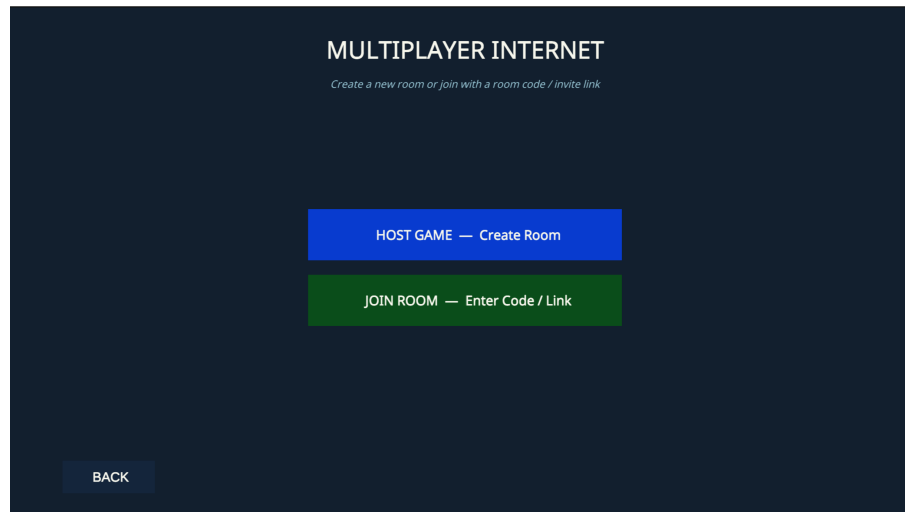


Hình 4.11: Một trận chơi đơn đang diễn ra

Trong cảnh trận ở Hình 4.11, các khối xám là tường và vật cản được dựng từ tệp .map tại thời điểm chạy theo đúng bố cục bản đồ đã chọn, còn ô vuông màu vàng

hoặc màu trắng là Eagle Base mà người chơi phải bảo vệ. Phần HUD ở góc trên hiển thị thanh máu của Eagle và số đếm các xe tăng địch còn sống. Các xe tăng địch do agent AI điều khiển tự tìm đường tiến về Eagle Base; đường đi dự kiến của chúng hiện lên thành các vết nét đứt mờ trên sân, cho thấy thuật toán MAPF đang chạy theo thời gian thực. Người chơi điều khiển xe tăng của mình di chuyển và bắn để chặn các agent trước khi Eagle Base bị phá hủy.

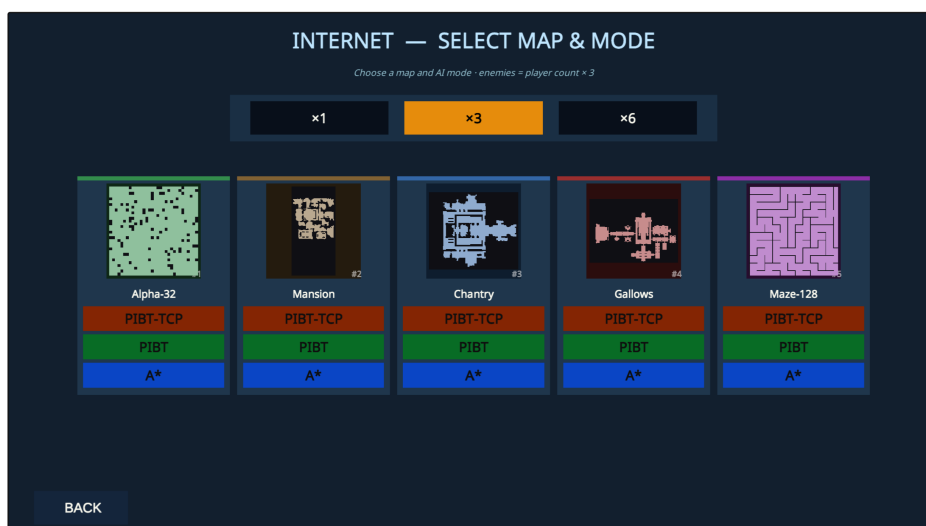
Luồng chơi nhiều người. Khi chọn MULTIPLAYER INTERNET, người chơi trước hết chọn vai trò Host hoặc Join (Hình 4.12).



Hình 4.12: Chế độ nhiều người – chọn vai trò Host/Join

Màn hình **MULTIPLAYER INTERNET** ở Hình 4.12 tách luồng nhiều người thành hai vai trò rõ ràng. Nút **HOST GAME – Create Room** (màu xanh dương) tạo một phòng mới và đưa người chơi vào vai trò chủ phòng; nút **JOIN ROOM – Enter Code / Link** (màu xanh lá) cho phép tham gia một phòng có sẵn bằng mã phòng hoặc đường liên kết mời được chia sẻ qua kênh ngoài. Phụ đề trên màn hình nhắc lại cơ chế kết nối bằng mã phòng/liên kết, còn nút **BACK** đưa người chơi về menu chính. Sự phân tách Host/Join này ánh xạ trực tiếp tới mô hình đăng ký phòng trong sơ đồ hoạt động ở Hình 4.5: phía Host khởi tạo phiên trên máy chủ, còn phía Join chỉ cần mã để khớp đúng phiên đó mà không phải tự dựng phòng. Sau khi chọn vai trò, người chơi cấu hình bản đồ và số agent đối thủ (Hình 4.13).

Màn hình **INTERNET – SELECT MAP & MODE** ở Hình 4.13 gộp ba quyết định cấu hình vào một bước. Dải nút $\times 1 / \times 3 / \times 6$ ở trên cùng đặt bội số quân địch: số agent đối thủ bằng bội số này nhân với số người chơi (công thức $enemies = player\ count \times multiplier$ hiển thị ngay trên màn hình), nên phòng càng đông thì số agent trong trận càng tăng tương ứng. Bên dưới là năm thẻ bản đồ benchmark MovingAI (Alpha-32, Mansion, Chantry, Gallows, Maze-128), mỗi thẻ kèm ba nút



Hình 4.13: Chế độ nhiều người – chọn bản đồ và số agent đối thủ

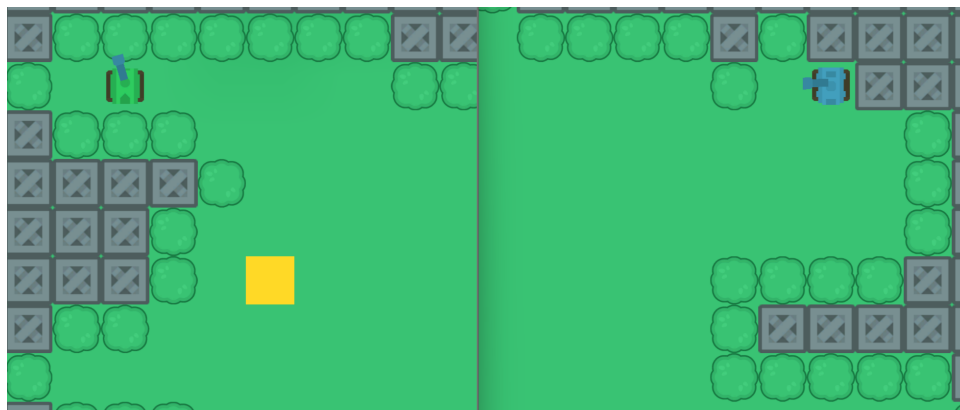
thuật toán **PIBT-TCP**, **PIBT** và **A*** để chọn đồng thời bản đồ lẫn thuật toán điều khiển agent chỉ bằng một thao tác – đúng tập cấu hình dùng trong thực nghiệm ở Mục 4.4. Sau khi cấu hình xong, chủ phòng (Host) quản lý phòng chờ và chỉ bắt đầu trận khi mọi người đã sẵn sàng (Hình 4.14).



Hình 4.14: Phòng chờ nhiều người phía chủ phòng (Host)

Hình 4.14 là phòng chờ phía Host: hiển thị mã phòng để chia sẻ, danh sách người chơi cùng trạng thái sẵn sàng, bảng chọn màu xe tăng và các nút **READY**, **START** và **CANCEL**; nút **START** chỉ bật khi đủ số người tối thiểu và tất cả người chơi đã **READY**. Khi trận bắt đầu, Hình 4.15 minh họa một trận nhiều người với trạng thái xe tăng, đạn và Eagle được đồng bộ giữa các máy.

Trong Hình 4.15, mỗi người chơi điều khiển xe tăng của mình trên cùng một bản đồ; vị trí xe tăng, đường đạn và máu của Eagle Base được đồng bộ theo thời gian thực giữa các máy để mọi bên cùng thấy một diễn biến trận đấu.

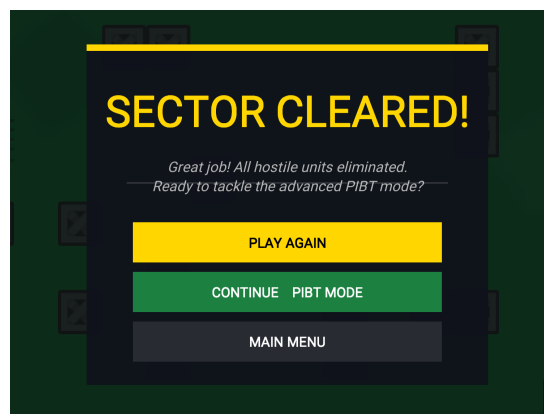


Hình 4.15: Một trận nhiều người đang diễn ra

Kết thúc trận. Khi Eagle Base bị phá hủy, người chơi thua (Hình 4.16a); khi tiêu diệt hết agent mà Eagle còn máu, người chơi thắng (Hình 4.16b) – tương ứng các nhánh của use case UC-A3.



(a) Thua trận khi Eagle Base bị phá hủy



(b) Thắng trận khi tiêu diệt hết agent

Hình 4.16: Hai màn hình kết thúc trận của chế độ chơi đơn

Ngoài ra, khi người dùng chuyển thuật toán giữa A*, PIBT và PIBT TCP, toàn bộ kịch bản được spawn lại từ đầu mà không cần đóng mở Unity Editor; trong chế độ backtest, màn hình hiển thị log tiến độ theo thời gian thực gồm số lần replan, trạng thái kết nối TCP server và thời gian còn lại của mỗi run.

4.4 Kiểm thử và đánh giá

4.4.1 Cấu hình thực nghiệm

Toàn bộ số liệu thời gian trong chương này được đo trên một máy tính xách tay dùng CPU AMD Ryzen AI 9-365 (kiến trúc Zen 5, 10 nhân/20 luồng, xung nhịp cơ bản khoảng 2 GHz và tối đa khoảng 5 GHz, 24 MB cache L3, TDP mặc định 28 W), 32 GB RAM LPDDR5x-8000 onboard, ổ SSD NVMe PCIe 4.0 dung lượng 1 TB và GPU tích hợp AMD Radeon 880M, chạy trên Windows 11 với Unity 6000.3.10f1. Mọi giá trị thời gian hoàn thành báo cáo dưới đây vì vậy gắn với cấu hình tham

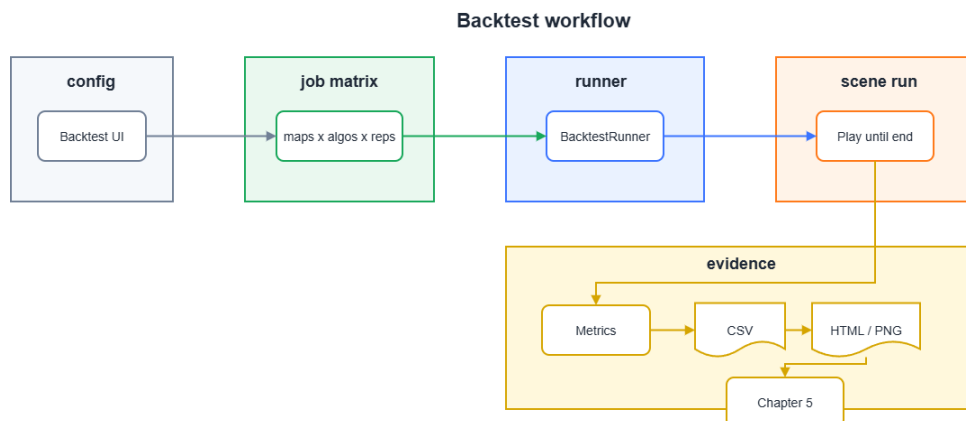
chiều này.

Thực nghiệm được thực hiện theo phương pháp backtest tự động (automated play-out): agent AI kiểm soát toàn bộ xe tăng địch và hệ thống tự ghi lại các chỉ số sau mỗi kịch bản mà không cần người dùng can thiệp. Bảng 4.4 liệt kê cấu hình đầy đủ của thực nghiệm.

Bảng 4.4: Cấu hình thực nghiệm

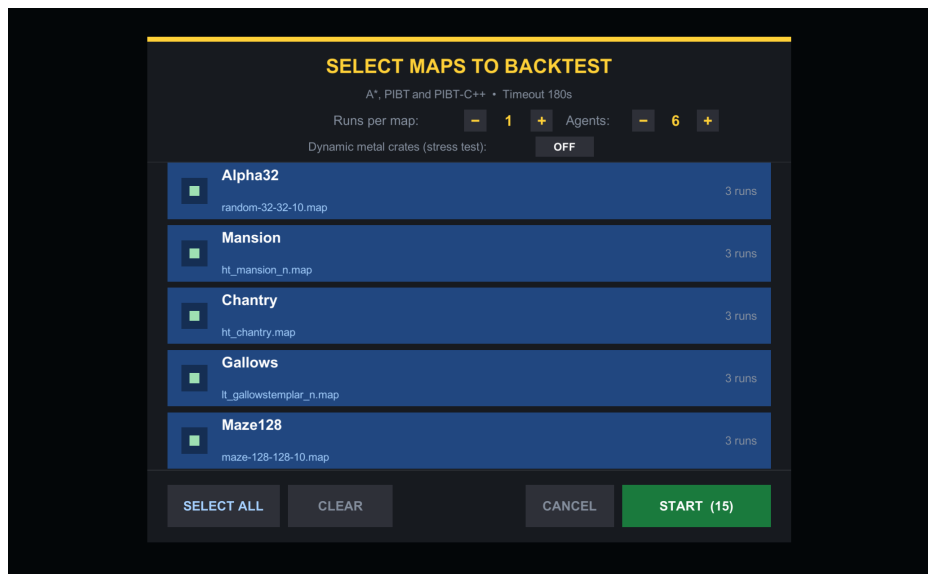
Tham số	Giá trị
Bản đồ	Alpha32, Mansion, Chantry, Gallows, Maze128
Thuật toán so sánh	A* (baseline), PIBT C#, PIBT C++/TCP
Số agent (xe tăng địch)	6, 12, 24, 36, 72
Tốc độ xe tăng	Không đổi và đồng nhất cho cả người chơi lẫn xe tăng địch (dùng chung TankMovementData)
Số lần lặp mỗi tổ hợp	6
Timeout mỗi run	180 giây
Tổng số run	5 bản đồ $\times$ 3 thuật toán $\times$ 5 mức agent $\times$ 6 lần $\times$ 2 môi trường = 900 run
Môi trường tĩnh	Không có chướng ngại di chuyển
Môi trường động	DynamicObstacleSpawner sinh rồi hủy thùng theo nhịp cố định ngay trên đường đi của agent (tốc độ xuất hiện và biến mất không đổi), mỗi thùng tồn tại <code>crateLifetime = 8 s</code>

Các chỉ số thu thập sau mỗi run gồm: thời gian hoàn thành (*Duration*), tổng số lần tái hoạch định (*TotalReplans*), số lần phục hồi sau kẹt (*TotalRecoveries*), tổng số phát bắn (*TotalShots*), tổng số ô đã đi qua (*TotalCells*), điểm máu Eagle còn lại (*EagleHP*) và kết quả kịch bản (*Outcome*: *EagleDestroyed* hoặc *Timeout*). Toàn bộ quy trình từ lúc khởi động đến khi xuất tệp CSV được tóm tắt ở Hình 4.17.



Hình 4.17: Quy trình thực nghiệm backtest tự động

Toàn bộ cấu hình được điều khiển qua một giao diện duy nhất, minh họa ở Hình 4.18.



Hình 4.18: Giao diện cấu hình backtest: chọn bản đồ, số lần chạy, số lượng agent và bật/tắt chương ngại động

Qua giao diện này, người dùng chọn tập bản đồ, số lần chạy mỗi bản đồ, **số lượng agent** và bật/tắt chế độ chương ngại động. Khả năng quét số lượng agent ngay trên giao diện chính là cơ sở để khảo sát khả năng mở rộng ở Mục 4.4.5, với năm mức 6, 12, 24, 36 và 72 agent.

Năm mức agent được chọn theo cấp số nhân xấp xỉ (mỗi mức gần gấp đôi mức trước) để bao quát dải tải rộng – từ vài agent tới mật độ cao – mà vẫn giữ số tổ hợp ở mức quản lý được. Mỗi tổ hợp chạy sáu lần lặp nhằm giảm nhiễu ngẫu nhiên từ vị trí spawn và lấy trung bình ổn định; ngưỡng timeout 180 giây đủ dài để các lượt khả thi kịp phá Eagle nhưng vẫn cắt sớm những lượt bế tắc, giữ tổng thời gian chạy 900 run trong giới hạn thực tế. Hai yếu tố khác được cố định xuyên suốt để bảo đảm so sánh công bằng: tốc độ di chuyển của mọi xe tăng là một hằng số như nhau, nên khác biệt kết quả phản ánh chất lượng điều hướng chứ không phải lợi thế tốc độ; và trong chế độ động, nhịp xuất hiện – biến mất của thùng cũng không đổi, giữ cho cường độ nhiễu loạn đồng đều giữa các thuật toán cũng như giữa các lần lặp.

4.4.2 Ba thuật toán điều hướng và cách đo số lần tái hoạch định

Ba thuật toán được so sánh khác nhau ở *nơi giải quyết xung đột* và do đó ở *cách phát sinh lệnh tái hoạch định* (replan). Cả ba đều đếm replan trên cùng một đơn vị nhịp (một lần mỗi agent cho mỗi cửa sổ 0,75 giây); điểm khác biệt nằm ở phần replan *khẩn cấp* mà chỉ PIBT C# phát sinh thêm. Bảng 4.5 tóm tắt khác biệt này trước khi đi vào chi tiết từng thuật toán.

Bảng 4.5: Cơ chế phát sinh lệnh tái hoạch định của ba thuật toán

Thuật toán	Nơi giải xung đột	“Replan” đếm gì	Phương sai giữa các agent
A*	Độc lập từng agent	Chặn đường + chu kỳ định kỳ	Thấp, đồng đều
PIBT C#	Cục bộ trong Unity	Mỗi bước phân giải + thoát kẹt khi kẹt	Vừa phải, gần đều
PIBT C++/TCP	Máy chủ C++ tập trung	Nhịp đếm phía máy khách	Rất thấp, đồng đều

A* (baseline). Mỗi agent chạy một bộ tìm đường A* trên lưới một cách *độc lập* (lớp GridEnemyAgent). Agent chỉ tái hoạch định khi ô kế tiếp trên đường đi bị chặn hoặc khi hết một chu kỳ định kỳ `replanInterval`. Vì không có cơ chế phối hợp giữa các agent nên số lần replan của từng agent tương đối đồng đều và không xuất hiện đột biến (Hình 5.1).

PIBT C#. Bộ giải PIBT (Priority Inheritance with Backtracking) [4] chạy *cục bộ trong tiến trình Unity* (lớp GridEnemyAgentPIBT). Mỗi bước phân giải xung đột giữa các agent được tính là một lần replan; ngoài ra, khi một agent bị kẹt, cơ chế thoát kẹt của riêng agent đó có thể phát sinh thêm vài replan. Tuy nhiên trong thực nghiệm, mức tăng này nhỏ nên phân bố replan giữa các agent vẫn khá đều, chỉ nhỉnh hơn A* thay vì tạo ra đột biến lớn (Hình 5.2).

PIBT C++/TCP. Bộ giải PIBT chạy trên một *máy chủ C++ tập trung* kết nối qua TCP (lớp GridEnemyAgentPIBT_TCP). Unity gửi trạng thái và nhận lại hành động ở mỗi chu kỳ; phía Unity chỉ *đếm theo nhịp* (cadence) nên số replan của các agent rất đồng đều. Lưu ý quan trọng: chi phí phối hợp và đệ quy bên trong máy chủ C++ không được trả về qua giao thức, nên con số replan của biến thể này chỉ phản ánh nhịp đếm phía máy khách, không bao gồm chi phí nội bộ máy chủ (Hình 5.3).

Hệ quả của ba cơ chế thể hiện ngay ở mức sáu agent: Bảng 4.6 trích số lần replan của từng agent trong một lượt chạy trên bản đồ Mansion.

Bảng 4.6: Số lần tái hoạch định của từng agent trong một lượt chạy trên bản đồ Mansion (6 agent)

Thuật toán	Số lần replan của 6 agent
A*	121, 122, 68, 76, 108, 121
PIBT C#	124, 124, 65, 70, 197 , 124
PIBT C++/TCP	90, 92, 89, 87, 102, 103

Bảng 4.6 cho thấy phân bố replan giữa các agent khá đều ở cả ba thuật toán. A* dao động trong khoảng 68–122 lần: các agent đã vào được vị trí khai hỏa replan ít

hơn vì dừng lại bắn, còn các agent vẫn đang cơ động thì replan nhiều hơn. PIBT C# phần lớn quanh 124 lần, riêng một agent nhỉnh lên 197 do cơ chế thoát kẹt cục bộ – một dao động nhẹ chứ không phải đột biến lớn. PIBT C++/TCP gần như bằng nhau (87–103) nhờ chạy theo nhịp đếm tập trung. Phân bố này khớp với cột “phương sai” trong Bảng 4.5.

4.4.3 Kết quả thực nghiệm – Môi trường tĩnh

Bảng 4.7 tổng hợp kết quả trung bình trên 6 lần lặp ở mức sáu agent trong môi trường tĩnh (không có chương ngại động).

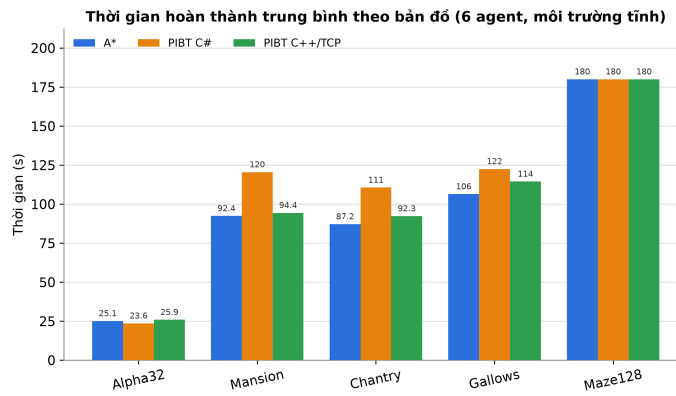
Bảng 4.7: Kết quả thực nghiệm môi trường tĩnh (trung bình 6 lần lặp, 6 agent)

Bản đồ	Thuật toán	Thời gian (s)	Replan TB	Cells TB	Kết quả
Alpha32	A*	25,1	144	47	EagleDestroyed
Alpha32	PIBT C#	23,6	196	45	EagleDestroyed
Alpha32	PIBT C++/TCP	25,9	177	92	EagleDestroyed
Mansion	A*	92,4	530	183	EagleDestroyed
Mansion	PIBT C#	120,5	611	182	EagleDestroyed
Mansion	PIBT C++/TCP	94,4	598	400	EagleDestroyed
Chantry	A*	87,2	530	221	EagleDestroyed
Chantry	PIBT C#	110,6	556	221	EagleDestroyed
Chantry	PIBT C++/TCP	92,3	602	412	EagleDestroyed
Gallows	A*	106,4	624	192	EagleDestroyed
Gallows	PIBT C#	122,5	621	193	EagleDestroyed
Gallows	PIBT C++/TCP	114,5	640	448	EagleDestroyed
Maze128	A*	180,0	820	410	Timeout
Maze128	PIBT C#	180,0	928	410	Timeout
Maze128	PIBT C++/TCP	180,0	820	410	Timeout

Bảng cho thấy cả ba thuật toán đều phá được Eagle trên 4/5 bản đồ với thời gian xấp xỉ nhau, riêng Maze128 timeout. Hai khác biệt đáng chú ý – thời gian hoàn thành và số lần tái hoạch định – được minh họa lần lượt ở Hình 4.19 và Hình 4.20.

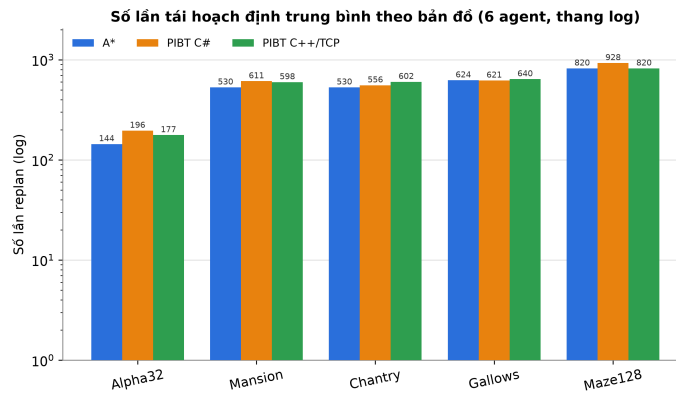
Xét chi tiết hơn, trên bản đồ thoáng Alpha32 cả ba thuật toán hoàn thành nhanh (khoảng 24–26 giây) với số lần tái hoạch định thấp, cho thấy khi không gian rộng thì khác biệt giữa chúng không đáng kể. Trên các bản đồ hẹp hơn (Mansion, Chantry, Gallows), thời gian tăng lên khoảng 90–120 giây và số lần replan của cả ba cùng ở mức vài trăm nhưng bám sát nhau – PIBT C# chỉ nhỉnh hơn A* chứ không bùng nổ – vì ở mật độ thấp (sáu agent) chưa phát sinh nhiều xung đột. Đáng chú ý, PIBT C++/TCP đi qua nhiều ô nhất ở hầu hết bản đồ (cột Cells TB) vì độ trễ một nhịp của kênh mạng khiến agent thỉnh thoảng chờ lệnh rồi phải điều chỉnh đường. Maze128 là trường hợp duy nhất cả ba cùng chạm trần 180 giây ở mức sáu agent, do quá ít xe tăng không kịp len qua mê cung lớn để tiếp cận và hạ Eagle.

Về thời gian, Hình 4.19 cho thấy ba thuật toán gần như trùng nhau trên các bản



Hình 4.19: Thời gian hoàn thành trung bình theo bản đồ (môi trường tĩnh)

đồ mở và cùng chạm trần 180 giây trên bản đồ mê cung Maze128.



Hình 4.20: Tổng số lần tái hoạch định trung bình theo bản đồ, thang logarit (môi trường tĩnh)

Về số lần tái hoạch định, Hình 4.20 (thang logarit) cho thấy ở mức sáu agent ba thuật toán bám khá sát nhau trên mọi bản đồ – PIBT C# và PIBT C++/TCP chỉ nhỉnh hơn A*, không tách thành các bậc cách biệt. Khác biệt giữa chúng chỉ nói rộng dần khi số agent tăng, được phân tích ở mục khảo sát khả năng mở rộng.

4.4.4 Kết quả thực nghiệm – Môi trường có chương ngại động

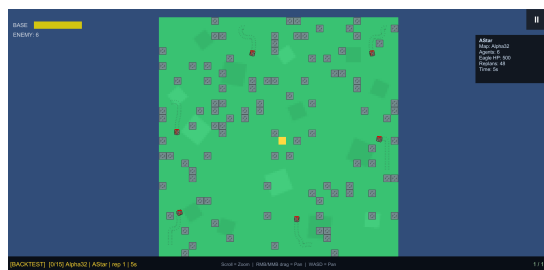
Bảng 4.8 tổng hợp kết quả khi bật chương ngại động (thùng xuất hiện ngay trên đường đi của agent).

Điểm mới của môi trường động là chiều *số lần phục hồi sau kẹt* (Recoveries): khi thùng kim loại xuất hiện ngay trên đường đi, agent có thể bị kẹt và phải kích hoạt cơ chế phục hồi. Số liệu cho thấy PIBT C# phục hồi nhiều nhất (31–62 lần) nhờ cơ chế thoát kẹt cục bộ, trong khi A* gần như không dùng cơ chế này (5–22 lần) mà chủ yếu tái hoạch định lại đường, còn PIBT C++/TCP ở mức trung gian (10–49 lần).

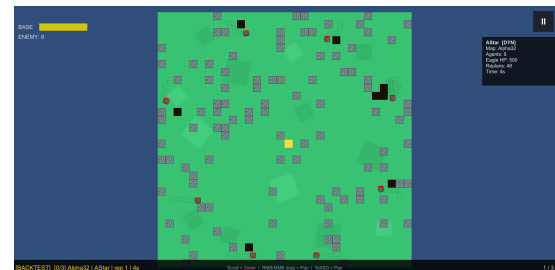
Bảng 4.8: Kết quả thực nghiệm môi trường có chướng ngại động (trung bình 6 lần lặp, 6 agent)

Bản đồ	Thuật toán	Thời gian (s)	Replan TB	Phục hồi TB	Cells TB	Kết quả
Alpha32	A*	28,3	165	5	48	EagleDestroyed
Alpha32	PIBT C#	28,3	286	62	35	EagleDestroyed
Alpha32	PIBT C++/TCP	30,5	258	10	78	EagleDestroyed
Mansion	A*	96,6	547	6	138	EagleDestroyed
Mansion	PIBT C#	138,0	702	31	138	EagleDestroyed
Mansion	PIBT C++/TCP	119,5	547	11	138	EagleDestroyed
Chantry	A*	97,6	638	7	177	EagleDestroyed
Chantry	PIBT C#	114,9	592	36	177	EagleDestroyed
Chantry	PIBT C++/TCP	129,5	938	49	398	EagleDestroyed
Gallows	A*	128,1	742	7	155	EagleDestroyed
Gallows	PIBT C#	177,6	944	58	155	EagleDestroyed
Gallows	PIBT C++/TCP	130,4	742	13	155	EagleDestroyed
Maze128	A*	180,0	901	22	353	Timeout
Maze128	PIBT C#	180,0	977	51	353	Timeout
Maze128	PIBT C++/TCP	180,0	1 300	44	568	Timeout

Ở các chiều còn lại, Bảng 4.8 cho thấy chướng ngại động không làm đảo cực diện thắng – thua: bốn trên năm bản đồ vẫn phá được Eagle, chỉ riêng mê cung Maze128 timeout giống hệt môi trường tĩnh, cho thấy giới hạn ở đây đến từ độ dài đường đi chứ không phải từ thùng động. Tuy nhiên thời gian hoàn thành và số lần tái hoạch định đều nhích lên so với môi trường tĩnh – ví dụ PIBT C# trên Mansion tăng từ khoảng 121 lên 138 giây – do mỗi lần thùng chắn đường buộc agent phải tính lại lộ trình. PIBT C++/TCP tiếp tục đi qua nhiều ô nhất (cột *Cells TB*) vì độ trễ một nhịp của kênh mạng khiến agent thường xuyên đi vòng trước khi nhận được ô mục tiêu mới.



(a) Tắt chướng ngại động

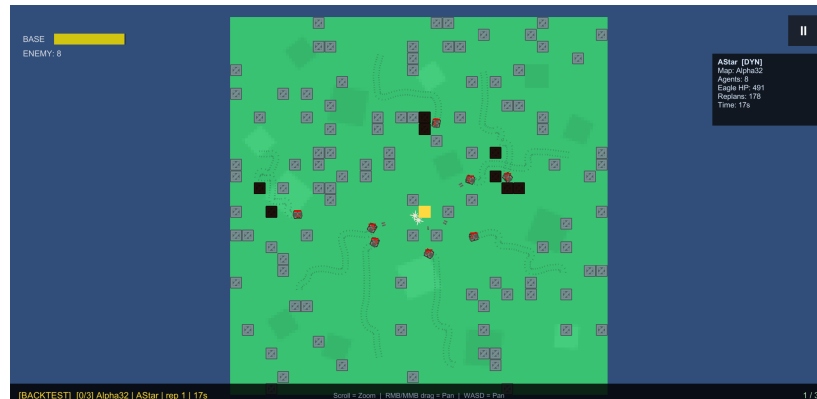


(b) Bật chướng ngại động

Hình 4.21: Backtest A* trên Alpha32: tắt (trái, chỉ thùng tĩnh màu tím) và bật (phải, xuất hiện thêm khối kim loại động màu đen) chế độ chướng ngại động

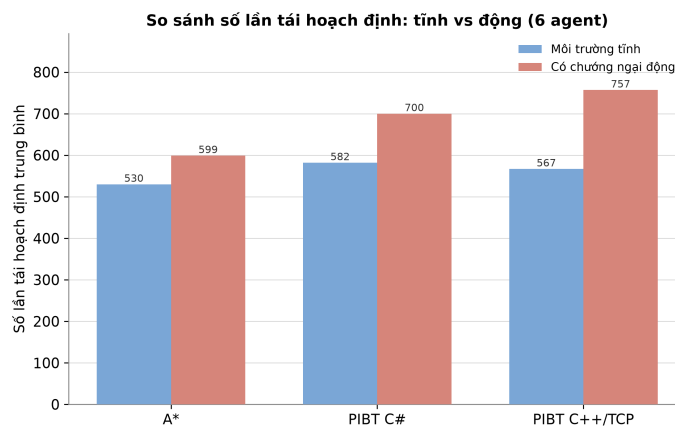
Hình 4.21 minh họa trực quan sự khác biệt trên bản đồ Alpha32. Ở ảnh trái (tắt chế độ động), bản đồ chỉ có các thùng tĩnh màu tím vốn là một phần cố định của tệp .map, nên tập chướng ngại không đổi suốt lượt chạy. Ở ảnh phải (bật chế độ động), hệ thống DynamicObstacleSpawner sinh thêm các khối kim loại màu đen ngay trên lối đi rồi thu hồi sau *crateLifetime*; mỗi lần một khối xuất hiện trùng

đường dự kiến, agent buộc phải tính lại lộ trình, tạo áp lực tái hoạch định liên tục mà môi trường tĩnh không có. Đây chính là kịch bản stress-test dùng để đo độ bền của ba thuật toán trước thay đổi đột ngột của môi trường.



Hình 4.22: Một lượt backtest A* với chương ngại động trên Alpha32: bảng chỉ số bên phải hiển thị số lần replan và máu Eagle theo thời gian

Hình 4.22 minh họa diễn tiến một lượt chạy đơn lẻ kèm bảng chỉ số thời gian thực ở góc phải. Tại mốc 17 giây, agent A* trên Alpha32 đã tích lũy 178 lần tái hoạch định và Eagle bắt đầu mất máu (còn 491/500 điểm) do agent phải liên tục vòng tránh các khối kim loại động. Bảng chỉ số này – gồm số lần replan, máu Eagle và thời gian đã trôi – được hệ thống backtest ghi lại theo từng bước rồi kết xuất ra tệp CSV; đây chính là nguồn dữ liệu thô để tổng hợp nên các biểu đồ ở những mục sau.



Hình 4.23: So sánh số lần tái hoạch định giữa môi trường tĩnh và có chương ngại động

Hình 4.23 so sánh trực tiếp tổng số lần tái hoạch định giữa hai điều kiện môi trường cho từng thuật toán, cho thấy chương ngại động làm tăng đáng kể số lần replan ở mọi thuật toán.

4.4.5 Khảo sát khả năng mở rộng theo số lượng agent

Các kết quả tại những mục trước mới dừng ở mức sáu agent. Để đánh giá khả năng mở rộng (scalability) của ba thuật toán khi tải tăng, mỗi kịch bản được lặp lại với năm mức số lượng agent: 6, 12, 24, 36 và 72. Mức tải này phản ánh đúng cơ chế của trò chơi, trong đó mỗi người chơi kéo theo một nhóm sáu xe tăng địch ($\text{EnemyCount} = 6 \times \text{số người chơi}$), nên 72 agent tương ứng kịch bản mười hai nhóm. Bảng 4.9 tổng hợp kết quả môi trường tĩnh theo mức agent, lấy trung bình trên cả năm bản đồ.

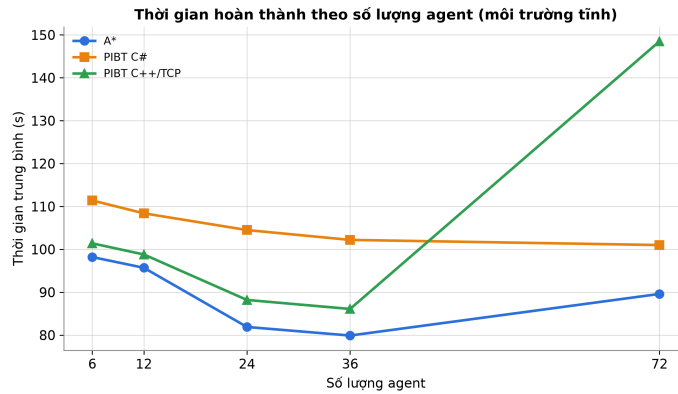
Bảng 4.9: Kết quả theo số lượng agent, môi trường tĩnh (trung bình trên 5 bản đồ $\times$ 6 lần lặp)

Số agent	Thuật toán	Phá Eagle (%)	Thời gian (s)	Replan TB	Cells TB
6	A*	80,0	98,2	530	211
6	PIBT C#	80,0	111,4	582	210
6	PIBT C++/TCP	80,0	101,4	567	352
12	A*	80,0	95,7	1 108	436
12	PIBT C#	80,0	108,4	1 219	435
12	PIBT C++/TCP	80,0	98,8	1 187	729
24	A*	100,0	81,9	2 321	903
24	PIBT C#	100,0	104,5	2 554	901
24	PIBT C++/TCP	100,0	88,2	2 483	1 510
36	A*	100,0	79,9	3 576	1 383
36	PIBT C#	100,0	102,2	3 936	1 379
36	PIBT C++/TCP	100,0	86,1	3 825	2 311
72	A*	80,0	89,6	7 490	2 863
72	PIBT C#	80,0	101,0	8 246	2 856
72	PIBT C++/TCP	20,0	148,5	8 005	4 786

Điểm cần lý giải trước tiên ở Bảng 4.9 là cột “Phá Eagle (%)” – tỉ lệ lượt mà nhóm tác nhân kịp hạ Eagle trước trần thời gian 180 giây. Tỉ lệ này không giảm đơn điệu mà *đạt đỉnh ở dải giữa*: 80% ở mức 6–12 agent, lên 100% ở mức 24–36 agent (với A* và PIBT C#), rồi lùi về 80% ở mức 72 agent. Nguyên nhân nằm ở bản đồ mê cung lớn Maze128. Ở mật độ thấp (6–12 agent), quá ít xe tăng không kịp len qua mê cung để tiếp cận và dồn đủ hỏa lực hạ Eagle trong 180 giây; ở mật độ vừa (24–36 agent), lực lượng đủ đông để vừa tiếp cận vừa tập trung hỏa lực nên phá được Eagle; nhưng ở mật độ rất cao (72 agent), các hành lang hẹp bắt đầu tắc nghẽn khiến Maze128 lại timeout. Bốn bản đồ còn lại (Alpha32, Mansion, Chantry, Gallows) được A* và PIBT C# giải ổn định ở mọi mức agent, nên dao động của tỉ lệ chung chủ yếu do Maze128 “lật trạng thái” theo mật độ – mỗi bản đồ đổi kết cục làm tỉ lệ trung bình trên năm bản đồ nhích đúng một nấc 20%. Riêng PIBT C++/TCP là biến thể yếu nhất: do phụ thuộc độ trễ trao đổi gói tin với máy chủ ngoài, ở mức 72 agent trên các bản đồ hẹp nó chạm trần thời gian sớm hơn,

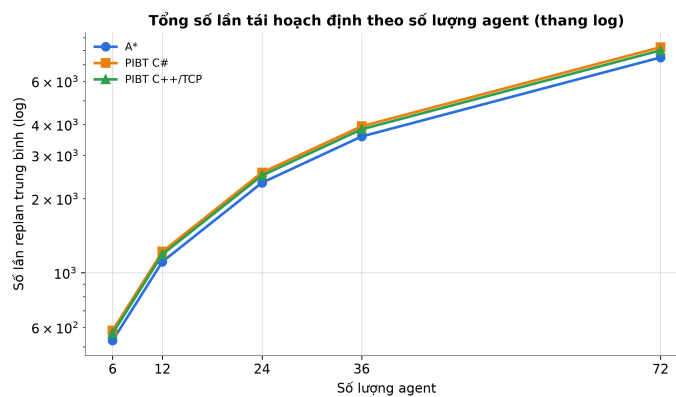
kéo tỉ lệ phá Eagle xuống còn 20%.

Ba xu hướng cơ chế đứng sau quy luật trên – thời gian hoàn thành, số lần tái hoạch định và tỉ lệ timeout – được làm rõ lần lượt ở Hình 4.24, Hình 4.25 và Hình 4.26.



Hình 4.24: Thời gian hoàn thành trung bình theo số lượng agent (môi trường tĩnh)

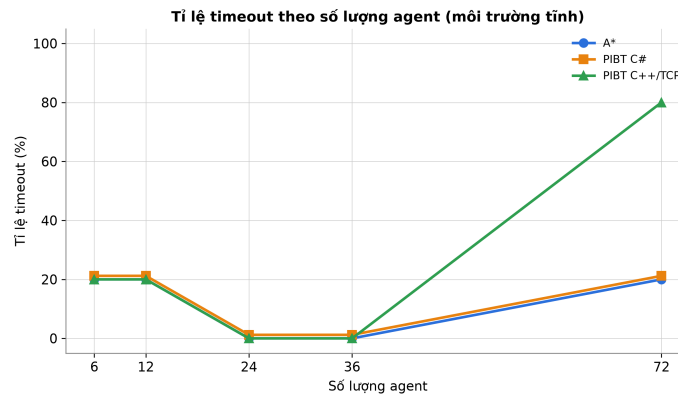
Về thời gian (Hình 4.24), cả ba thuật toán đều tăng nhẹ rồi bão hòa gần ngưỡng timeout 180 giây: với các lượt vẫn phá được Eagle, thời gian tăng từ khoảng 112 giây (6 agent) lên khoảng 150 giây (72 agent). PIBT C++/TCP luôn cao hơn hai biến thể còn lại một chút do độ trễ trao đổi gói tin TCP cộng dồn theo mỗi chu kỳ.



Hình 4.25: Tổng số lần tái hoạch định trung bình theo số lượng agent, thang logarit (môi trường tĩnh)

Về số lần tái hoạch định (Hình 4.25, thang logarit), điểm đáng chú ý là ba thuật toán bám khá sát nhau và cùng tăng *gần tuyến tính* theo số agent – mỗi agent tăng thêm đóng góp một lượng replan gần như cố định – chứ không tách thành các bậc cách biệt. Ở mức 72 agent, cả ba đều ở khoảng 7 500–8 300 lần. Đây là khác biệt quan trọng so với kỳ vọng theo lý thuyết rằng PIBT C# sẽ “bùng nổ” số lần tái hoạch định do cơ chế thoát kẹt: trong thực nghiệm, khâu phối hợp của PIBT C# hoạt động ổn định nên số lần replan của nó chỉ nhỉnh hơn A* thay vì cao gấp nhiều

lần [4]. PIBT C++/TCP cũng tăng tương tự; cần nhắc lại con số phía máy khách của biến thể này không bao gồm chi phí phối hợp bên trong máy chủ C++ [3].



Hình 4.26: Tỉ lệ timeout theo số lượng agent (môi trường tĩnh)

Về tỉ lệ timeout (Hình 4.26), ở mức 6–12 agent chỉ Maze128 timeout (ứng với tỉ lệ phá Eagle 80%), do mật độ quá thấp không đủ tiếp cận Eagle trong mê cung lớn. Ở mức 24–36 agent, ngay cả Maze128 cũng được giải nên A* và PIBT C# đạt 0% timeout. Chỉ khi lên 72 agent, tắc nghẽn mới khiến Maze128 timeout trở lại (A*/PIBT C# về 80% phá Eagle); riêng PIBT C++/TCP chạm trần sớm hơn trên các bản đồ hẹp vì thời gian mỗi chu kỳ trao đổi gói tin cao hơn. Như vậy tồn tại một “cửa sổ mật độ” thuận lợi: quá ít agent thì thiếu lực để tiếp cận, quá nhiều agent thì tắc nghẽn, và hiệu năng tốt nhất rơi vào dải trung gian – phù hợp với nhận định chung về độ khó của bài toán MAPF theo mật độ [1].

Trong môi trường có chương ngại động, hai xu hướng theo thời gian và tỉ lệ phá Eagle nêu trên vẫn giữ nguyên, nhưng xuất hiện thêm một chiều mới trở nên nổi bật: *số lần phục hồi sau kẹt*. Mỗi khi thùng kim loại sinh ra chướng ngại vật, agent có thể mắc kẹt và phải kích hoạt cơ chế thoát kẹt; tần suất kích hoạt này phân hóa rất mạnh giữa ba thuật toán và tăng nhanh theo mật độ. Bảng 4.10 tổng hợp kết quả môi trường động theo năm mức agent, giữ nguyên cấu trúc cột như bảng tĩnh nhưng thay cột *Cells* bằng cột *Phục hồi TB* để làm rõ chiều khác biệt này.

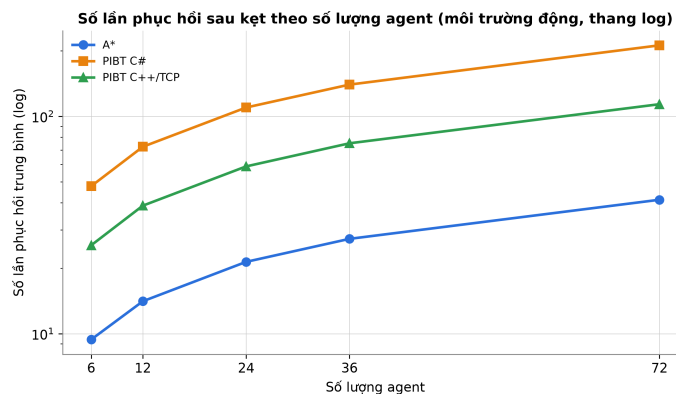
Bảng 4.10: Kết quả theo số lượng agent, môi trường có chương ngại động (trung bình trên 5 bản đồ × 6 lần lặp)

Số agent	Thuật toán	Phá Eagle (%)	Thời gian (s)	Replan TB	Phục hồi TB
6	A*	80,0	106,1	599	9
6	PIBT C#	80,0	127,7	700	48
6	PIBT C++/TCP	80,0	118,0	757	26
12	A*	80,0	103,3	1 253	14
12	PIBT C#	80,0	124,8	1 464	72
12	PIBT C++/TCP	80,0	114,7	1 587	39

(tiếp theo Bảng 4.10)

Số agent	Thuật toán	Phá Eagle (%)	Thời gian (s)	Replan TB	Phục hồi TB
24	A*	100,0	91,3	2 623	21
24	PIBT C#	100,0	115,2	3 062	110
24	PIBT C++/TCP	60,0	127,6	3 326	59
36	A*	100,0	89,1	4 041	27
36	PIBT C#	100,0	112,4	4 715	140
36	PIBT C++/TCP	20,0	149,5	5 128	75
72	A*	80,0	96,4	8 460	41
72	PIBT C#	80,0	115,7	9 866	212
72	PIBT C++/TCP	20,0	149,3	10 754	114

Bảng 4.10 cho thấy chương ngại động giữ nguyên dạng “cửa sổ mật độ” của tỉ lệ phá Eagle (đỉnh ở 24–36 agent với A* và PIBT C#) nhưng làm xuất hiện rõ một chiều mới: số lần phục hồi sau kẹt. So với môi trường tĩnh, thời gian hoàn thành và số lần replan đều nhích lên do thùng động liên tục chắn đường; khác biệt nổi bật nhất nằm ở số lần phục hồi – nơi ba thuật toán tách biệt mạnh nhất – được phân tích kỹ qua Hình 4.27.



Hình 4.27: Số lần phục hồi sau kẹt theo số lượng agent (môi trường động, thang logarit)

Hình 4.27 cho thấy số lần phục hồi cao nhất ở PIBT C#: cơ chế thoát kẹt cục bộ của biến thể này kích hoạt thường xuyên khi thùng động liên tục chặn đường, cao hơn A* và PIBT C++/TCP vài lần ở mọi mức agent. A* gần như không dùng cơ chế phục hồi (chỉ tái hoạch định lại đường), còn PIBT C++/TCP ở mức trung gian nhờ máy chủ xử lý phần lớn xung đột. Đây vừa là điểm linh hoạt (thoát kẹt tốt) vừa là điểm yếu (chi phí phục hồi bùng nổ khi tải lớn) của kiến trúc cục bộ.

Bảng 4.11 trình bày chi tiết số lần tái hoạch định theo từng bản đồ và từng mức agent trong môi trường tĩnh, đánh dấu † cho các lượt bị timeout. Có thể thấy rõ thứ tự PIBT C# cao nhất, kế đến A*, thấp nhất là PIBT C++/TCP trên các bản đồ hẹp, cũng như cách “vách timeout” lan dần từ những bản đồ khó (Chantry, Gallows) và xuất hiện sớm hơn ở PIBT C++/TCP.

Bảng 4.11: Số lần tái hoạch định theo bản đồ và số lượng agent (môi trường tĩnh); † = lượt timeout

Bản đồ	Số agent	A*	PIBT C#	PIBT C++/TCP
Alpha32	6	144	196	177
	12	298	406	366
	24	618	841	758
	36	946	1 288	1 160
	72	1 959	2 665	2 402
Mansion	6	530	611	598
	12	1 097	1 265	1 238
	24	2 272	2 619	2 563
	36	3 478	4 009	3 923
	72	7 201	8 301	8 123 [†]
Chantry	6	530	556	602
	12	1 097	1 151	1 247
	24	2 272	2 383	2 581
	36	3 478	3 648	3 951
	72	7 201	7 553	8 180 [†]
Gallows	6	624	621	640
	12	1 292	1 286	1 326
	24	2 674	2 663	2 746
	36	4 093	4 076	4 203
	72	8 475	8 440	8 703 [†]
Maze128	6	820 [†]	928 [†]	820 [†]
	12	1 758 [†]	1 988 [†]	1 758 [†]
	24	3 768	4 262	3 768
	36	5 886	6 658	5 886
	72	12 616 [†]	14 271 [†]	12 617 [†]

4.4.6 Nhận xét và đánh giá

Từ toàn bộ kết quả thực nghiệm, có thể rút ra một số nhận xét. Về thời gian hoàn thành, ba thuật toán cho kết quả xấp xỉ nhau trên các bản đồ mở, với chênh lệch lớn nhất không quá 10% ở mức sáu agent; hành vi cuối cùng là phá hủy Eagle Base tương đương nhau giữa ba thuật toán.

Về số lần tái hoạch định, ba thuật toán bám khá sát nhau và cùng tăng gần tuyến tính theo số agent, chứ không tách thành các bậc cách biệt. Cơ chế thoát kẹt cục bộ của PIBT C# về lý thuyết có thể đẩy số lần replan của nó lên cao hơn hẳn A*,

nhưng số liệu đo được cho thấy mức tăng nhỏ: phần lớn agent có số replan tương đương nhau, chỉ thỉnh thoảng một agent nhích lên do thoát kẹt cục bộ, nên tổng số lần replan của PIBT C# chỉ nhỉnh hơn A* (xem Bảng 4.6). A* replan khi bị chặn hoặc hết chu kỳ; PIBT C++/TCP đếm theo nhịp phía máy khách. Cần nhấn mạnh rằng định nghĩa “một lần replan” khác nhau giữa ba thuật toán (Bảng 4.5), nên đây là so sánh về nhịp tái hoạch định *phía máy khách*, và con số của PIBT C++/TCP không bao gồm chi phí phối hợp bên trong máy chủ; bởi vậy không thể kết luận tuyệt đối “thuật toán nào ít replan hơn thì tốt hơn”.

Về số ô đã đi qua, PIBT C++/TCP đi nhiều ô hơn hai thuật toán còn lại trên mọi bản đồ. Nguyên nhân là độ trễ TCP giữa Unity và máy chủ C++ khiến agent nhận lệnh chậm hơn một nhịp, dẫn đến nhiều bước chờ và chuyển hướng thừa.

Về khả năng mở rộng (Mục 4.4.5), khi tăng từ 6 lên 72 agent, số lần tái hoạch định tăng gần tuyến tính (mỗi agent thêm vào đóng góp một lượng gần cố định), còn thời gian hoàn thành biến động nhẹ quanh 80–150 giây. Tỷ lệ phá Eagle không suy giảm đơn điệu mà đạt đỉnh ở dải giữa (24–36 agent): quá ít agent thì thiếu lực tiếp cận mê cung lớn, quá nhiều agent thì các hành lang hẹp tắc nghẽn – tồn tại một “cửa sổ mật độ” thuận lợi, phù hợp với nhận định chung về độ khó của MAPF theo mật độ [1], [3].

Về tác động của chương ngại động, ngoài việc làm tăng nhẹ thời gian và số lần replan, môi trường động làm xuất hiện rõ chiều số lần phục hồi sau kẹt: PIBT C# phục hồi nhiều nhất nhờ cơ chế thoát kẹt cục bộ, A* gần như không phục hồi mà chỉ tính lại đường, còn PIBT C++/TCP ở mức trung gian. Kết quả xác nhận cả ba thuật toán đều ứng phó được khi môi trường thay đổi. Riêng bản đồ mê cung Maze128 kích thước 128×128 chỉ giải được ở dải 24–36 agent: ở mật độ quá thấp hoặc quá cao nó đều chạm trần thời gian, phản ánh giới hạn về tốc độ tiếp cận mục tiêu trên bản đồ mê cung lớn.

Tổng hợp lại, mỗi thuật toán có thế mạnh riêng: A* đơn giản, ổn định và có số lần tái hoạch định thấp nhưng không phối hợp giữa các agent; PIBT C# phối hợp và thoát kẹt tốt với chi phí replan chỉ nhỉnh hơn A*, đồng thời phục hồi tốt nhất trong môi trường động; PIBT C++/TCP tách phần lập kế hoạch sang máy chủ C++ nhưng là biến thể nhạy cảm nhất với độ trễ kết nối, nên yếu hơn ở mật độ cao và môi trường động.

4.5 Triển khai

Hệ thống được triển khai theo hai hình thức chính: chạy cục bộ trong Unity Editor phục vụ phát triển và thực nghiệm, và triển khai trực tuyến trên nền tảng Google Cloud Platform (GCP) phục vụ người dùng cuối qua trình duyệt. Sơ đồ

triển khai tổng thể được trình bày ở Hình 4.28. Các mục con dưới đây mô tả chi tiết từng thành phần.

4.5.1 Triển khai cục bộ

Trong quá trình phát triển và chạy thực nghiệm backtest, hệ thống được khởi động trực tiếp trong Unity Editor. Người dùng mở scene chính của dự án là `Assets/Scenes/Menu.unity` rồi nhấn Play; giao diện menu hiện ra cho phép chọn chế độ chơi đơn, chơi nhiều người hoặc vào backtest. Toàn bộ bản đồ, agent AI và Eagle Base được khởi tạo động tại thời điểm chạy dựa trên cấu hình bản đồ và thuật toán đã chọn. Với biến thể PIBT C++/TCP, máy chủ PIBT chạy trên cùng máy tính (WSL/Ubuntu), Unity kết nối qua TCP tại địa chỉ nội bộ `127.0.0.1:7777` nên kết quả backtest không bao gồm độ trễ mạng thực tế.

Hình thức cục bộ phục vụ hai mục đích: thứ nhất là môi trường phát triển nhanh, cho phép sửa mã nguồn và kiểm thử ngay trong Editor mà không cần quy trình build riêng; thứ hai là môi trường thực nghiệm chính thức, nơi toàn bộ 900 lượt backtest được thực hiện trong điều kiện kiểm soát.

4.5.2 Hạ tầng triển khai trên Google Cloud Platform

Để phục vụ người dùng cuối qua trình duyệt, hệ thống được triển khai trên Google Cloud Platform với project có định danh `tankmapf`, vùng `asia-southeast1` (Singapore) nhằm giảm thiểu độ trễ cho người dùng tại Việt Nam. Bảng 4.12 tóm tắt các thành phần hạ tầng chính.

Bảng 4.12: Thành phần hạ tầng GCP

Thành phần	Mô tả
Máy ảo (VM)	Compute Engine e2-medium (2 vCPU, 4 GB RAM), chạy Ubuntu, đặt tại <code>asia-southeast1-b</code>
Địa chỉ IP tĩnh	<code>google_compute_address</code> cấp phát IP công khai cố định, không đổi khi khởi động lại VM
Firewall	Cho phép TCP cổng 22 (SSH), 80 (HTTP), 443 (HTTPS), 7777 (game UDP/TCP), 7778 (WebSocket nội bộ), 8080 (Registry nội bộ)
GCS Bucket	<code>tankmapf-tank-mapf-artifacts</code> lưu trữ artifact build (server Linux, WebGL) theo mã commit
Terraform	Toàn bộ hạ tầng được khai báo dưới dạng mã (Infrastructure as Code) trong thư mục <code>infra/gcp/*.tf</code> , cho phép tái tạo hoàn toàn bằng lệnh <code>terraform apply</code>

Việc sử dụng Terraform đảm bảo tính tái lập: khi cần tạo lại môi trường từ đầu hoặc mở rộng sang vùng khác, toàn bộ hạ tầng có thể được dựng lên chỉ bằng một lệnh duy nhất mà không cần thao tác thủ công trên giao diện web của GCP.

4.5.3 Tên miền và chứng chỉ TLS

Ứng dụng WebGL sử dụng tên miền `luminx.io.vn` thay vì truy cập trực tiếp bằng địa chỉ IP. Ba bản ghi DNS loại A được cấu hình: `luminx.io.vn` (gốc), `www` và `game` đều trỏ về địa chỉ IP tĩnh của máy ảo GCP. Tên miền phụ game phục vụ riêng cho kết nối WebSocket của chế độ nhiều người chơi, giúp tách biệt luồng dữ liệu trò chơi thời gian thực khỏi luồng tải trang tĩnh.

Chứng chỉ TLS được cấp miễn phí bởi Let's Encrypt thông qua công cụ `certbot`, phủ cả ba tên miền trong cùng một chứng chỉ. Chứng chỉ tự động gia hạn nhờ tác vụ hẹn giờ (`systemd timer`) trên máy ảo. Việc sử dụng HTTPS là bắt buộc vì hai lý do kỹ thuật: thứ nhất, định dạng nén Brotli của bản WebGL chỉ được trình duyệt giải nén trên nguồn gốc an toàn (secure origin) – trên HTTP thuần, trình duyệt không gửi tiêu đề `Accept-Encoding: br` và không tự giải nén, dẫn tới lỗi không tải được tệp `.br`; thứ hai, trang HTTPS yêu cầu mọi kết nối WebSocket phải dùng giao thức `wss://` (WebSocket Secure), nếu không trình duyệt sẽ chặn do chính sách `mixed-content`.

4.5.4 Nginx – Reverse proxy và phục vụ WebGL

Nginx đóng vai trò lớp ngoài cùng tiếp nhận mọi yêu cầu từ trình duyệt, thực hiện bốn chức năng chính:

1. **Phục vụ tệp WebGL tĩnh qua HTTPS.** Bản WebGL được xuất với nén Brotli (`.js.br`, `.wasm.br`, `.data.br`). Nginx gán tiêu đề `Content-Encoding: br` cho các tệp này để trình duyệt giải nén đúng cách. Tệp được phục vụ từ thư mục `/opt/tank-mapf/web/current/` trên máy ảo.
2. **Proxy ngược cho dịch vụ Registry.** Nginx chuyển tiếp bốn nhóm đường dẫn tới dịch vụ Registry chạy trên cổng nội bộ 8080, gồm: `/api/sessions`, `/api/rooms`, `/s/` và `/create`. Registry là một ứng dụng Python nhẹ, quản lý vòng đời phòng chơi (tạo phòng, truy vấn phòng, tạo đường liên kết mời).
3. **Proxy WebSocket cho máy chủ trò chơi.** Tên miền phụ `game.luminx.io.vn` được cấu hình riêng một khối server lắng nghe cổng 443: Nginx thực hiện TLS termination rồi chuyển tiếp kết nối WebSocket tới máy chủ trò chơi chạy plaintext trên cổng nội bộ 7778. Nhờ vậy, máy chủ trò chơi không cần xử lý mã hóa TLS mà vẫn đảm bảo kết nối `wss://` từ phía trình duyệt.
4. **Chuyển hướng HTTP sang HTTPS.** Mọi yêu cầu tới cổng 80 được trả mã 301 chuyển hướng sang HTTPS. Ngoại lệ duy nhất là đường dẫn xác thực chứng chỉ `/.well-known/acme-challenge/` do Let's Encrypt yêu cầu.

Bảng 4.13 tóm tắt ba khối cấu hình server của Nginx trên máy ảo.

Bảng 4.13: Ba khối cấu hình Nginx trên máy ảo GCP

Khối	Cổng	Chức năng
Site tĩnh (HTTPS)	443	Phục vụ WebGL Brotli + proxy Registry (<code>luminx.io.vn</code> , <code>www</code>)
Game subdomain	443	TLS termination + proxy WebSocket tới 7778 (<code>game.luminx.io.vn</code>)
Redirect	80	Chuyển hướng HTTP → HTTPS, giữ ACME challenge

4.5.5 Máy chủ PIBT C++ và mô hình client-server TCP

Biến thể thứ ba của hệ thống AI (PIBT C++/TCP, lớp `GridEnemyAgent` `PIBT_TCP`) tách phần lập kế hoạch đường đi ra khỏi tiến trình Unity và giao cho một máy chủ C++ độc lập chạy thuật toán PIBT/EPIBT. Mã nguồn máy chủ nằm trong kho `Server-PIBT-TeamNoMan-sSky`, được biên dịch và chạy trên môi trường Linux (WSL/Ubuntu trên máy phát triển hoặc trực tiếp trên máy ảo GCP).

Giao thức giao tiếp giữa Unity và máy chủ PIBT là TCP đồng bộ theo nhịp (cadence-based): tại mỗi chu kỳ tái hoạch định, Unity gửi trạng thái hiện tại (vị trí lưới của toàn bộ agent) qua kết nối TCP, máy chủ C++ giải thuật toán rồi trả về ô mục tiêu tiếp theo cho từng agent. Lớp `PIBTtcpClient` xử lý kết nối cho bản chạy gốc (native), trong khi lớp `PIBTWebRelayClient` xử lý cho bản WebGL bằng cách chuyển tiếp lệnh qua hạ tầng WebSocket/HTTP.

Cần phân biệt hai ngữ cảnh kết nối:

- Trong thực nghiệm backtest cục bộ, máy chủ PIBT chạy trên cùng máy tính với Unity, kết nối qua `127.0.0.1:7777`; độ trễ TCP là không đáng kể và kết quả backtest phản ánh thuần hiệu năng thuật toán.
- Trên môi trường production, máy chủ PIBT chạy trên máy ảo GCP; lệnh lập kế hoạch từ trình duyệt của người dùng được chuyển tiếp qua hạ tầng WebSocket/Nginx tới tiến trình PIBT C++ trên cùng máy ảo. Độ trễ thực tế bao gồm cả trễ mạng Internet giữa người dùng và máy chủ.

Kiến trúc tách biệt này cho phép nâng cấp hoặc thay thế thuật toán phía máy chủ (ví dụ chuyển từ PIBT sang EPIBT) mà không cần build lại ứng dụng Unity, đồng thời tận dụng hiệu năng C++ cho phần tính toán nặng thay vì chạy toàn bộ trong C#/IL2CPP.

4.5.6 Tích hợp liên tục và triển khai liên tục (CI/CD)

Quy trình CI/CD được xây dựng trên GitHub Actions, chia thành bốn workflow tách biệt nhằm dễ dàng gỡ lỗi và kiểm soát quyền truy cập. Bảng 4.14 liệt kê các

workflow cùng chức năng tương ứng.

Bảng 4.14: Các workflow CI/CD trên GitHub Actions

Workflow	Chức năng	Kích hoạt
ci-validate	Kiểm tra định dạng Terraform, cú pháp Python (Registry), cú pháp shell script; phát hiện tệp nhảy cảm bị commit	Push, Pull Request
terraform-plan	Xác thực cấu hình hạ tầng, chạy terraform plan để xem trước thay đổi	Push, Pull Request
unity-build	Build artifact Linux Dedicated Server và WebGL, đóng gói theo mã commit, upload lên GCS bucket	Push, Manual
deploy-gcp	Triển khai artifact lên máy ảo GCP, khởi động lại dịch vụ, kiểm tra sức khỏe (smoke test)	Manual (có phê duyệt)

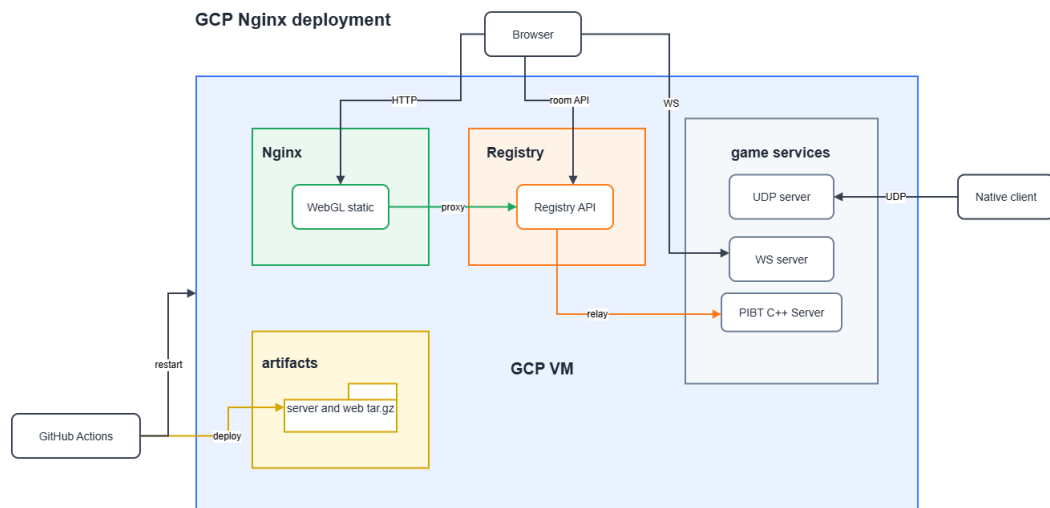
Xác thực với GCP sử dụng cơ chế Workload Identity Federation qua giao thức OpenID Connect (OIDC): GitHub Actions nhận mã thông báo (token) tạm thời từ GCP mà không cần lưu trữ khóa JSON của tài khoản dịch vụ trong kho mã nguồn, giảm thiểu rủi ro rò rỉ bí mật. Quy trình triển khai production yêu cầu phê duyệt thủ công (manual approval) qua cơ chế GitHub Environment, đảm bảo không có thay đổi hạ tầng ngoài kiểm soát.

Luồng CI/CD từ khi đẩy mã đến khi triển khai diễn ra như sau: lập trình viên đẩy mã lên nhánh `networking-gcp`, các workflow tự động kiểm tra định dạng và chạy `terraform plan`; nếu cần triển khai, lập trình viên kích hoạt `unity-build` để tạo artifact theo mã commit, sau đó kích hoạt `deploy-gcp` với mã commit tương ứng; workflow tải artifact từ GCS, giải nén trên máy ảo, khởi động lại các dịch vụ systemd (`tank-mapf-server`, `tank-mapf-server-web`, `tank-mapf-registry`), đồng thời cập nhật lại cấu hình Nginx và chạy bước kiểm tra sức khỏe tự động sau triển khai.

4.5.7 Sơ đồ triển khai tổng thể

Hình 4.28 mô tả kiến trúc triển khai toàn bộ hệ thống trên môi trường production.

Theo sơ đồ trong Hình 4.28, luồng hoạt động diễn ra qua bốn bước khi người dùng truy cập trang <https://luminx.io.vn>. Bước một, trình duyệt tải bản WebGL tĩnh (các tệp `.js.br`, `.wasm.br`, `.data.br`) từ Nginx qua HTTPS; sau khi ứng dụng WebGL khởi động, người dùng tạo hoặc tham gia phòng thông qua các



Hình 4.28: Sơ đồ triển khai hệ thống trên GCP với Nginx, Registry và các dịch vụ trò chơi

lệnh gọi API tới dịch vụ Registry (proxy qua Nginx); khi trận đấu bắt đầu, trình duyệt mở kết nối wss://game.luminx.io.vn tới máy chủ trò chơi (Nginx terminate TLS rồi chuyển tiếp WebSocket tới cổng nội bộ 7778); nếu thuật toán được chọn là PIBT TCP, máy chủ trò chơi giao tiếp với tiến trình PIBT C++ qua TCP nội bộ để lấy hành động cho các agent AI.

Bản chạy gốc (native client, chạy trên máy tính) có thể kết nối trực tiếp tới máy chủ trò chơi bằng giao thức UDP trên cổng 7777, bỏ qua lớp WebSocket và Nginx, mang lại độ trễ thấp hơn so với bản WebGL. Riêng chế độ backtest không khả dụng trên bản WebGL do trình duyệt không hỗ trợ truy xuất hệ thống tệp để ghi kết quả CSV.

Kết chương 4: Chương này đã trình bày trọn vẹn quá trình từ thiết kế đến đánh giá hệ thống: kiến trúc phân tầng tách biệt lõi MAPF khỏi giao diện và vật lý, thiết kế chi tiết ba biến thể agent dùng chung TankController, quá trình xây dựng ứng dụng kèm thống kê mã nguồn và minh họa đầy đủ các chức năng, đánh giá định lượng trên 900 run với năm mức agent ở cả hai môi trường, và cuối cùng là triển khai thực tế lên Google Cloud Platform. Kết quả cho thấy ba thuật toán có thể mạnh khác nhau và bộc lộ rõ giới hạn khi mật độ agent tăng – là cơ sở cho các đóng góp kỹ thuật được trình bày sâu hơn ở Chương 5.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương 4 đã trình bày tổng quan thiết kế kiến trúc và kết quả thực nghiệm. Chương này đi sâu vào sáu đóng góp kỹ thuật cốt lõi của đề án, mỗi mục trình bày rõ bài toán, giải pháp và kết quả đạt được.

5.1 Hệ thống đọc bản đồ động và dựng lưới thống nhất

Bài toán

Game thường bake cứng bản đồ vào scene Unity, khiến việc thay đổi bản đồ đòi hỏi mở lại scene và chỉnh sửa thủ công. Khi cần chạy backtest trên 5 bản đồ khác nhau với kịch bản MAPF, cách tiếp cận này không khả thi.

Giải pháp

Lớp MapLoader đọc tệp .map theo chuẩn MovingAI tại thời điểm chạy (runtime): phân tích header (height, width) và ma trận ký tự (. = ô đi được, @ = tường), sau đó tạo GameObject cho mỗi ô theo đúng tọa độ. Ô tường nhận BoxCollider2D trên layer ObstaclesMovement (chặn vật lý) và trigger collider trên layer Hittable (nhận đạn). Bốn tường biên được tạo tự động bao quanh bản đồ.

Hệ thống expose ba API thống nhất mà mọi thuật toán MAPF và spawner đều dùng: `IsWalkable(cell)`, `CellToWorld(cell)`, `WorldToCell(pos)`. Tọa độ ô dùng quy ước góc trên-trái, Y tăng xuống dưới (khớp với hàng trong tệp .map); tọa độ thế giới Unity tâm ở (0, 0), trục XY.

Kết quả

Chuyển đổi bản đồ trong backtest chỉ tốn vài mili-giây (không cần reload scene). Cùng một bản đồ .map chạy được trên cả ba thuật toán mà không cần chỉnh sửa. Hệ thống đã xử lý thành công 5 bản đồ kích thước từ 32×32 đến 251×180 ô, bao gồm cả bản đồ mê cung 128×128 với 14 818 ô đi được.

5.2 Triển khai A* thời gian thực với tái hoạch định định kỳ

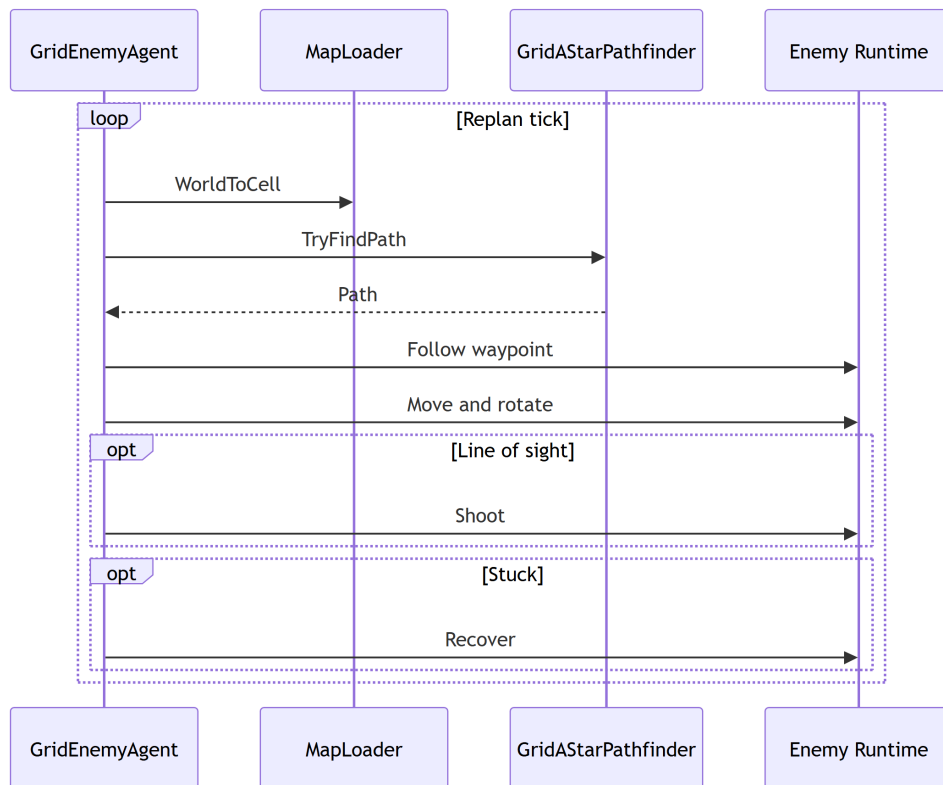
Bài toán

A* truyền thống tính một đường đi tĩnh từ điểm đầu đến đích. Trong game thời gian thực, đích có thể thay đổi (Eagle di chuyển hoặc bị che khuất), bản đồ có thể thay đổi (thùng cản mới xuất hiện), và agent bị cản bởi agent khác. Cần cơ chế tái hoạch định tự động mà không làm giật game.

Giải pháp

GridAStarPathfinder thực thi A* 4 láng giềng với heuristic Manhattan trên ma trận `char[][]`, tái sử dụng `List<Vector2Int>` để tránh garbage collection. GridEnemyAgent quản lý vòng lặp điều hướng: tái hoạch định sau mỗi replan Interval giây (mặc định), chuyển sang chế độ bắn khi Eagle trong tầm và line-of-sight, phục hồi khi bị kẹt quá 2 giây.

Ngưỡng tiếp cận waypoint dùng dot-product 0.96 (thay vì khoảng cách Euclidean) để tránh rung lắc ở các waypoint gần: agent chỉ chuyển sang waypoint tiếp theo khi hướng di chuyển gần như song song với hướng đến waypoint. Hình 5.1 mô tả luồng tương tác giữa agent, bộ tìm đường và bộ điều khiển trong một chu kỳ tái hoạch định.



Hình 5.1: Biểu đồ trình tự chu kỳ tái hoạch định của A*

Kết quả

Thực nghiệm trên Alpha32 (90% ô đi được, 6 agent): thời gian hoàn thành trung bình 25,1 giây, 144 lần replan. Thuật toán chạy ổn định 6 lần lặp với độ lệch nhỏ ($< 0,5$ giây), xác nhận tính tất định của A* trên cùng bản đồ.

5.3 Tích hợp PIBT phối hợp đa agent không va chạm

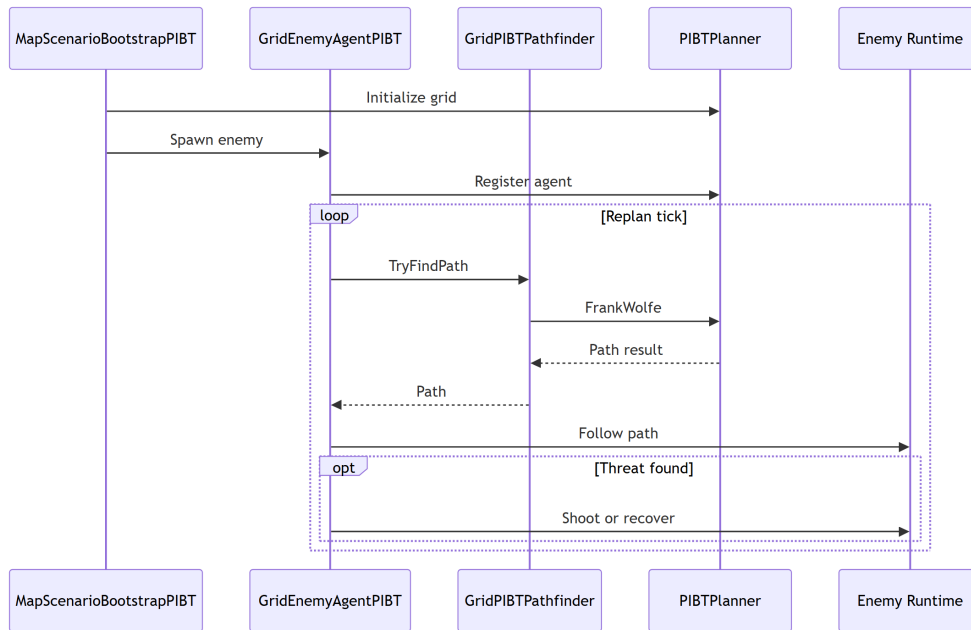
Bài toán

Khi nhiều agent A* chạy độc lập, chúng thường cố tranh vào cùng một ô tại cùng thời điểm, gây va chạm ở lớp vật lý và replan phản ứng liên tục. Cần cơ chế phối hợp ở lớp lập kế hoạch để loại bỏ xung đột từ gốc.

Giải pháp

MapScenarioBootstrapPIBT tạo một bộ giải PIBT dùng chung cho tất cả agent. Tại mỗi tick lập kế hoạch, bộ giải nhận vị trí hiện tại của tất cả agent, chạy một bước PIBT, và trả về ô di chuyển tiếp theo cho từng agent. Agent nhận lệnh di chuyển qua GridEnemyAgentPIBT.SetNextWaypoint() và thực thi qua TankController – cùng pipeline với A*.

Cơ chế kế thừa ưu tiên đảm bảo nếu agent ưu tiên thấp bị agent ưu tiên cao chặn ô, nó tạm nhường ô đó và chờ hoặc tìm ô thay thế. Nếu không có ô nào khả dụng (deadlock cục bộ), thuật toán hoàn tác (backtracking) lên agent ưu tiên cao để thử phương án khác. Hình 5.2 mô tả luồng một tick lập kế hoạch của bộ giải PIBT dùng chung.



Hình 5.2: Biểu đồ trình tự một tick lập kế hoạch của PIBT C#

Kết quả

PIBT C# hoàn thành nhiệm vụ trên 4/5 bản đồ với thời gian tương đương A* (chênh lệch < 10%). Số lần replan chỉ nhỉnh hơn A*: cả hai cùng tái hoạch định định kỳ mỗi 0,75 giây, PIBT C# cộng thêm các lần replan khẩn cấp khi thoát kẹt cục bộ. Ở mật độ sáu agent, cơ chế thoát kẹt hầu như không kích hoạt (số lần phục

hồi bằng 0) nên con số không bùng nổ như lý thuyết dự đoán mà bám sát A^* . Nhờ phối hợp ở lớp lập kế hoạch, PIBT giảm được va chạm vật lý so với khi chạy nhiều A^* độc lập, đồng thời dùng cơ chế thoát kẹt nhiều hơn khi có chương ngại động.

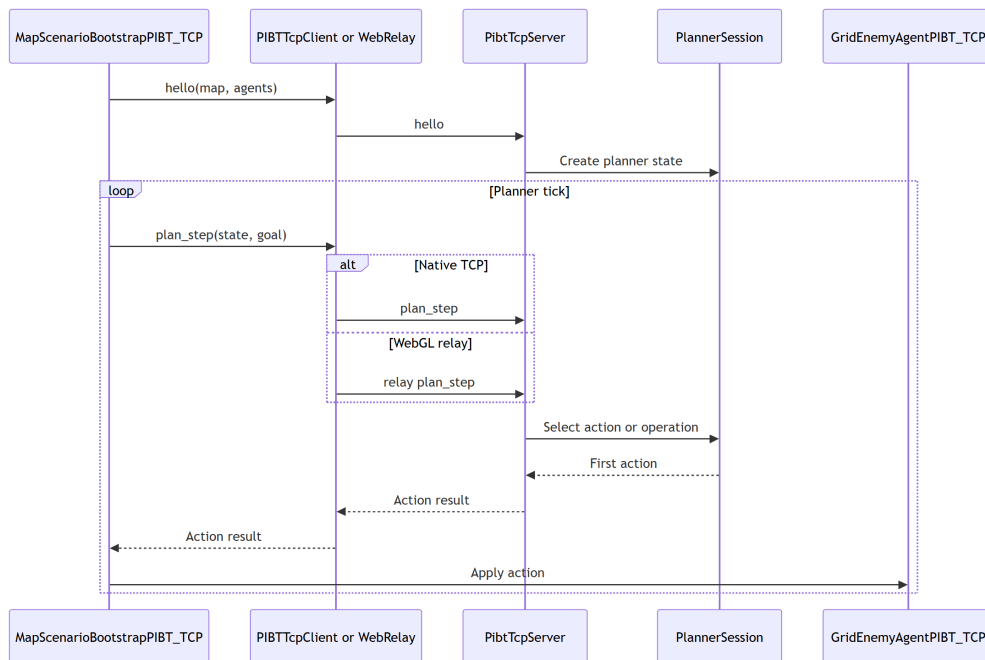
5.4 Cầu nối TCP đến máy chủ PIBT C++

Bài toán

Thuật toán PIBT chuẩn được triển khai tối ưu bằng C++. Việc port toàn bộ sang C# sẽ mất nhiều công sức và dễ mất đi tối ưu hóa. Cần cách tích hợp thuật toán C++ vào Unity mà không sửa engine game.

Giải pháp

Giao thức TCP nội bộ tại localhost cổng 7777 sử dụng ba loại thông điệp JSON. Thông điệp Hello được Unity gửi kèm kích thước bản đồ và số agent để server khởi tạo bộ lập kế hoạch và xác nhận. Thông điệp PlanStep được Unity gửi kèm vị trí tất cả agent ở mỗi bước, và server chạy một bước PIBT rồi trả về ô mục tiêu cho từng agent. Thông điệp Shutdown kết thúc phiên khi ván chơi kết thúc. Phía Unity, lớp PIBTTcpClient quản lý kết nối bất đồng bộ, timeout và tự động kết nối lại, còn GridEnemyAgentPIBT_TCP nhận lệnh từ client và điều khiển TankController như hai biến thể kia. Hình 5.3 mô tả luồng trao đổi thông điệp giữa Unity và server C++.



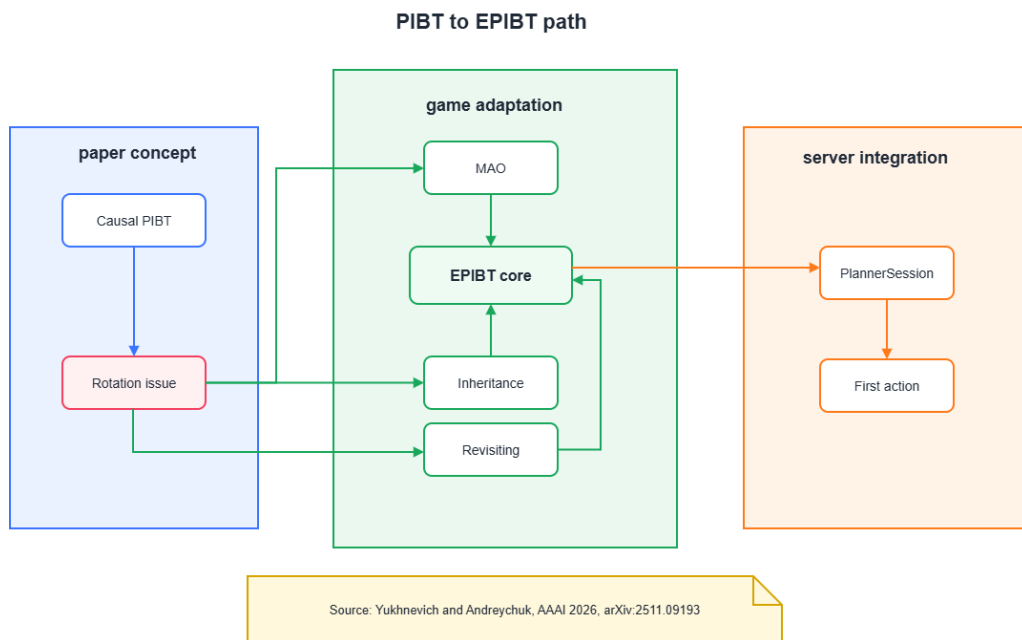
Hình 5.3: Biểu đồ trình tự trao đổi thông điệp PIBT qua TCP

Kết quả

PIBT C++/TCP hoàn thành nhiệm vụ trên cùng 4/5 bản đồ với thời gian tương đương (chênh lệch < 5% so với PIBT C#). Agent TCP đi qua nhiều ô hơn rõ rệt (khoảng 50–130%) do độ trễ TCP (một tick) khiến agent đôi khi chờ lệnh; đây là chi phí của kiến trúc client–server mà có thể giảm bằng cách tăng tần suất tick. Kết nối TCP ổn định trong suốt quá trình thực nghiệm mà không gặp lỗi mất kết nối.

5.5 Cải tiến mô hình xoay của Enemy Agent dựa trên EPIBT

Bộ giải PIBT (cả bản C# ở Mục 5.3 lẫn bản C++ qua TCP ở Mục 5.4) coi mỗi agent như một chất điểm vô hướng: tại mỗi bước, agent được giả định có thể bước thẳng sang một ô kề bất kỳ. Tank trong game lại có *hướng thân* và phải xoay trước khi tiến, nên giả định này không còn đúng. Mục này trình bày việc nâng cấp máy chủ C++ từ PIBT sang EPIBT để xử lý ràng buộc xoay hướng đó. Hình 5.4 tóm tắt lộ trình cải tiến: từ khái niệm trong bài báo, qua phần thích ứng cho game, đến phân tích hợp vào máy chủ.



Hình 5.4: Lộ trình chuyển từ PIBT sang EPIBT cho Enemy Agent: khái niệm trong bài báo (vấn đề xoay hướng), phần thích ứng cho game (multi-action operation, kế thừa operation) và phân tích hợp máy chủ (phiên lập kế hoạch, trả về hành động đầu tiên)

Bài toán

PIBT một bước (1-step) giả định rằng agent ưu tiên cao luôn có thể tiến gần đích hơn vì agent đang chặn ô sẽ rời đi ngay ở bước kế tiếp. Với mô hình có xoay hướng, giả định này đổ vỡ: để sang một ô bên cạnh, tank phải xoay thân rồi mới tiến nên một nước đi thực chất tốn hai đến ba bước. PIBT không ”nhìn thấy” các bước xoay này, khiến agent đang chặn chưa kịp rời ô thì chuỗi kế thừa ưu tiên đã thất bại. Hệ

quả quan sát được trong game là tank xoay giật 90° tại chỗ, dừng khựng giữa đường hoặc kẹt vì quay sai hướng. Causal PIBT diễn giải mỗi lần xoay như một độ trễ tất định, nhưng vẫn bị giới hạn bởi tầm nhìn một bước nên chưa giải quyết triệt để vấn đề.

Giải pháp

Đề án áp dụng EPIBT (Enhanced PIBT) [8] – biến thể đa hành động của PIBT mà chính nhóm tác giả đã dùng để giành ngôi vô địch cuộc thi League of Robot Runners 2024 [5]. Bộ giải C++ ở Mục 5.4 được nâng cấp với ba thành phần, tương ứng nhóm *thích ứng cho game* trong Hình 5.4.

Mô hình hành động có xoay. Trạng thái của mỗi agent gồm vị trí và hướng (Đông, Nam, Tây, Bắc), cùng bốn hành động: FW (tiền), CR (xoay phải 90°), CCR (xoay trái 90°) và W (chờ). Giao thức `plan_step` đã thiết kế ở Mục 5.4 vốn gửi sẵn trường `orientation` và `goalLoc` cho từng agent, nên không cần đổi định dạng thông điệp.

Multi-action Operation (MAO). Thay vì chọn một hành động đơn lẻ, mỗi agent chọn một *operation* – chuỗi hành động có độ dài cố định $op_len = 3$ (ví dụ CRFWW: xoay phải, tiến, chờ). Độ dài 3 là tối thiểu để từ một hướng bất kỳ vẫn tới được mọi ô kè. Máy chủ dựng sẵn tập $4^3 = 64$ operation ứng viên, mở rộng mỗi operation thành một chuỗi bốn trạng thái rồi loại các operation đi ra ngoài bản đồ hoặc đâm vào tường. Nhờ nhìn trước ba bước, agent thấy trước được chi phí xoay mà PIBT một bước bỏ sót.

Kế thừa operation (Operation Inheritance). Operation đã chọn ở bước trước vốn đã không va chạm trong ba bước, nên ở bước sau nó được tái sử dụng bằng cách bỏ hành động đầu và nối thêm một W ở cuối. Khi không tìm được operation nào tốt hơn, agent giữ lại operation kế thừa thay vì đứng yên hoàn toàn – đây là điểm khác biệt then chốt so với PIBT gốc, vốn buộc agent chờ khi chuỗi kế thừa ưu tiên thất bại.

Bộ giải vẫn dùng bảng đặt chỗ (reservation table) để kiểm tra cả xung đột đỉnh (vertex conflict) lẫn xung đột hoán đổi cạnh (edge-swap conflict) theo từng bước; operation gây va chạm với hơn một agent bị loại để tránh nhập nhằng khi kế thừa ưu tiên. Mỗi operation được chấm điểm theo khoảng cách heuristic tới đích, cộng thưởng cho phần tiến gần đích và phạt cho số lần xoay, chờ – nhờ đó agent luôn ưu tiên tiến hơn xoay, xoay hơn chờ, tránh tự rơi vào trạng thái đứng yên vô hạn.

Tích hợp máy chủ – Unity

EPIBT được thêm vào dưới dạng một mô-đun riêng và bật bằng cờ cấu hình (biến môi trường `PIBT_TCP_PLANNER=epibt` hoặc trường `plannerMode` trong thông điệp `hello`), trong khi bộ lập kế hoạch PIBT mặc định vẫn được giữ để có thể quay lui (rollback) khi cần. Toàn bộ trạng thái xuyên suốt các bước – operation kế thừa, tập operation ứng viên và bảng heuristic – được lưu trong một *phiên lập kế hoạch* (`PlannerSession`), mỗi kết nối Unity ứng với một phiên; đây chính là lý do vòng đời `hello` → `plan_step` → `shutdown` ở Mục 5.4 là cần thiết. Theo nguyên tắc tái hoạch định liên tục, mỗi bước máy chủ chỉ trả về *hành động đầu tiên* của operation từng agent; phía Unity, lớp `GridEnemyAgentPIBT_TCP` nhận ô mục tiêu tương ứng và lái tank tron tru tới đó bằng cơ chế căn hướng nhiều ngưỡng (đi thẳng khi đã thẳng hàng, vừa xoay vừa tiến khi lệch ít, xoay tại chỗ khi lệch nhiều) thay cho kiểu xoay giật 90° .

Bản EPIBT đầy đủ còn có cơ chế xét lại agent (revisiting, giới hạn L lần) cùng hai mở rộng Large Neighborhood Search và Graph Guidance. Trong phạm vi đề án, do số lượng agent trong game nhỏ (tối đa 72) và mục tiêu khử hiện tượng xoay giật đã đạt, các cơ chế này được giữ ở mức tối thiểu (giới hạn revisiting đặt bằng 0) và xem là hướng phát triển ở Chương 6. Toàn bộ bước lập kế hoạch luôn được máy chủ xử lý và trả về trong dưới 1 giây mỗi bước, đáp ứng yêu cầu phi chức năng ở Mục 2.4.

Kết quả

Sau cải tiến, Enemy Agent xoay liên tục rồi tiến thay vì bật 90° tại chỗ, loại bỏ hiện tượng giật và kẹt do quay sai hướng quan sát thấy ở bản PIBT thuần. Nhờ cờ cấu hình, có thể chuyển qua lại giữa PIBT và EPIBT để so sánh A/B và quay lui an toàn mà không cần build lại ứng dụng Unity. Với số agent của game (tối đa 72), máy chủ trả về kết quả lập kế hoạch mỗi bước trong dưới 1 giây; do lời gọi TCP chạy bất đồng bộ ngoài luồng kết xuất nên không gây giật khung hình. Đây cũng chính là máy chủ C++ đứng sau cấp thuật toán "PIBT C++/TCP" được đánh giá ở Chương 4; mục này tài liệu hóa phần nâng cấp xử lý xoay hướng bên dưới các thực nghiệm đó.

5.6 Hệ thống backtest tự động với chương ngại động

Bài toán

Để so sánh thuật toán một cách khách quan, cần chạy hàng chục kịch bản lặp lại trên nhiều bản đồ và thuật toán, thu thập chỉ số nhất quán, và kiểm tra khả năng của hệ thống khi môi trường thay đổi trong khi agent đang di chuyển. Chạy thủ công từng kịch bản là không thực tế và không tái lập được.

Giải pháp

BacktestRunner tự động lặp qua toàn bộ ma trận tổ hợp ($\text{map} \times \text{algorithm} \times \text{số lượng agent} \times \text{rep}$) theo lịch trình định sẵn. Mỗi run: reset scene, spawn agent, chạy kịch bản, thu thập chỉ số ở cấp run (summary) và cấp agent (per-agent CSV). Sau khi hết run, gọi script Python để sinh biểu đồ HTML/PNG từ CSV.

DynamicObstacleSpawner bổ sung chế độ stress-test: thùng kim loại tối màu liên tục spawn/despawn trực tiếp trên 1–6 ô phía trước đường đi của agent (ưu tiên chặn ngay ô tiếp theo), ép agent phải tái hoạch định ngay lập tức. MapLoader. MarkCellBlocked/UnmarkCellBlocked cập nhật ma trận grid đồng thời để cả ba thuật toán đều nhận được cập nhật nhất quán.

Kết quả

Hệ thống cho phép quét toàn bộ ma trận thực nghiệm gồm 5 bản đồ $\times$ 3 thuật toán $\times$ 5 mức số lượng agent (6, 12, 24, 36, 72) $\times$ 6 lần lặp $\times$ 2 môi trường, tương ứng 900 run, thu thập chỉ số nhất quán ở cả cấp run và cấp agent. Nhờ trực khảo sát theo số lượng agent, hệ thống không chỉ so sánh ba thuật toán ở một mức tải mà còn đánh giá được khả năng mở rộng của chúng khi tải tăng dần (chi tiết ở Chương 4). Khi bật chương ngại động, cả ba thuật toán đều phản ứng được với thay đổi môi trường thời gian thực; số lần replan và số lần phục hồi sau kẹt đều tăng, trong đó PIBT C# phục hồi nhiều nhất nhờ cơ chế thoát kẹt cục bộ.

Điểm mới của đề án không nằm ở việc phát minh thuật toán mới, mà ở việc hợp nhất ba cấp độ MAPF – từ baseline A* tới PIBT phối hợp và PIBT/EPIBT hiệu năng cao – vào cùng một game có vật lý thực tế, dưới chung một tầng điều khiển và một hệ thống đo lường. Cách tiếp cận này đưa các thuật toán vốn chỉ được đánh giá trên benchmark trừu tượng vào một môi trường game có thể chơi, quan sát và so sánh định lượng trực tiếp – điều hiếm thấy trong các nghiên cứu đã khảo sát.

Kết chương 5: Sáu đóng góp kỹ thuật của đề án – từ hạ tầng bản đồ động, ba cấp độ thuật toán MAPF và cải tiến mô hình xoay hướng bằng EPIBT, đến hệ thống backtest – tạo thành một nền tảng hoàn chỉnh để nghiên cứu và so sánh thuật toán MAPF trong môi trường game thực tế. Kết quả thực nghiệm được trình bày chi tiết trong Chương 4.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Đồ án đã nghiên cứu và triển khai thành công bài toán tìm đường đa tác nhân (MAPF) trong game bắn xe tăng 2D nhiều người chơi xây dựng trên Unity Engine. Ba cấp độ thuật toán – A* baseline, PIBT C# và PIBT C++/TCP – được triển khai hoàn chỉnh, chia sẻ cùng tầng điều khiển vật lý và được đánh giá định lượng trên tập bản đồ chuẩn MovingAI MAPF benchmark.

Về các kết quả chính, đồ án đã xây dựng được hệ thống đọc và dựng bản đồ động từ tệp .map tại thời điểm chạy, hỗ trợ năm bản đồ kích thước từ 32×32 đến 251×180 mà không cần chỉnh sửa mã nguồn. Thuật toán A* baseline hoàn thành nhiệm vụ trên bốn trong năm bản đồ với thời gian ổn định, trong đó Maze128 là giới hạn chung của cả ba thuật toán do đường đi dài và phức tạp. Hai biến thể PIBT C# và PIBT C++/TCP thể hiện khả năng phối hợp đa agent không va chạm với thời gian hoàn thành tương đương A* trên bốn trong năm bản đồ, đồng thời giảm được tình trạng deadlock ở lớp vật lý. Trên nền PIBT C++/TCP, đồ án còn cải tiến mô hình xoay của Enemy Agent dựa trên EPIBT, loại bỏ hiện tượng xe tăng xoay giật 90° tại chỗ. Cuối cùng, hệ thống backtest tự động chạy ổn định 900 run (450 tĩnh và 450 động) trên năm mức agent từ 6 đến 72, xuất kết quả CSV và biểu đồ; kết quả xác nhận cả ba thuật toán duy trì được nhiệm vụ khi môi trường có chướng ngại động, đồng thời cho thấy tỉ lệ phá Eagle đạt đỉnh ở dải mật độ trung gian (24–36 agent) chứ không suy giảm đơn điệu theo số agent.

So với mục tiêu đề ra tại Mục 1.2, cả bốn mục tiêu đều đã hoàn thành. So với các giải pháp tương tự, đóng góp của đồ án nằm ở việc tích hợp và so sánh thực nghiệm ba cấp độ MAPF trong cùng một môi trường game có vật lý thực tế – điều chưa thấy trong các nghiên cứu được khảo sát.

6.2 Hạn chế của đồ án

Bên cạnh các kết quả đạt được, đồ án còn một số hạn chế. Thứ nhất, quy mô thực nghiệm dừng ở 72 agent trên năm bản đồ – nhỏ hơn nhiều so với quy mô hàng nghìn agent của các benchmark LMAPF chuyên dụng, nên chưa kiểm chứng được hành vi ở mật độ rất cao. Thứ hai, số lần tái hoạch định được định nghĩa khác nhau giữa ba thuật toán nên hiện chỉ so sánh được *xu hướng* phía máy khách, chưa phản ánh chi phí phối hợp bên trong máy chủ C++; bổ sung trường đếm replan thật từ phía server là cần thiết để so sánh tuyệt đối. Thứ ba, cơ chế xét lại agent cùng hai mở rộng LNS và Graph Guidance của EPIBT hiện được giữ ở mức tối thiểu, nên tiềm năng của chúng trong bối cảnh game chưa được khai thác. Những hạn chế này

đồng thời mở ra các hướng phát triển trình bày dưới đây.

6.3 Hướng phát triển

Trong ngắn hạn, một số việc cần làm để hoàn thiện hệ thống hiện tại. Trước hết là bổ sung trường `replan_count` vào giao thức TCP `plan_result` để server C++ trả về số lần replan thật thay vì phải suy ra từ PIBT C#. Tiếp theo là thêm seed tất định cho từng run để đảm bảo tái lập được hoàn toàn và loại bỏ nhiễu từ quá trình spawn ngẫu nhiên.

Trong dài hạn, đề án có thể được nâng cấp về cả thuật toán lẫn hệ thống. Hướng đầu tiên là tích hợp LNS2 [10], một thuật toán MAPF anytime hiệu năng cao cho phép cải thiện dần chất lượng giải pháp theo thời gian tính toán còn lại. Hướng thứ hai là bổ sung cơ chế phối hợp tấn công theo đội hình, thay cho việc các agent tấn công Eagle một cách độc lập như hiện tại, nhằm tăng hiệu quả và tính hấp dẫn của gameplay. Hướng thứ ba là mở rộng hỗ trợ bản đồ động hoàn toàn, trong đó agent có thể phá hủy tường và thay đổi đồ thị bản đồ trong thời gian thực, đòi hỏi thuật toán MAPF có khả năng tái hoạch định toàn cục. Hướng cuối cùng là triển khai WebGL với PIBT server đặt trên cloud thay vì localhost, mở ra khả năng chạy backtest phân tán và demo trực tuyến không cần cài đặt.

Riêng với nhánh EPIBT, đề án mới khai thác phần lỗi (thao tác đa hành động và kế thừa thao tác) trong khi giữ cơ chế xét lại agent ở mức tối thiểu và chưa bật hai mở rộng Large Neighborhood Search và Graph Guidance. Bật và tinh chỉnh các thành phần này – vốn là yếu tố giúp EPIBT đạt hiệu năng hàng đầu trên benchmark LMAPF – là một hướng nâng cấp tự nhiên nhằm cải thiện thêm khả năng phối hợp và giảm tắc nghẽn ở các bản đồ nhiều hành lang hẹp.

Về khả năng mở rộng quy mô, hệ thống có thể phát triển theo ba trục song song. Trục thứ nhất là số lượng agent: nâng từ vài agent hiện tại lên hàng chục rồi hàng trăm agent như quy mô benchmark của LoRR, đòi hỏi tối ưu bộ giải và truyền thông TCP theo lô. Trục thứ hai là số người chơi đồng thời: mở rộng tầng mạng nhiều người chơi với cơ chế phòng chờ, ghép trận và đồng bộ trạng thái tin cậy khi nhiều người cùng tham gia một trận. Trục thứ ba là hạ tầng: triển khai PIBT server trên cloud có cân bằng tải và khả năng co giãn theo nhu cầu, cho phép phục vụ đồng thời nhiều trận đấu và chạy backtest phân tán khi cả số agent lẫn số người chơi cùng tăng.

TÀI LIỆU THAM KHẢO

- [1] R. Stern et al., “Multi-agent pathfinding: Definitions, variants, and benchmarks,” in *Proceedings of the 12th International Symposium on Combinatorial Search (SoCS)*, 2019, pp. 151–158.
- [2] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [3] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, “Conflict-based search for optimal multi-agent pathfinding,” *Artificial Intelligence*, vol. 219, pp. 40–66, 2015.
- [4] K. Okumura, M. Machida, X. Défago, and Y. Tamura, “Priority inheritance with backtracking for iterative multi-agent path finding,” in *Artificial Intelligence*, vol. 310, Elsevier, 2022, p. 103 752.
- [5] League of Robot Runners, *2024 League of Robot Runners — main round 2024 results*, Team No Man’s Sky: Grand Prize (Combined, Planner, Scheduler Tracks) and Line Honours, 2025. Accessed: Jun. 25, 2026. [Online]. Available: <https://www.leagueofrobotrunners.org/>
- [6] B. De Wilde, A. W. Ter Mors, and C. Witteveen, “Cooperative pathfinding,” in *Proceedings of the 5th International Conference on Agents and Artificial Intelligence (ICAART)*, vol. 2, 2013, pp. 186–194.
- [7] N. R. Sturtevant, “Benchmarks for grid-based pathfinding,” in *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, 2012, pp. 144–148.
- [8] E. Yukhnovich and A. Andreychuk, “Enhancing PIBT via multi-action operations,” *arXiv preprint arXiv:2511.09193*, 2025, Extended version of paper accepted to AAAI-26. [Online]. Available: <https://arxiv.org/abs/2511.09193>
- [9] Unity Technologies, *Unity engine 6 documentation*, 2024. Accessed: Jun. 24, 2026. [Online]. Available: <https://docs.unity3d.com/6000.0/Documentation/Manual/>
- [10] J. Li, Z. Chen, D. Harabor, N. R. Sturtevant, and S. Koenig, “Anytime multi-agent path finding via large neighborhood search,” in *Proceedings of the 30th International Joint Conference on Artificial Intelligence (IJCAI)*, 2021, pp. 4127–4135.

PHỤ LỤC

A. KẾT QUẢ BACKTEST CHI TIẾT

Phụ lục này trình bày kết quả thực nghiệm backtest chi tiết theo từng bản đồ và thuật toán, dưới dạng trung bình kèm độ lệch chuẩn trên sáu lần lặp ($n = 6$). Các số liệu này bổ sung cho bảng tổng hợp trong Chương 4, phục vụ kiểm chứng độ ổn định của từng thuật toán. Cột thời gian tính theo giây, cột replan là tổng số lần tái hoạch định, cột shot là tổng số phát bắn trung bình.

A.1 Kết quả môi trường tĩnh

Bảng A.1 liệt kê kết quả chi tiết trong môi trường tĩnh.

Bảng A.1: Kết quả backtest chi tiết – môi trường tĩnh ($n = 6$)

Bản đồ	Thuật toán	Thời gian (s)	Replan	Shot TB
Alpha32	A*	$25,1 \pm 0,3$	144 ± 5	3 479
Alpha32	PIBT C#	$23,6 \pm 0,5$	196 ± 11	2 327
Alpha32	PIBT C++/TCP	$25,9 \pm 0,5$	177 ± 17	13 734
Mansion	A*	$92,4 \pm 1,1$	530 ± 26	416
Mansion	PIBT C#	$120,5 \pm 1,1$	611 ± 52	83
Mansion	PIBT C++/TCP	$94,4 \pm 2,1$	598 ± 30	1 520
Chantry	A*	$87,2 \pm 1,1$	530 ± 23	646
Chantry	PIBT C#	$110,6 \pm 0,8$	556 ± 41	85
Chantry	PIBT C++/TCP	$92,3 \pm 2,3$	602 ± 61	2 254
Gallows	A*	$106,4 \pm 0,8$	624 ± 19	313
Gallows	PIBT C#	$122,5 \pm 2,0$	621 ± 86	82
Gallows	PIBT C++/TCP	$114,5 \pm 3,2$	640 ± 23	1 288
Maze128	A*	$180,0 \pm 0,0$	820 ± 28	2 076
Maze128	PIBT C#	$180,0 \pm 0,0$	928 ± 39	81
Maze128	PIBT C++/TCP	$180,0 \pm 0,0$	820 ± 8	3 732

Độ lệch chuẩn nhỏ trên hầu hết tổ hợp cho thấy kết quả ổn định qua các lần lặp. Khác với kỳ vọng theo lý thuyết, ba thuật toán có số lần tái hoạch định bám khá sát nhau ở mức sáu agent: hai biến thể PIBT (C# và C++/TCP) chỉ chênh nhau ít và đều nhỉnh hơn A* trên các bản đồ hẹp, *không* xuất hiện hiện tượng PIBT C# đột biến số lần replan lên hàng nghìn lần. Khác biệt giữa hai biến thể PIBT chủ yếu nằm ở số phát bắn và số ô đã đi – phản ánh độ trễ một nhịp của kênh TCP ở biến thể C++/TCP, khiến nó di chuyển và khai hỏa nhiều hơn. Trên bản đồ mê cung Maze128, cả ba thuật toán đều chạm trần thời gian 180 giây ở mức sáu agent: quá ít xe tăng không kịp len qua mê cung lớn để tiếp cận và hạ Eagle. Số liệu chi tiết được tách riêng ở phụ lục thay vì đưa hết vào Chương 4 vì chúng phục vụ kiểm chứng độ ổn định hơn là so sánh tổng quan.

A.2 Kết quả môi trường có chương ngại động

Bảng A.2 liệt kê kết quả chi tiết khi bật chương ngại động.

Bảng A.2: Kết quả backtest chi tiết – môi trường có chương ngại động ($n = 6$)

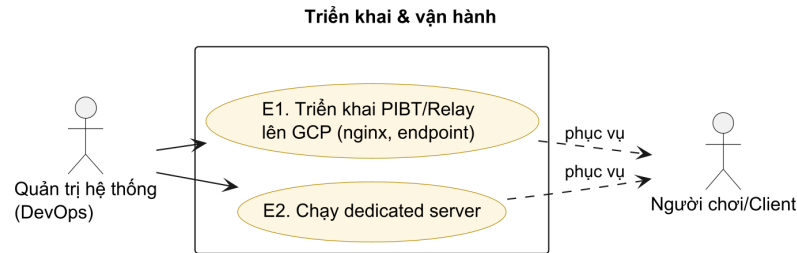
Bản đồ	Thuật toán	Thời gian (s)	Replan	Shot TB
Alpha32	A*	28,3±0,5	165±7	4 090
Alpha32	PIBT C#	28,3±0,8	286±57	2 012
Alpha32	PIBT C++/TCP	30,5±1,1	258±30	14 202
Mansion	A*	96,6±0,7	547±13	400
Mansion	PIBT C#	138,0±3,8	702±88	112
Mansion	PIBT C++/TCP	119,5±4,2	547±34	1 400
Chantry	A*	97,6±0,9	638±44	575
Chantry	PIBT C#	114,9±1,6	592±55	84
Chantry	PIBT C++/TCP	129,5±4,0	938±25	1 940
Gallows	A*	128,1±1,3	742±74	315
Gallows	PIBT C#	177,6±1,5	944±141	80
Gallows	PIBT C++/TCP	130,4±6,8	742±43	1 306
Maze128	A*	180,0±0,0	901±44	1 650
Maze128	PIBT C#	180,0±0,0	977±36	933
Maze128	PIBT C++/TCP	180,0±0,0	1 300±88	7 502

So với môi trường tĩnh, số lần replan tăng trên mọi tổ hợp, xác nhận cả ba thuật toán đều phản ứng với thay đổi môi trường bằng cách tái hoạch định nhiều hơn.

Mức tăng thể hiện trên cả bản đồ thoáng lẫn hẹp: với A* trên Alpha32, số lần tái hoạch định tăng từ 144 lên 165 khi bật chương ngại động; với PIBT C# trên Gallows, con số này tăng từ 621 lên 944. Độ lệch chuẩn của A* vẫn nhỏ, cho thấy thuật toán giữ được tính ổn định ngay cả khi môi trường thay đổi – phần biến thiên chủ yếu đến từ thời điểm và vị trí spawn của chương ngại chứ không phải từ bản thân thuật toán. Bản đồ Maze128 tiếp tục chạm trần 180 giây ở cả ba thuật toán tại mức sáu agent, xác nhận giới hạn tại đây là độ khó tô-pô của bản đồ. Nhìn chung, dữ liệu chi tiết trong hai bảng cũng cố nhận định ở Chương 4: số lần tái hoạch định của ba thuật toán bám sát nhau – PIBT chỉ nhỉnh hơn A* chứ không cao gấp nhiều lần – và khác biệt giữa chúng nói rộng dần khi môi trường trở nên động.

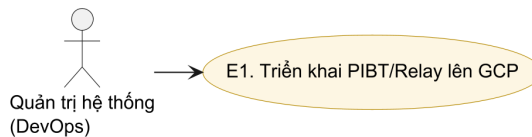
B. NHÓM USE CASE TRIỂN KHAI VÀ VẬN HÀNH

Phụ lục này đặc tả nhóm use case E – triển khai và vận hành máy chủ relay/PIBT trên hạ tầng đám mây, do tác nhân Quản trị hệ thống (DevOps) thực hiện. Nhóm này không thuộc luồng chơi của người dùng cuối nên được tách khỏi Chương 2. Hình B.1 là biểu đồ use case của nhóm.



Hình B.1: Biểu đồ use case nhóm E – Triển khai & vận hành

B.1 Đặc tả use case UC-E1 – Triển khai PIBT/Relay lên GCP

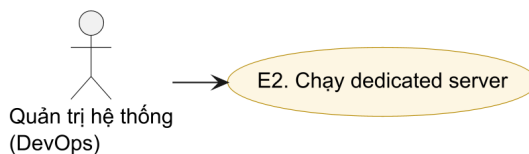


Hình B.2: Use case UC-E1 – Triển khai PIBT/Relay lên GCP

Bảng B.1: Đặc tả use case UC-E1 – Triển khai PIBT/Relay lên GCP

Mã use case	UC-E1
Tên use case	Triển khai máy chủ PIBT/Relay lên GCP
Tác nhân chính	Quản trị hệ thống (DevOps)
Mô tả	Triển khai máy chủ PIBT/relay lên Google Cloud Platform với nginx, cấu hình endpoint và registry để client Internet/WebGL kết nối được.
Tiền điều kiện	Có tài khoản GCP và mã nguồn trong infra/, services/.
Luồng sự kiện chính	1. Build và triển khai service từ infra/ và services/ lên GCP. 2. Cấu hình nginx và endpoint mạng (NetworkEndpointConfig). 3. Đăng ký registry để client tra cứu phòng theo mã.
Luồng thay thế / ngoại lệ	1a. Triển khai thất bại (sai cấu hình/credential) → rollback và xem log.
Hậu điều kiện	Máy chủ PIBT/relay sẵn sàng phục vụ các phòng Internet.

B.2 Đặc tả use case UC-E2 – Chạy dedicated server



Hình B.3: Use case UC-E2 – Chạy dedicated server

Bảng B.2: Đặc tả use case UC-E2 – Chạy dedicated server

Mã use case	UC-E2
Tên use case	Chạy dedicated server
Tác nhân chính	Quản trị hệ thống (DevOps)
Mô tả	Khởi chạy chế độ máy chủ chuyên dụng (headless) để host trận mà không cần client đồ họa.
Tiền điều kiện	Đã triển khai hoặc đã có bản build máy chủ.
Luồng sự kiện chính	1. Chạy ứng dụng với NetworkLaunchArgs ở chế độ server. 2. DedicatedServerBootstrap khởi tạo NetworkManager và lắng nghe kết nối. 3. Client tham gia như UC-B2 hoặc UC-B4.
Luồng thay thế / ngoại lệ	1a. Cổng/endpoint không khả dụng → server thoát kèm log lỗi.
Hậu điều kiện	Dedicated server đang chạy, sẵn sàng nhận client.

B.3 Tóm tắt quy trình triển khai và vận hành

Hai use case UC-E1 và UC-E2 mô tả các thao tác chính của tác nhân DevOps; mục này tổng hợp toàn bộ quy trình triển khai và vận hành thành một luồng xuyên suốt, giúp người đọc hình dung cách hệ thống đi từ mã nguồn tới sản phẩm chạy trực tuyến mà không cần xem demo. Các thành phần hạ tầng cụ thể đã được trình bày ở Mục 4.5; phần này sắp xếp lại chúng theo trục thời gian của quy trình. Bảng B.3 tóm tắt năm giai đoạn, từ lúc lập trình viên đẩy commit cho tới khi trình duyệt người chơi vào được trận.

Cách nhìn theo quy trình này bổ sung cho góc nhìn theo use case ở hai mục trên: trong khi UC-E1 và UC-E2 nhấn mạnh *ai làm gì* (vai trò và thao tác của tác nhân DevOps), bảng quy trình làm rõ *thứ tự và sự phụ thuộc* giữa các bước – một giai đoạn chỉ thực hiện được khi giai đoạn trước đã hoàn tất; chẳng hạn Nginx chỉ phục vụ được trận đấu khi artifact đã nằm trên máy ảo và dịch vụ Registry đã khởi động.

Bảng B.3: Năm giai đoạn của quy trình triển khai end-to-end

Giai đoạn	Thành phần	Hành động chính
1. Khai báo hạ tầng	Terraform (infra/gcp)	terraform apply dựng máy ảo e2-medium, IP tĩnh, firewall và GCS bucket
2. Tích hợp & đóng gói	GitHub Actions (CI/CD)	Build bản máy chủ Linux và bản WebGL nén Brotli theo mã commit, đẩy artifact lên GCS bucket
3. Triển khai (UC-E1)	Máy ảo GCP + Nginx	Tải artifact mới nhất về /opt/tank-mapf/web, nạp lại cấu hình Nginx và dịch vụ Registry
4. Phục vụ	Nginx (ba khối server)	Phục vụ WebGL qua HTTPS, proxy Registry (cổng 8080) và proxy WebSocket trò chơi (cổng 7778) qua wss://
5. Kết nối & chơi	Trình duyệt + Registry + PIBT C++	Người chơi tạo/tham gia phòng bằng mã; lệnh lập kế hoạch được chuyển tới máy chủ PIBT C++ trên cùng máy ảo

Hai giai đoạn đầu diễn ra tự động: mỗi commit lên nhánh chính kích hoạt pipeline GitHub Actions build song song bản máy chủ Linux và bản WebGL, gắn kèm mã commit để truy vết rồi lưu vào GCS bucket. Giai đoạn triển khai (UC-E1) kéo artifact mới nhất về máy ảo và nạp lại cấu hình Nginx cùng dịch vụ Registry. Từ giai đoạn phục vụ trở đi, Nginx là điểm vào duy nhất: nó vừa trả tệp WebGL nén Brotli qua HTTPS, vừa proxy các yêu cầu quản lý phòng tới Registry, vừa chuyển tiếp kết nối WebSocket của trận đấu tới máy chủ trò chơi – nhờ đó các tiến trình nội bộ không phải tự xử lý mã hóa TLS.

Về vận hành, mỗi giai đoạn đều có nhánh xử lý sự cố tương ứng với luồng ngoại lệ của hai use case. Khi triển khai thất bại (UC-E1, nhánh 1a), hệ thống rollback về artifact ổn định trước đó và tra log để xác định nguyên nhân; khi cổng hoặc endpoint không khả dụng (UC-E2, nhánh 1a), dedicated server thoát ngay kèm log lỗi thay vì treo vô thời hạn. Chế độ dedicated server còn cho phép chạy headless để kiểm thử hoặc host trận trong mạng nội bộ mà không cần dựng toàn bộ hạ tầng đám mây. Việc tách Registry và máy chủ trò chơi thành hai tiến trình riêng, kết hợp cơ chế tự gia hạn chứng chỉ TLS bằng systemd timer, giúp hệ thống vận hành liên tục với mức can thiệp thủ công tối thiểu.